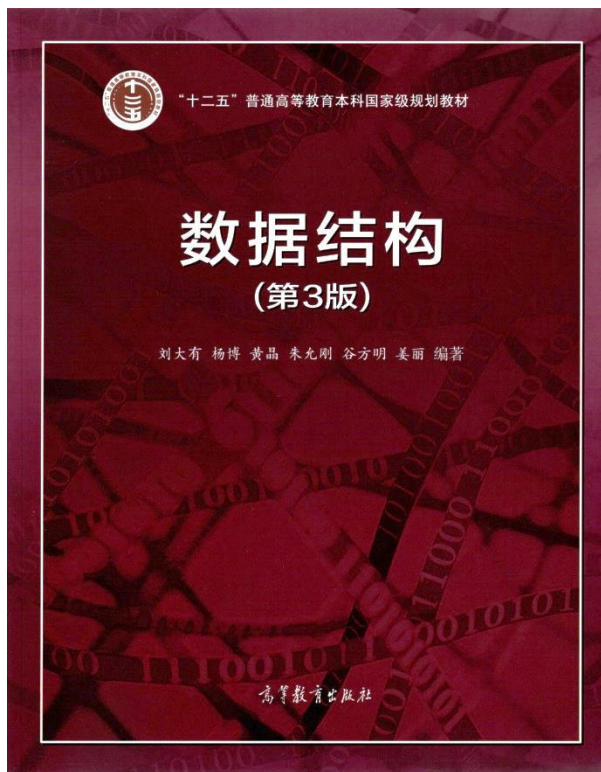


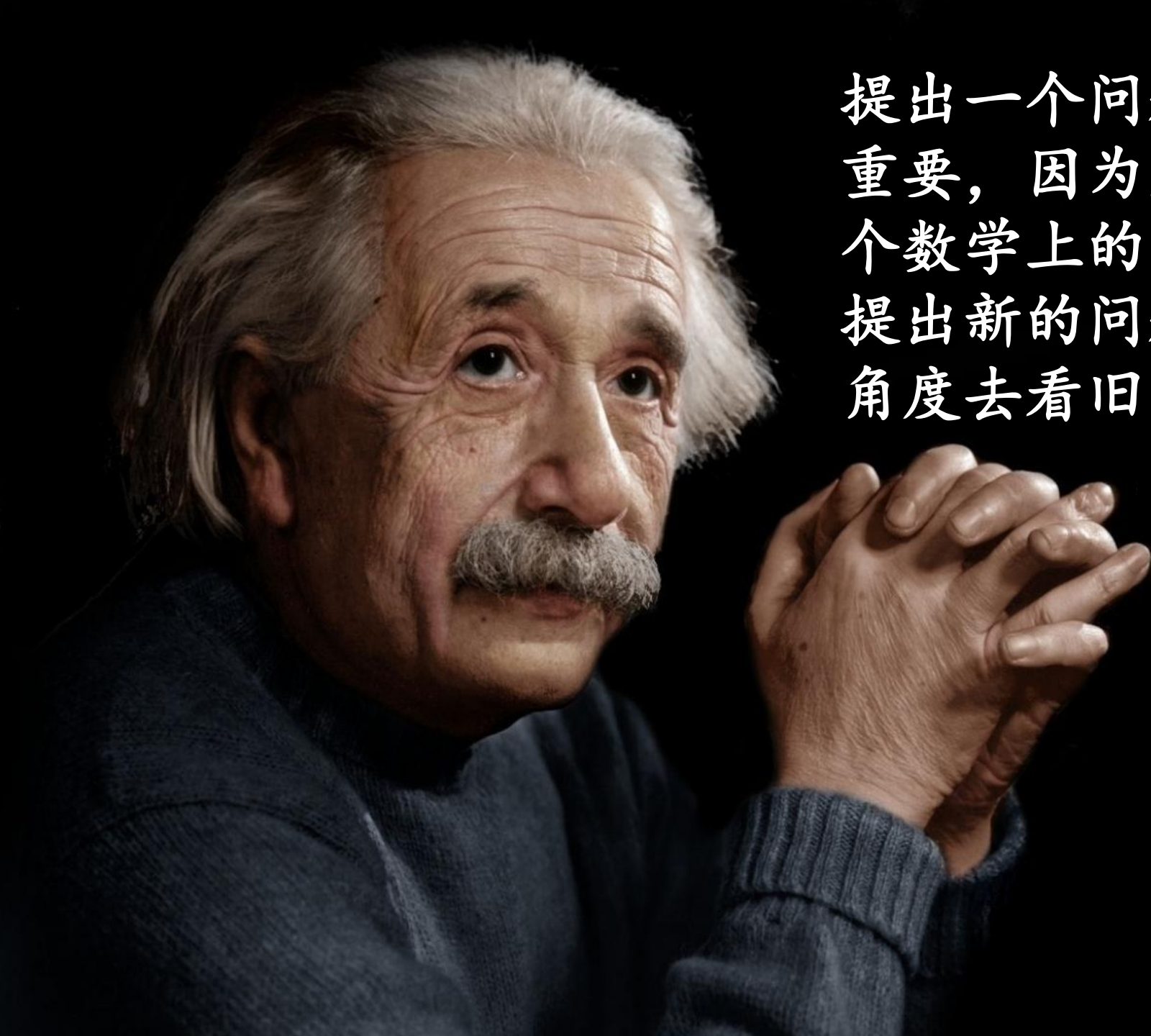


图的最短路径

- 无权图最短路径
- Dijkstra算法
- A*算法初探
- Floyd算法
- 满足约束的最短路径

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn



提出一个问题往往比解决一个问题更重要，因为解决一个问题也许仅是一个数学上的或实验上的技能而已。而提出新的问题，新的可能性，从新的角度去看旧的问题，却需要有创造性的想象力，而且标志着科学的真正进步。

—— 爱因斯坦

最短路径问题

➤ 路径的长度:

- ✓ 无权图: 路径包含的边的条数。
- ✓ 带权图: 路径包含的各边权值之和。

➤ 最短路径: 长度最小的路径称, 两个顶点间最短路径的长度也称两个顶点间的最短距离。

最短路径问题

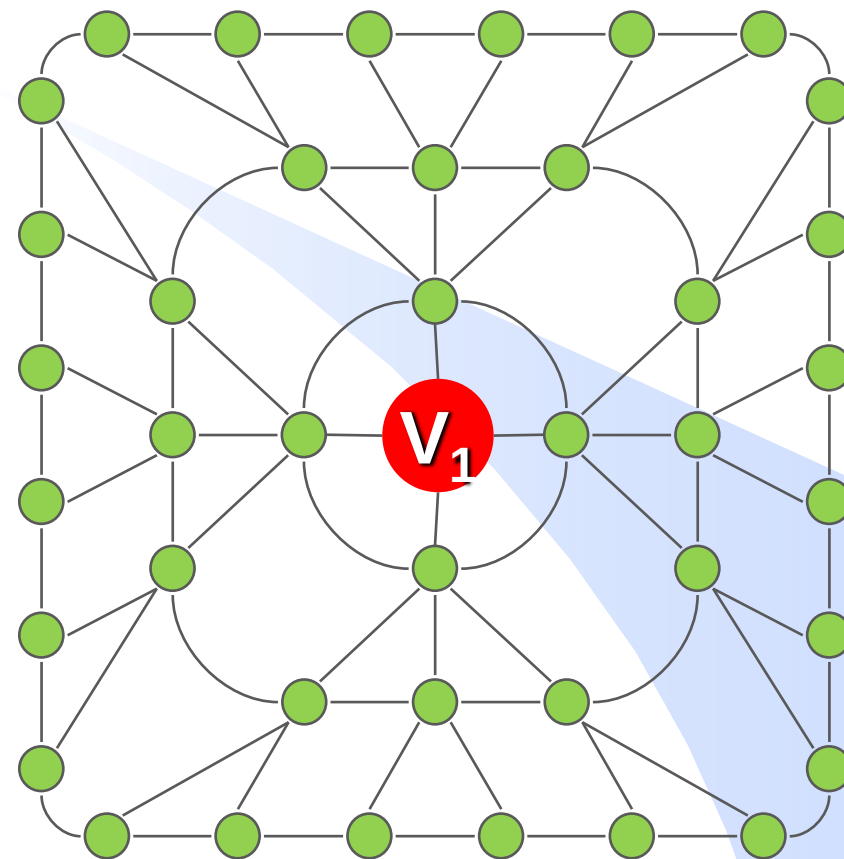
无权图的单
源最短路径
问题

无权图的单源最短路径问题

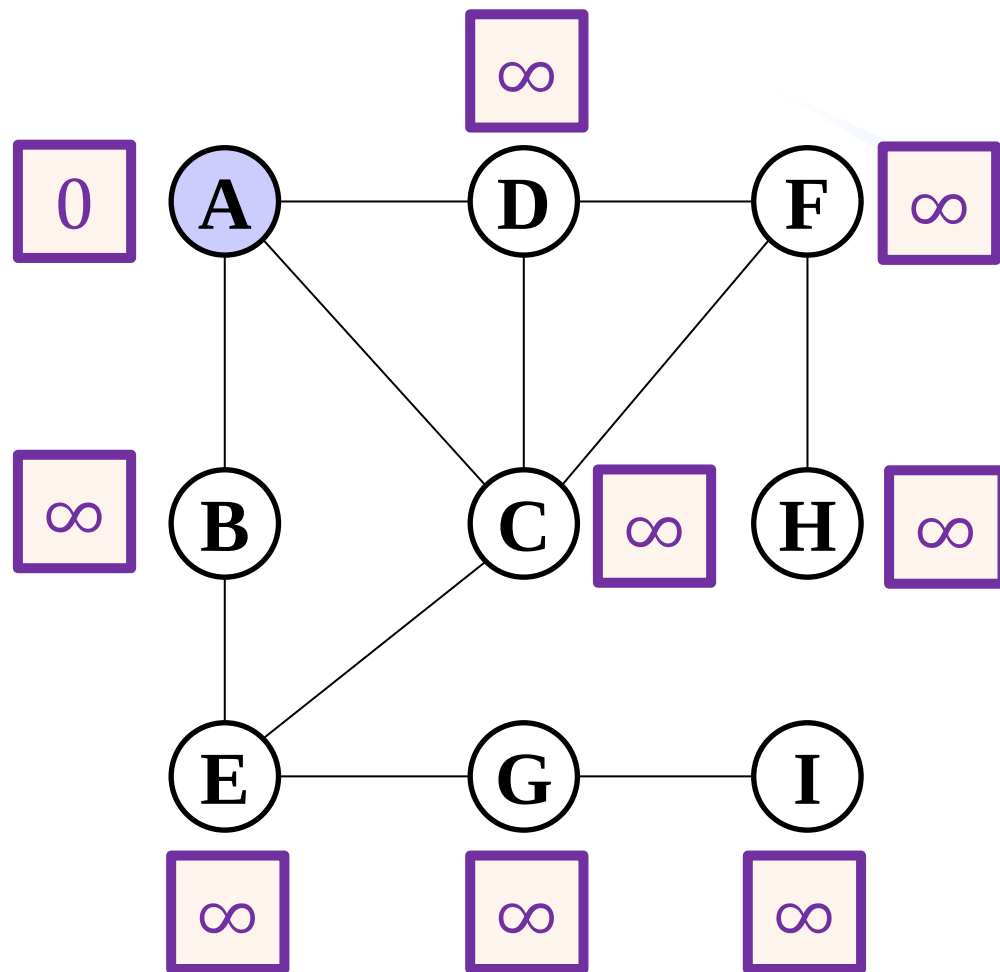
- 单源最短路径：给定一个顶点（称为源点），求该点到图中其它所有顶点的最短路径。
- BFS过程中，当访问某个顶点时，就确定了该点与源点的最短距离。
- 可以通过BFS，从源点开始由近及远求各顶点的最短路径。

回顾：广度优先搜索 *BFS*

- ✓ 首先访问源点 v_1 ;
- ✓ 依次访问 v_1 的邻接顶点 $w_1, \dots, w_k$;
(与 v_1 最短距离为1的点)
- ✓ 然后再依次访问 $w_1, \dots, w_k$ 的 **未访问** 的邻接顶点 (与 v_1 最短距离为2的顶点);
- ✓ 再从这些顶点出发, 依次访问它们的尚未访问的邻接顶点 (与 v_1 最短距离为3的顶点)
- ✓ 依此类推, 直到连通图中的所有顶点全部访问完为止。

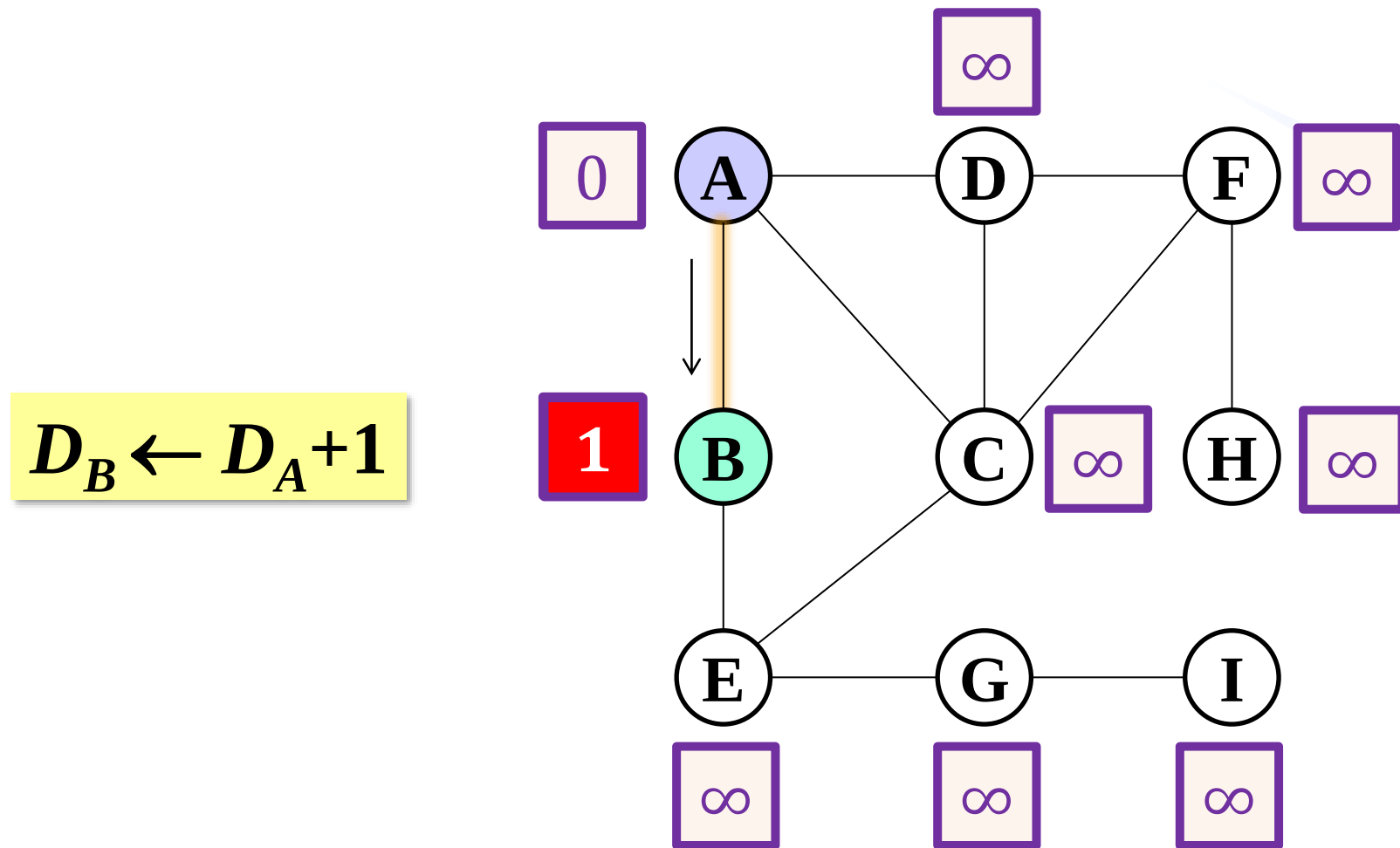


基于BFS的无权图最短路径算法

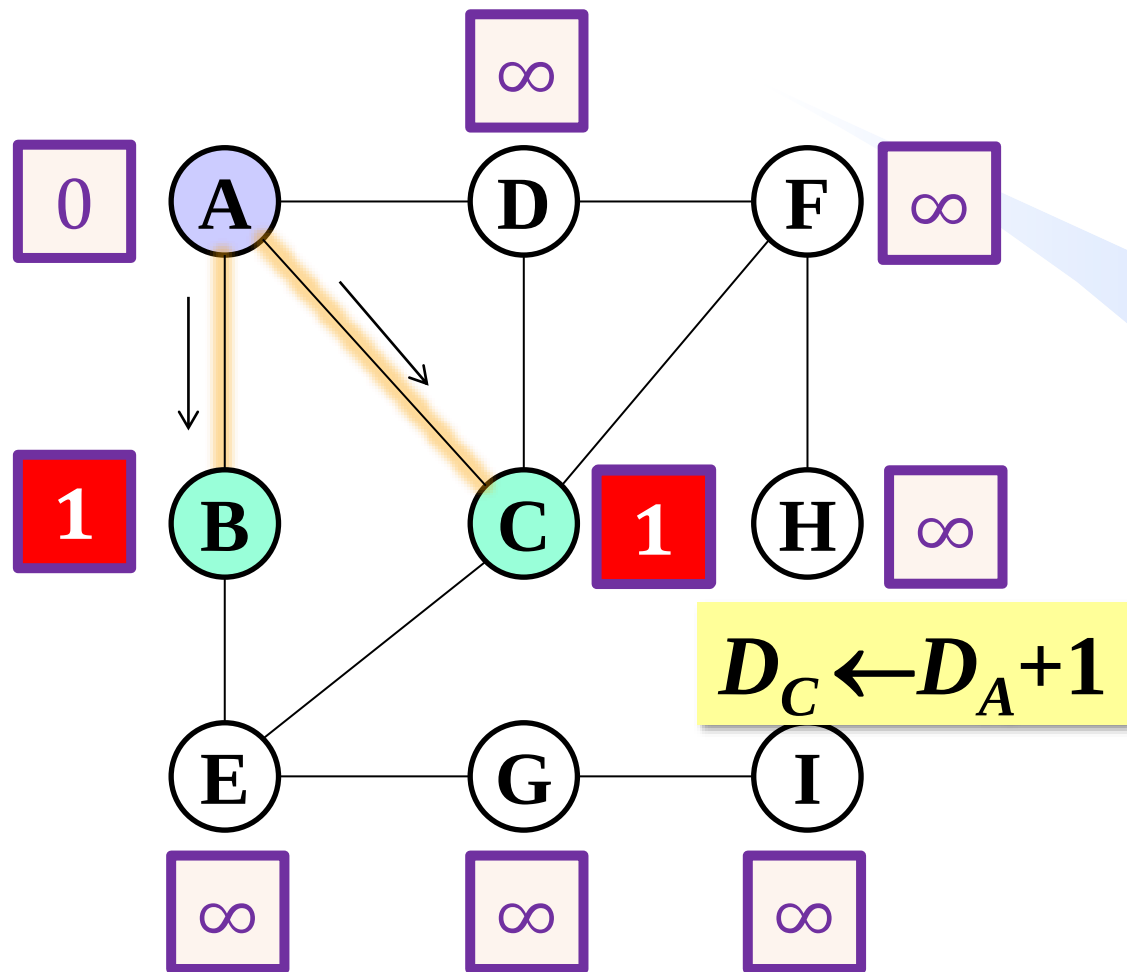


令 D_i 表示从源点到顶点 i 的最短距离

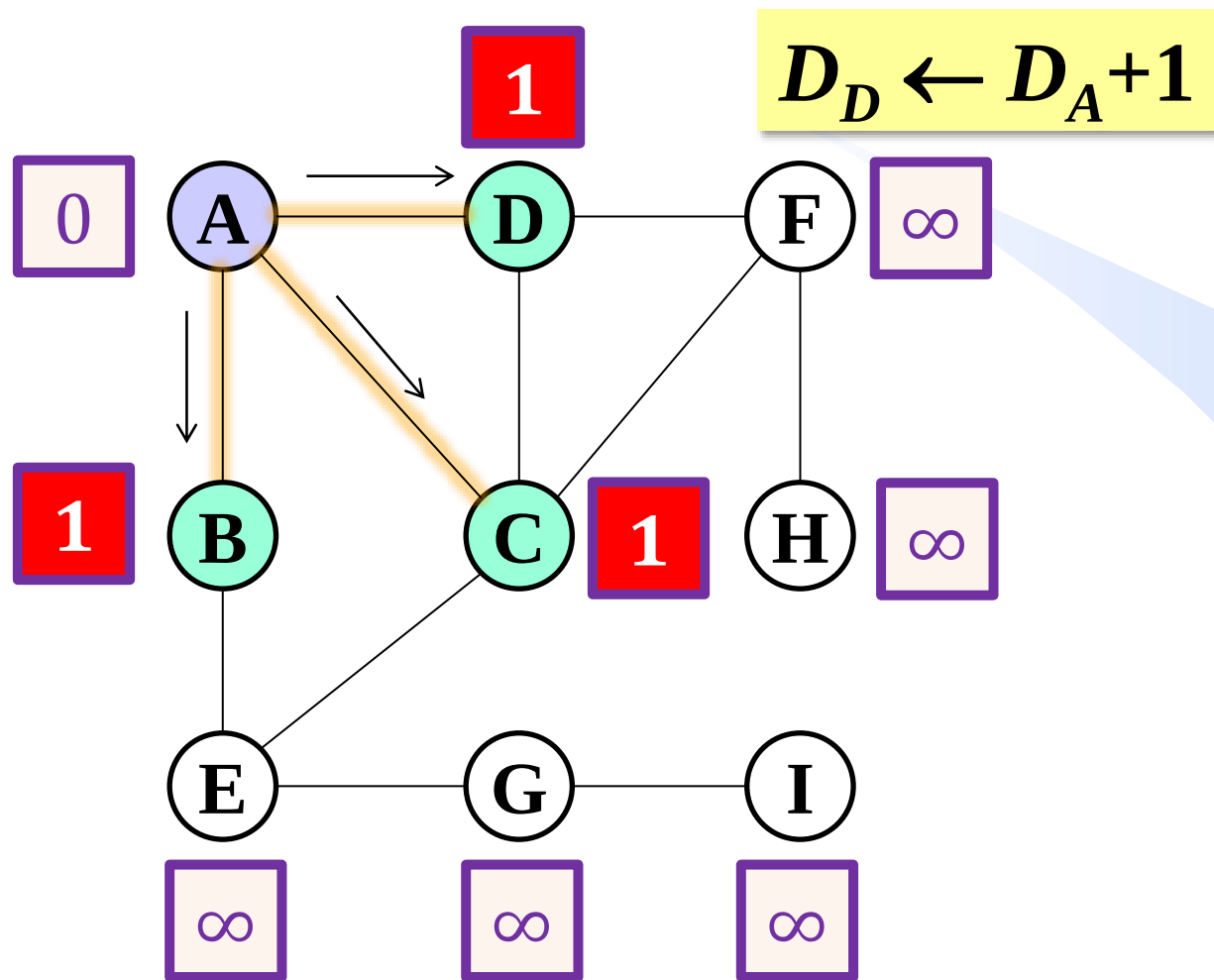
基于BFS的无权图最短路径算法



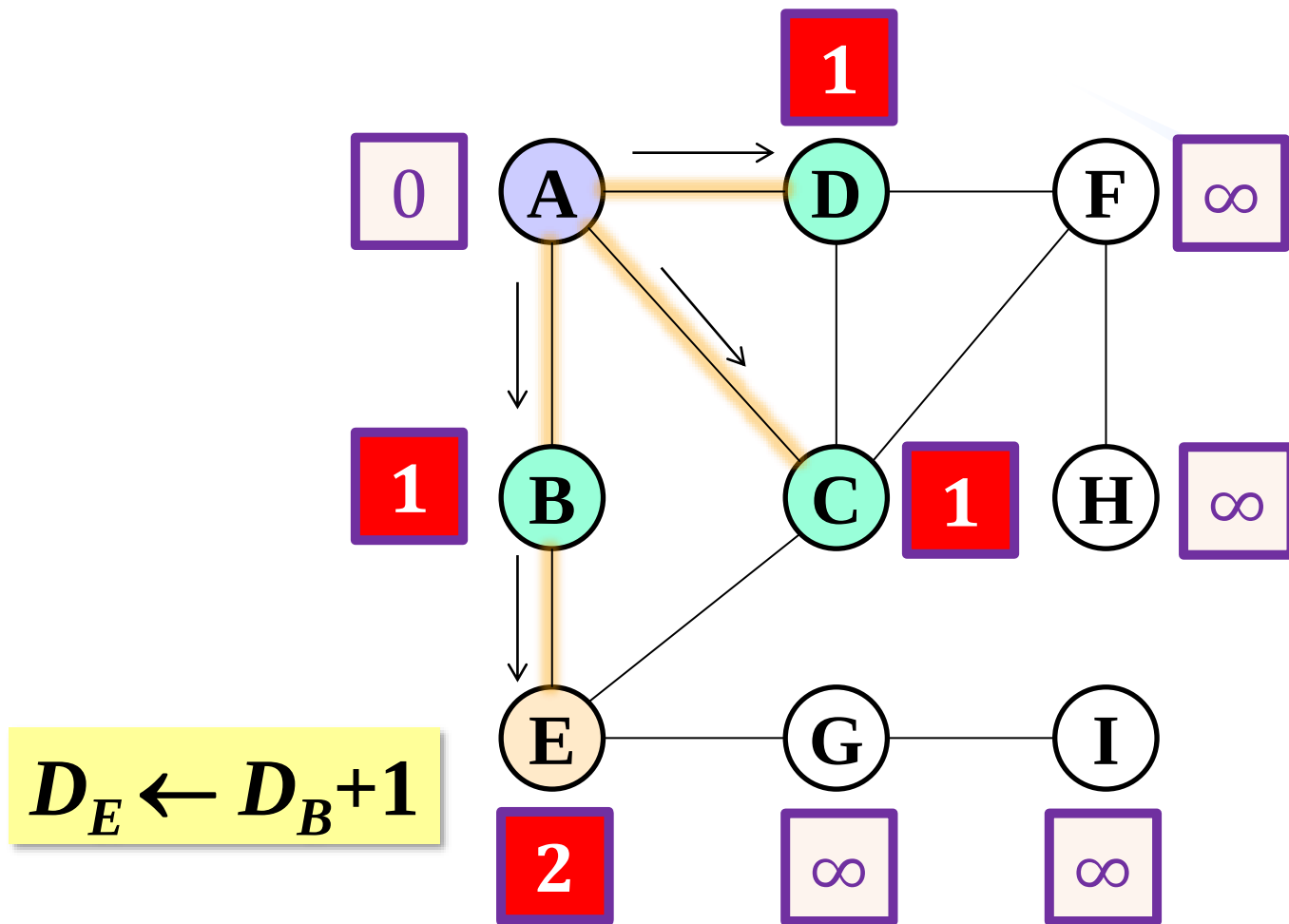
基于BFS的无权图最短路径算法



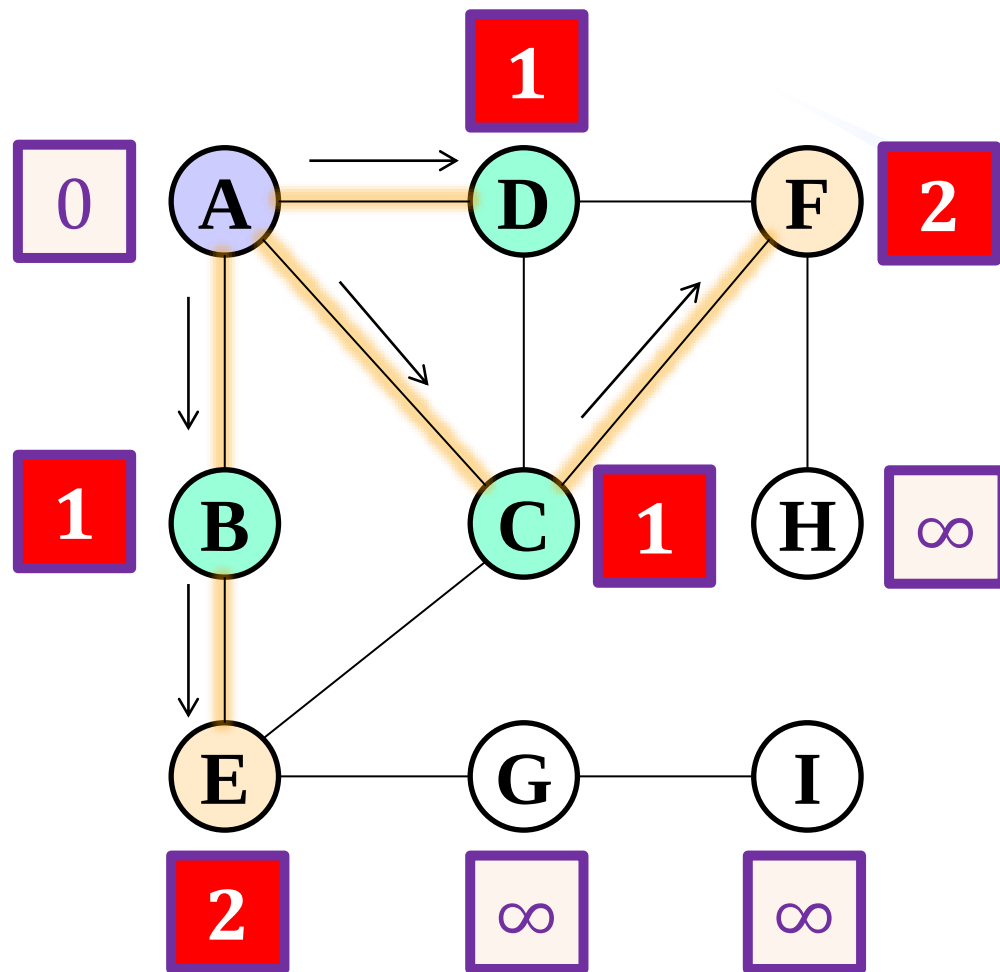
基于BFS的无权图最短路径算法



基于BFS的无权图最短路径算法

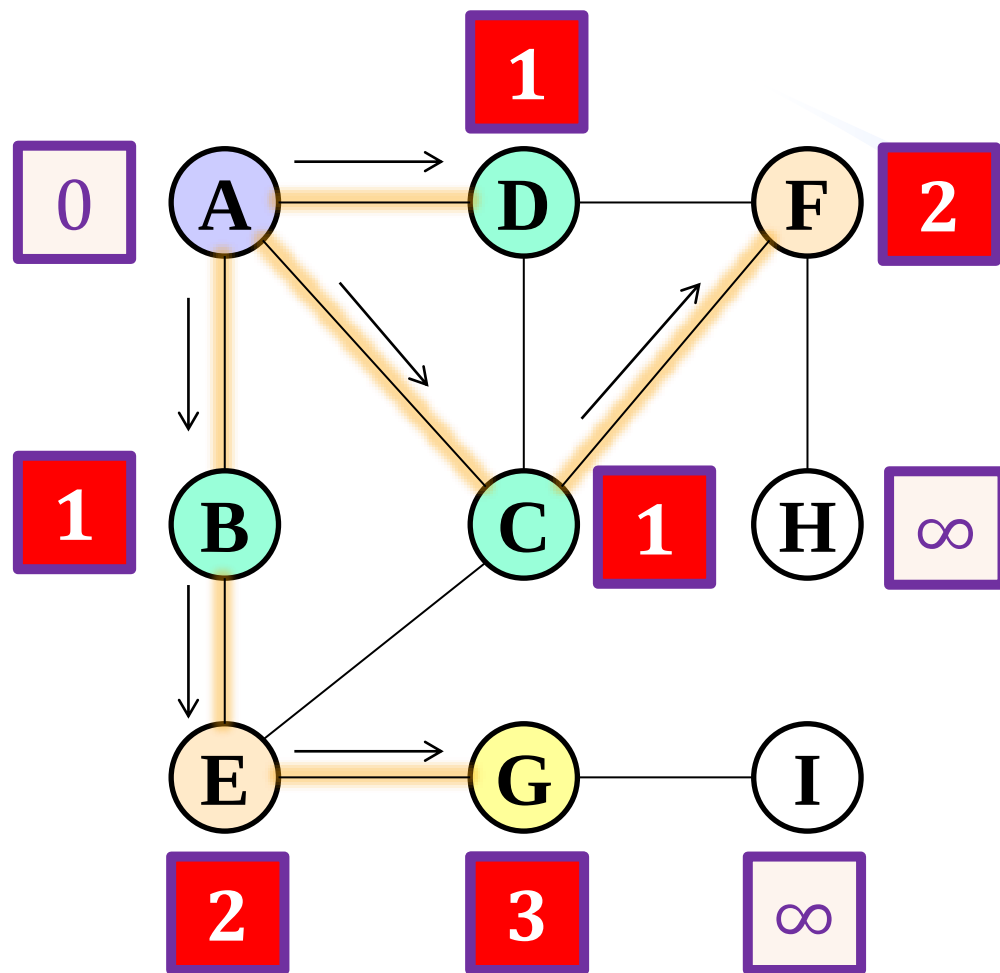


基于BFS的无权图最短路径算法



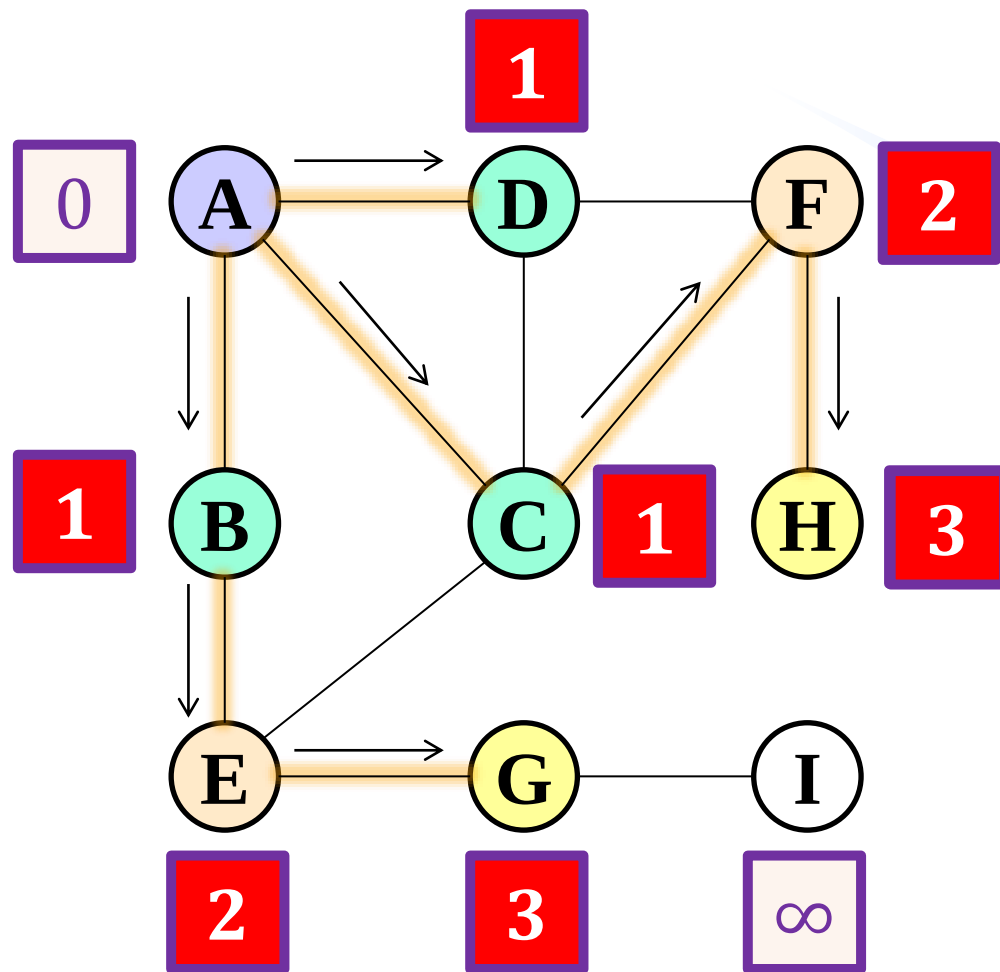
$$D_F \leftarrow D_C + 1$$

基于BFS的无权图最短路径算法

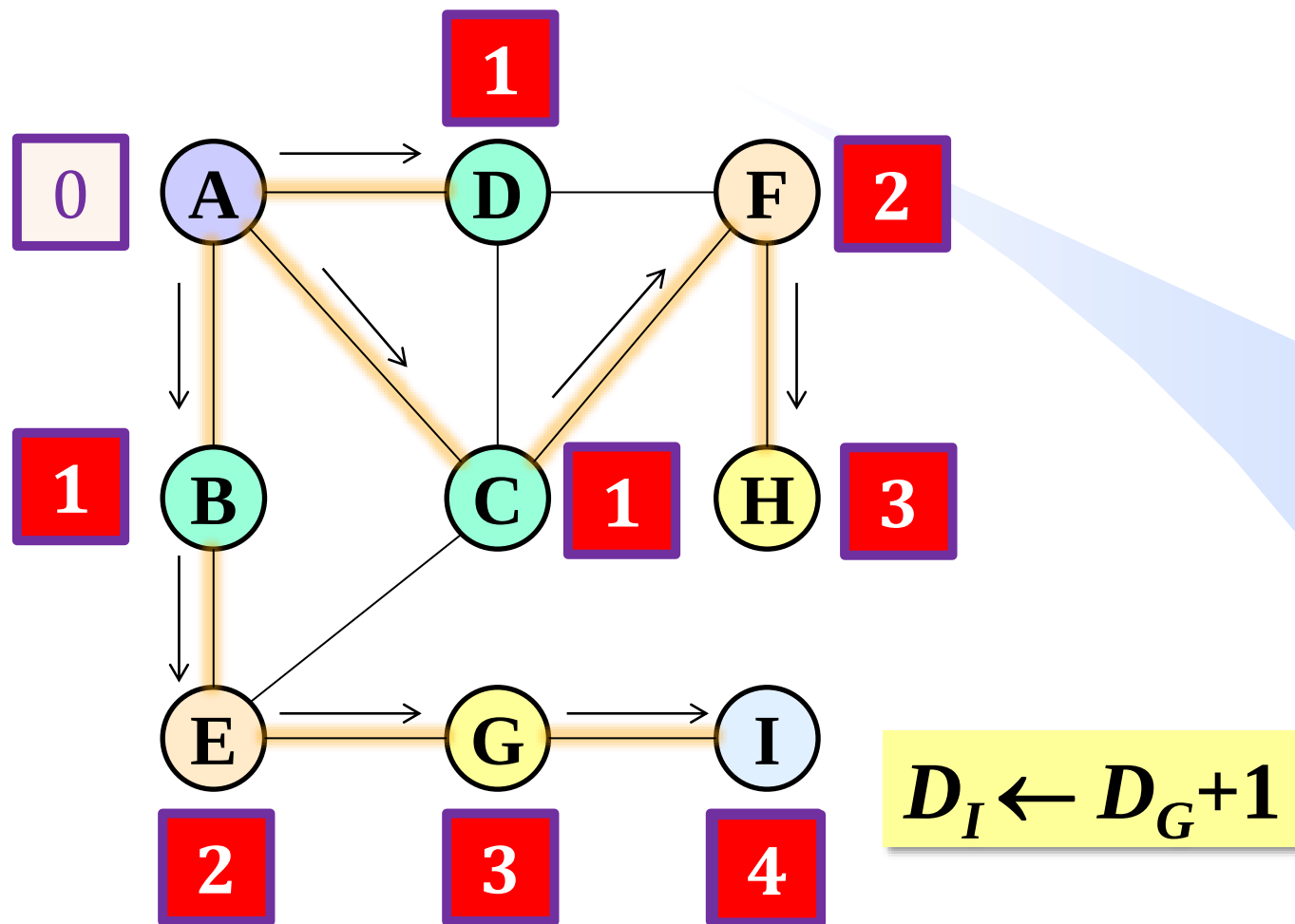


$$D_G \leftarrow D_E + 1$$

基于BFS的无权图最短路径算法



基于BFS的无权图最短路径算法



基于BFS的无权图最短路径算法

- 解决最短路径问题，使用两个数组 ***dist[]*** 和 ***pre[]***.
 - ✓ ***dist[i]***: 从源点 *u* 到顶点 *i* 的最短距离, 即 D_i , 初始时 $dist[u]=0$, $dist[i]=\infty (\forall i \neq u)$;
 - ✓ ***pre[i]***: 从源点到顶点 *i* 的最短路径中顶点 *i* 的前驱顶点, 初值-1.

Path	1					
	2					
	...					
	<i>n</i>					

二维数组 *Path*?
并无必要

- C/C++ 如何表示 $+\infty$
 - ✓ **INT_MAX**: `limits.h` 头文件中的预定义宏, 值为 $2^{31}-1$, (`#define INT_MAX 2147483647`), 加法或翻倍会溢出。
 - ✓ **0x3f3f3f3f**: 值为 1061109567, 加法或翻倍一般不会溢出, 在图算法中更常用, 一般用法: `const int INF=0x3f3f3f3f;`

基于BFS的无权图最短路径算法

解决最短路径问题，使用两个数组 $dist[]$ 和 $pre[]$ 。

- $dist[i]$: 从源点到顶点 i 的最短距离，源点初值0，其他点初值 ∞ ；
- $pre[i]$: 从源点到顶点 i 的最短路径中顶点 i 的前驱顶点，初值-1。

- ① 初始化 $dist$ 和 pre 数组，然后对源点 u 置 $dist[u]$ 为0， u 入队；
- ② 检测队列是否为空，若队列为空，则迭代结束；
- ③ 出队一个顶点 v ，考察其每个邻接顶点 w ：

如果 w 未被访问过，则

$$dist[w] \leftarrow dist[v] + 1.$$

$$pre[w] \leftarrow v.$$

将 w 入队；

- ④ 执行步骤②。



- ✓ 顶点 i 没被访问时： $dist[i]$ 为 ∞
- ✓ 顶点 i 第1次被访问时： $dist[i]$ 被更新为源点到 i 的最短距离

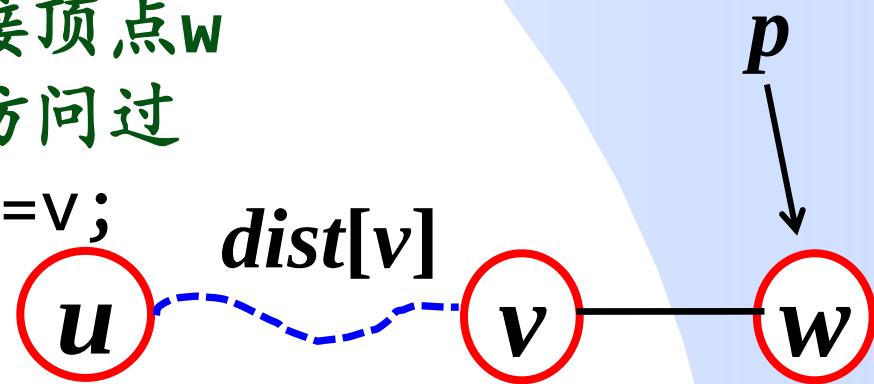
基于BFS的无权图最短路径算法

```

void BFS(Vertex* Head, int n, int u, int dist[], int pre[]){
    Queue<int> Q;    //创建队列Q, 队列需预先实现
    for(int i=1; i<=n; i++) {pre[i]=-1; dist[i]=INF;}
    dist[u]=0; Q.enqueue(u);    //源点u入队
    while(!Q.empty()){
        int v=Q.dequeue();    //出队一个点
        for(Edge* p=Head[v].adjacent; p!=NULL; p=p->link){
            int w=p->VerAdj;    //考察v的邻接顶点w
            if(dist[w]==INF){    //如果w没被访问过
                dist[w]=dist[v]+1; pre[w]=v;
                Q.enqueue(w);
            }
        }
    }
}

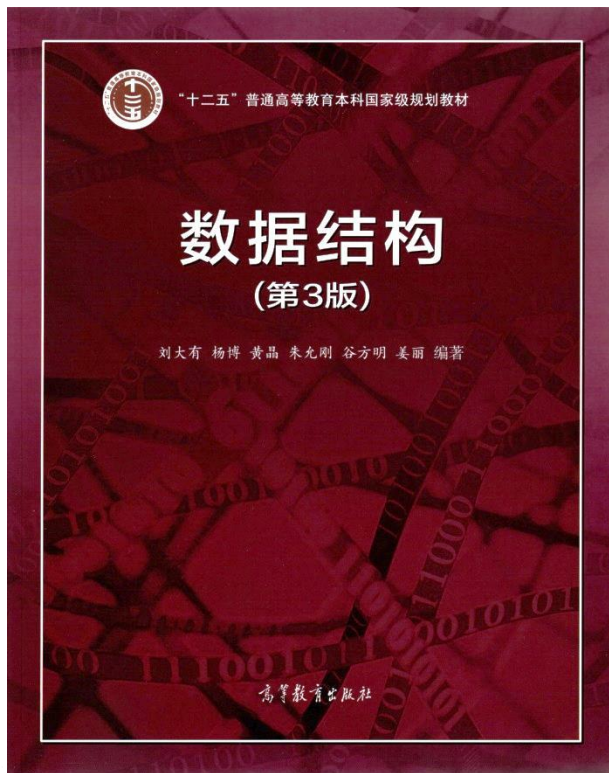
```

时间复杂度
 $O(n+e)$



$dist[i]$:从源点到顶点 i 的最短距离

$pre[i]$:从源点到顶点 i 的最短路径上顶点 i 的前驱顶点



图的最短路径

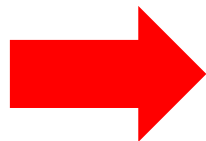
- 无权图最短路径
- **Dijkstra算法**
- A*算法
- Floyd算法
- 满足约束的最短路径

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

最短路径问题

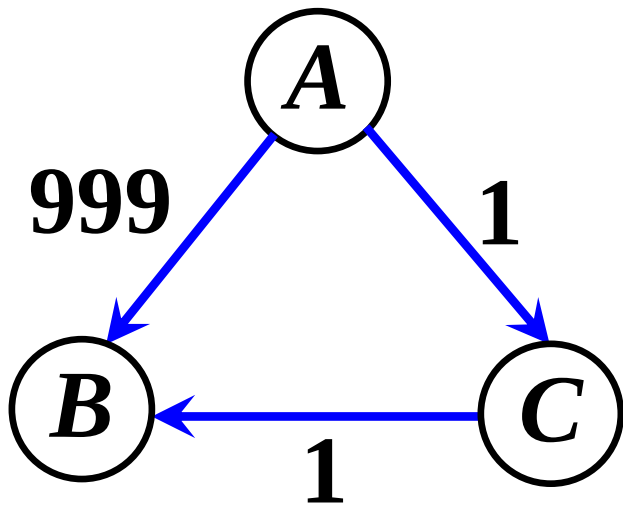
无权图的单
源最短路径
问题



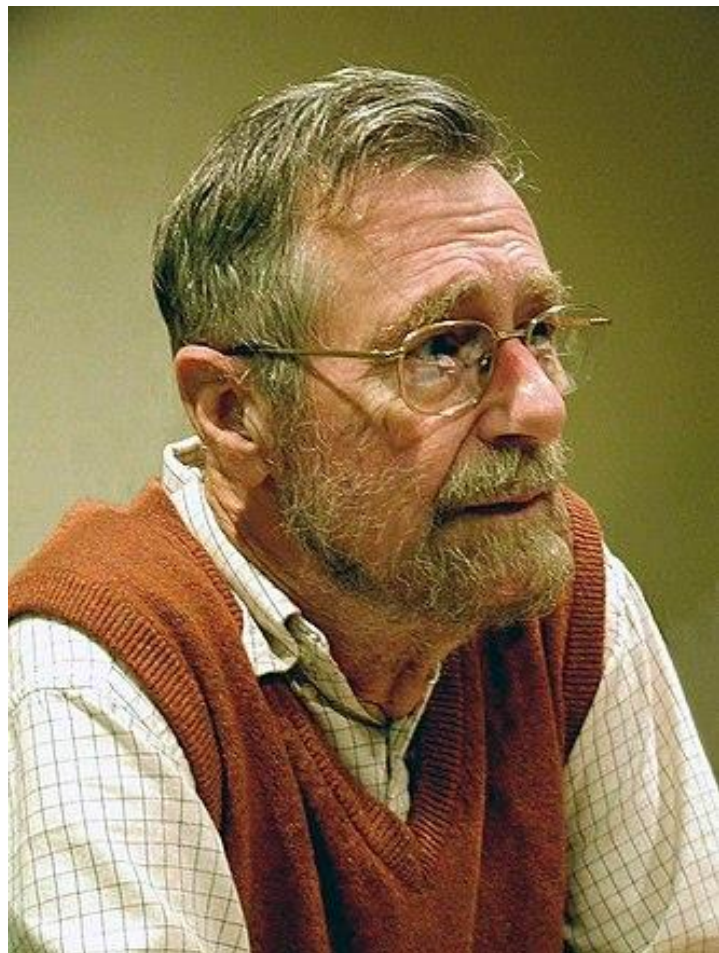
正权图的单
源最短路径
问题

正权图的单源最短路径问题

A



- 带权图，权值非负。BFS可行么？
- 若**直接**按BFS：设A为源点，A的邻接顶点为B，从而有 $\text{dist}[B] = 999$ 。但是从A到B还有一条经由C的路径，长度为**2**。
- 路径的长度不再是边的条数，而是边的权值之和。BFS本质上只能反映源点到各个点最少经过多少条边。



Edsger Dijkstra (1930-2002)
德州大学奥斯汀分校教授
图灵奖获得者 (首位非英美籍获奖者)
荷兰皇家科学院院士



Edsger Dijkstra 及其夫人 Ria Debets

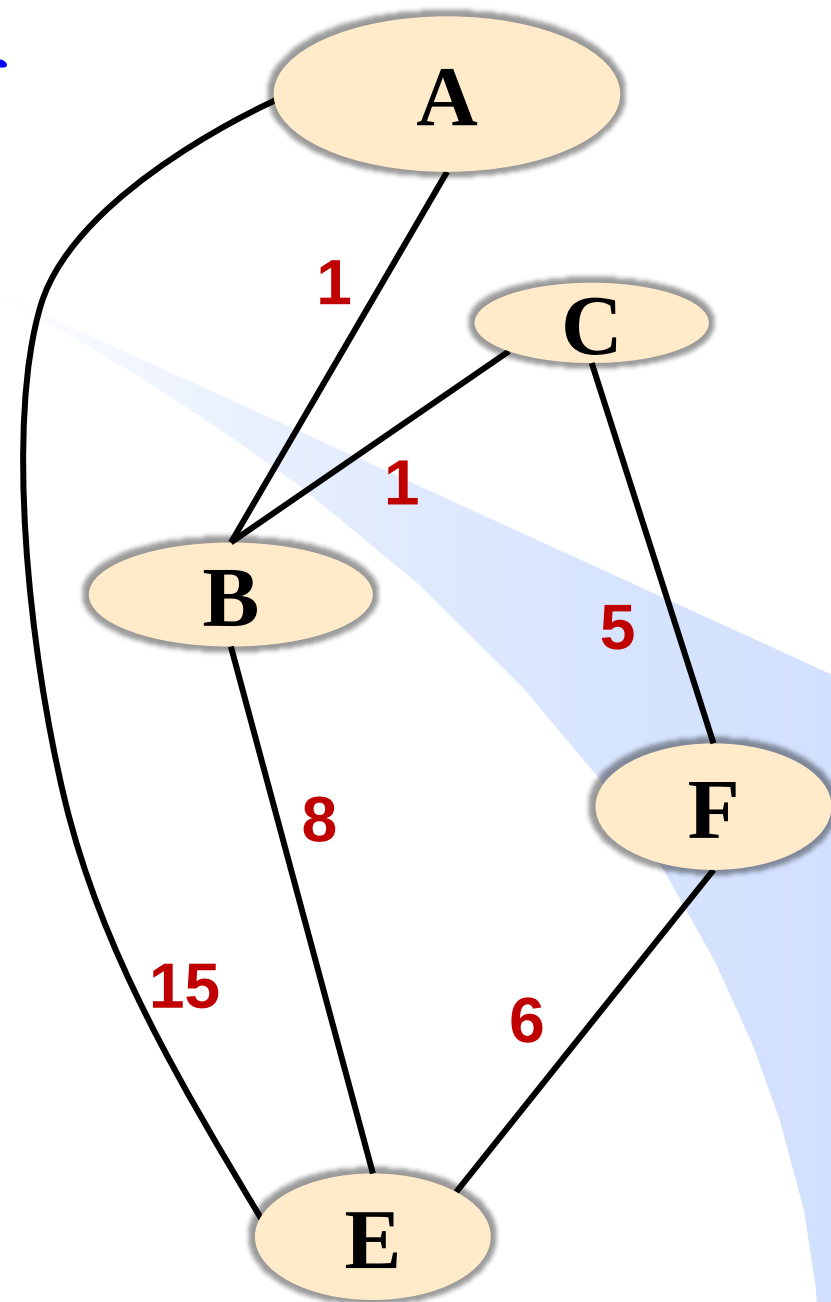


One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the cafe terrace to drink a cup of coffee. I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path. As I said, it was a 20-minute invention.

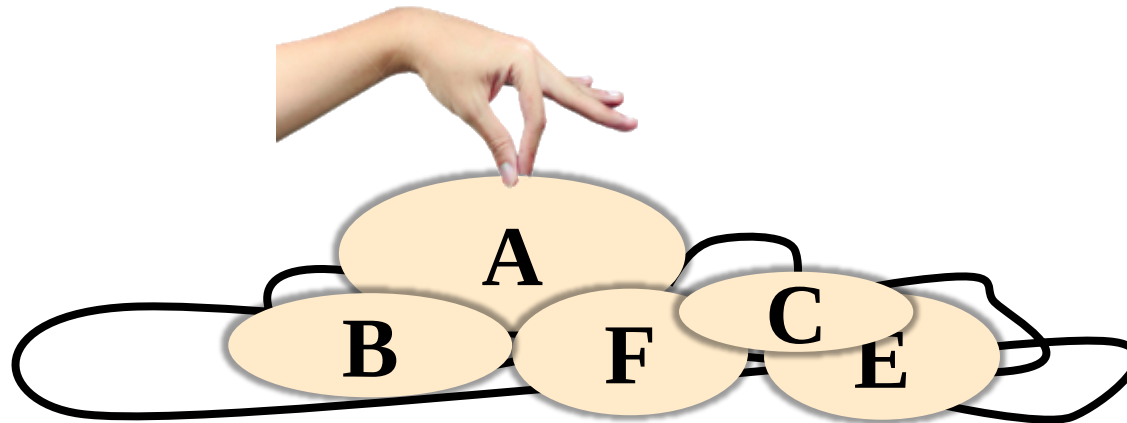
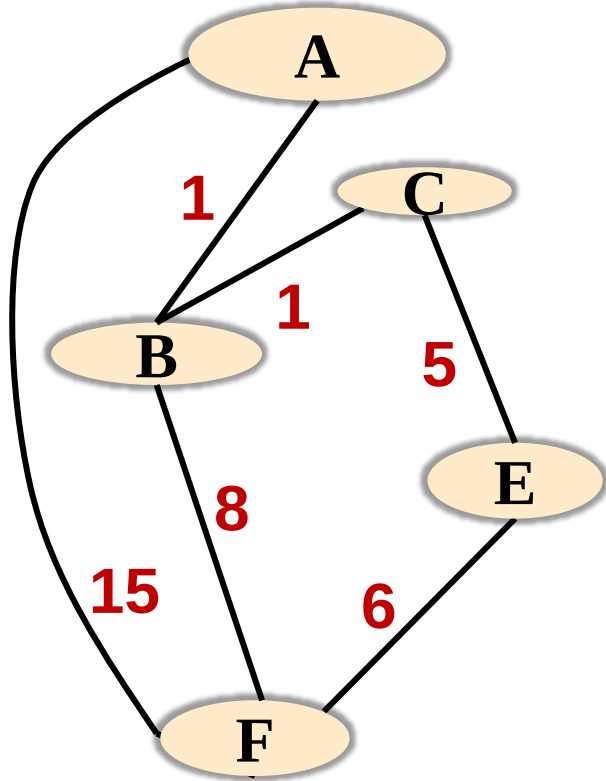
—— Edsger Dijkstra 23

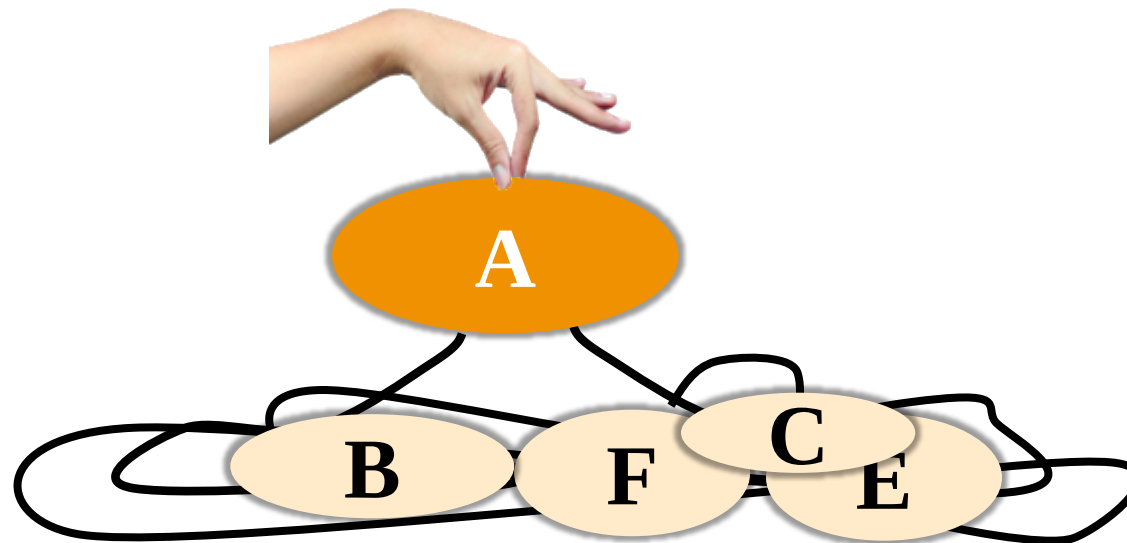
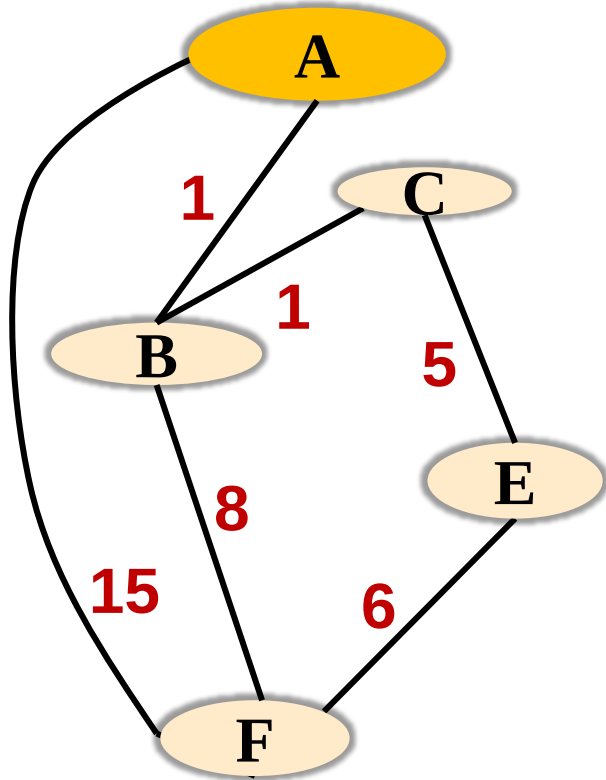
Dijkstra算法

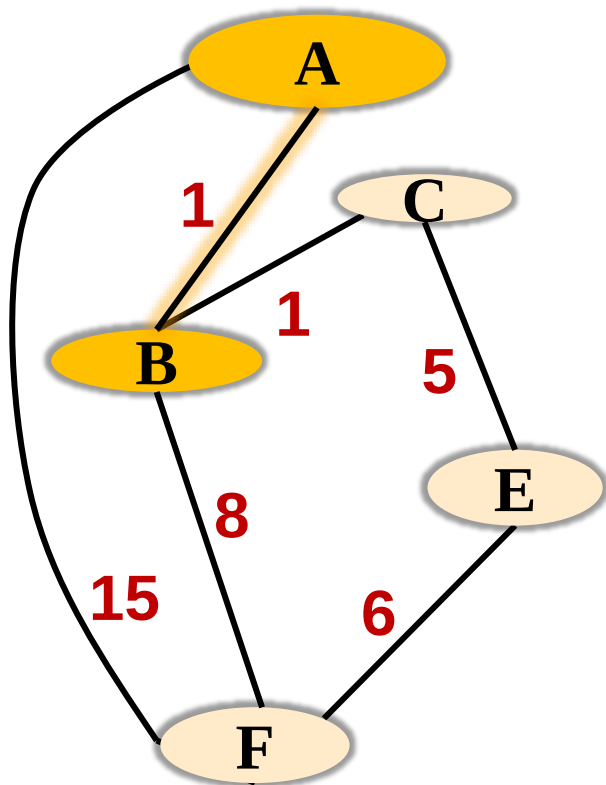
找源点A到其他顶
点的最短路径



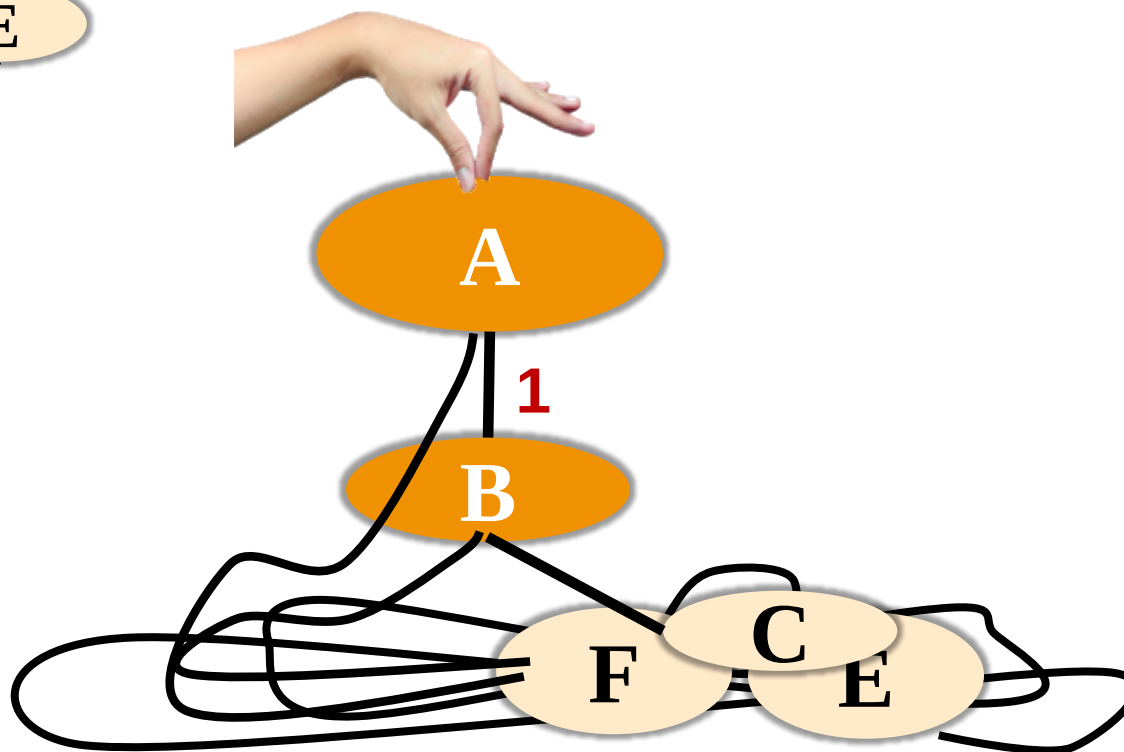
A

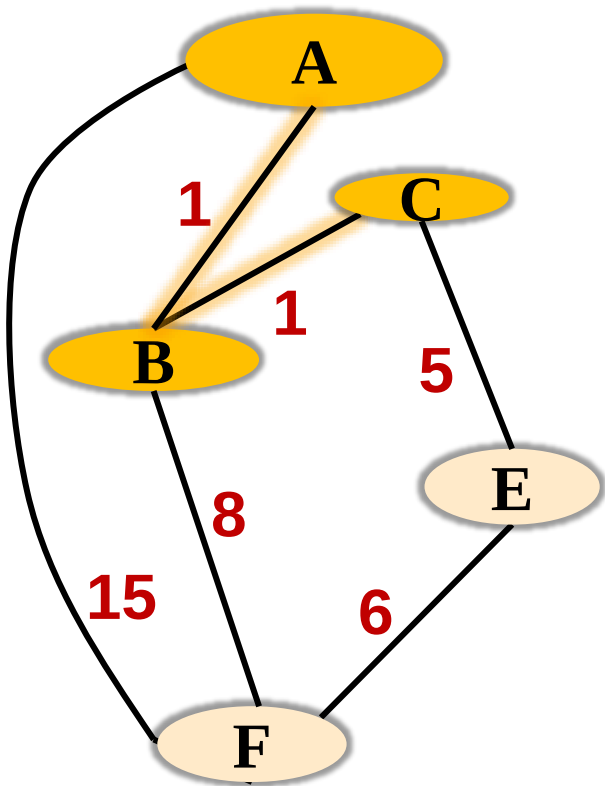






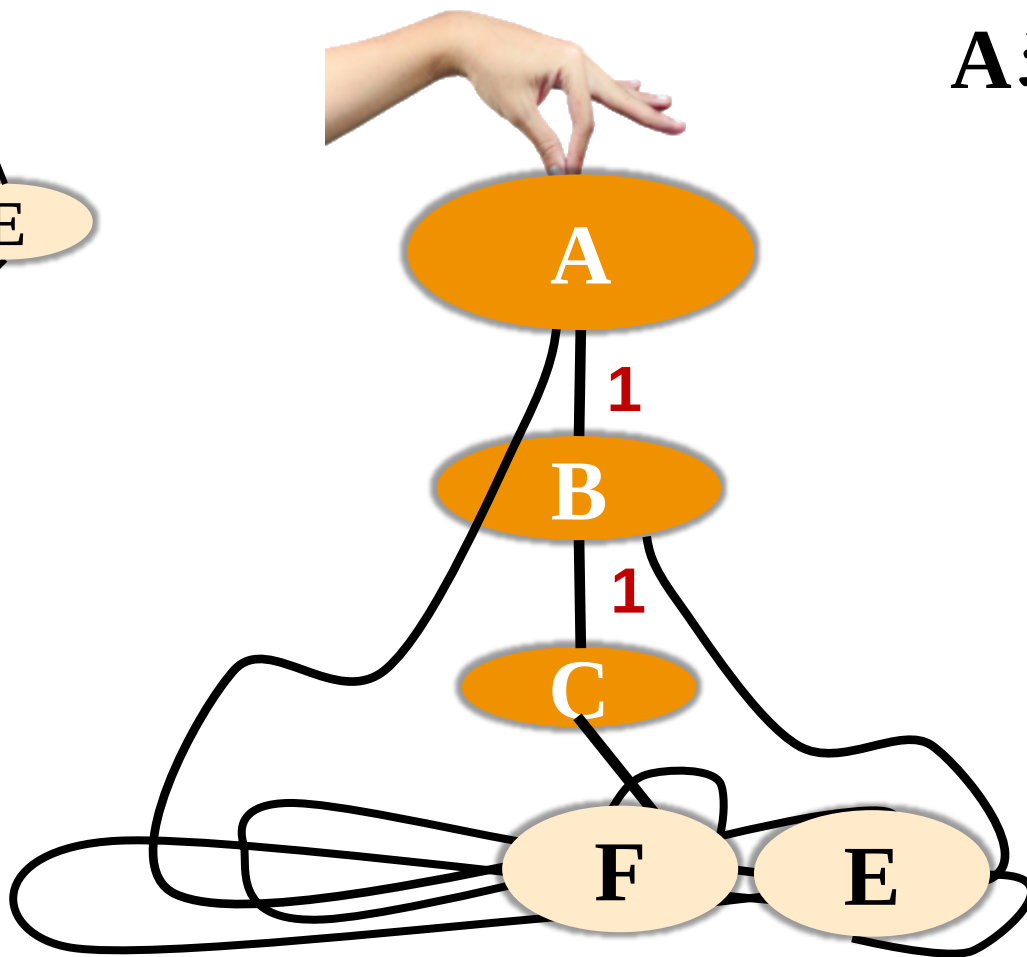
A到B的最短距离：1

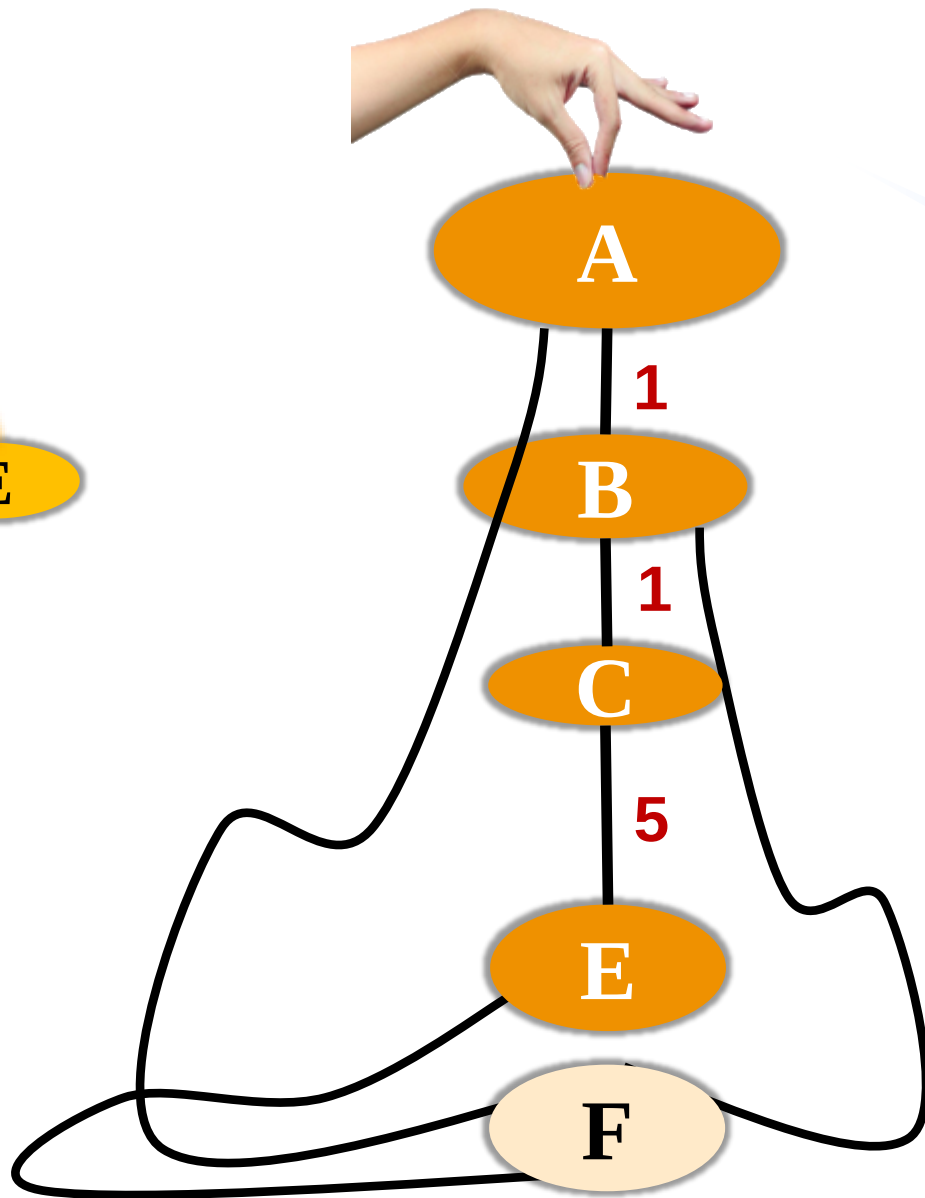
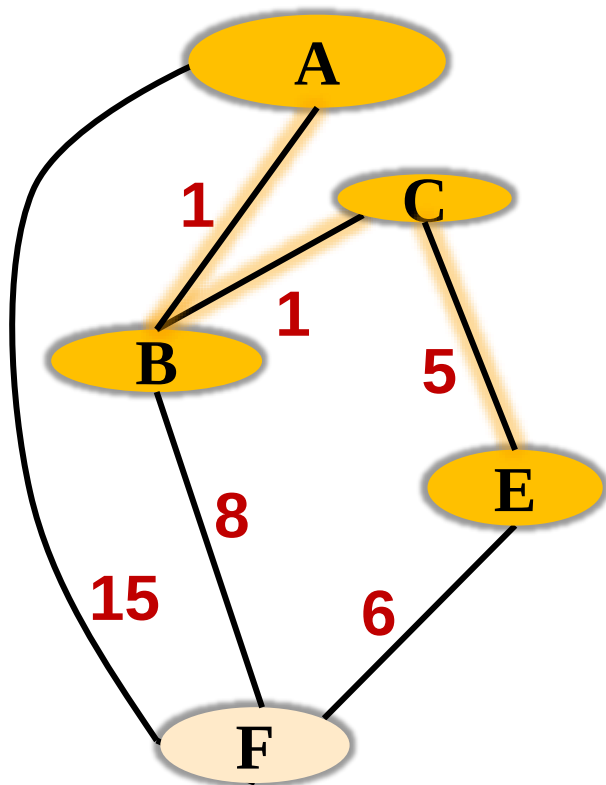




A到B的最短距离：1

A到C的最短距离：2

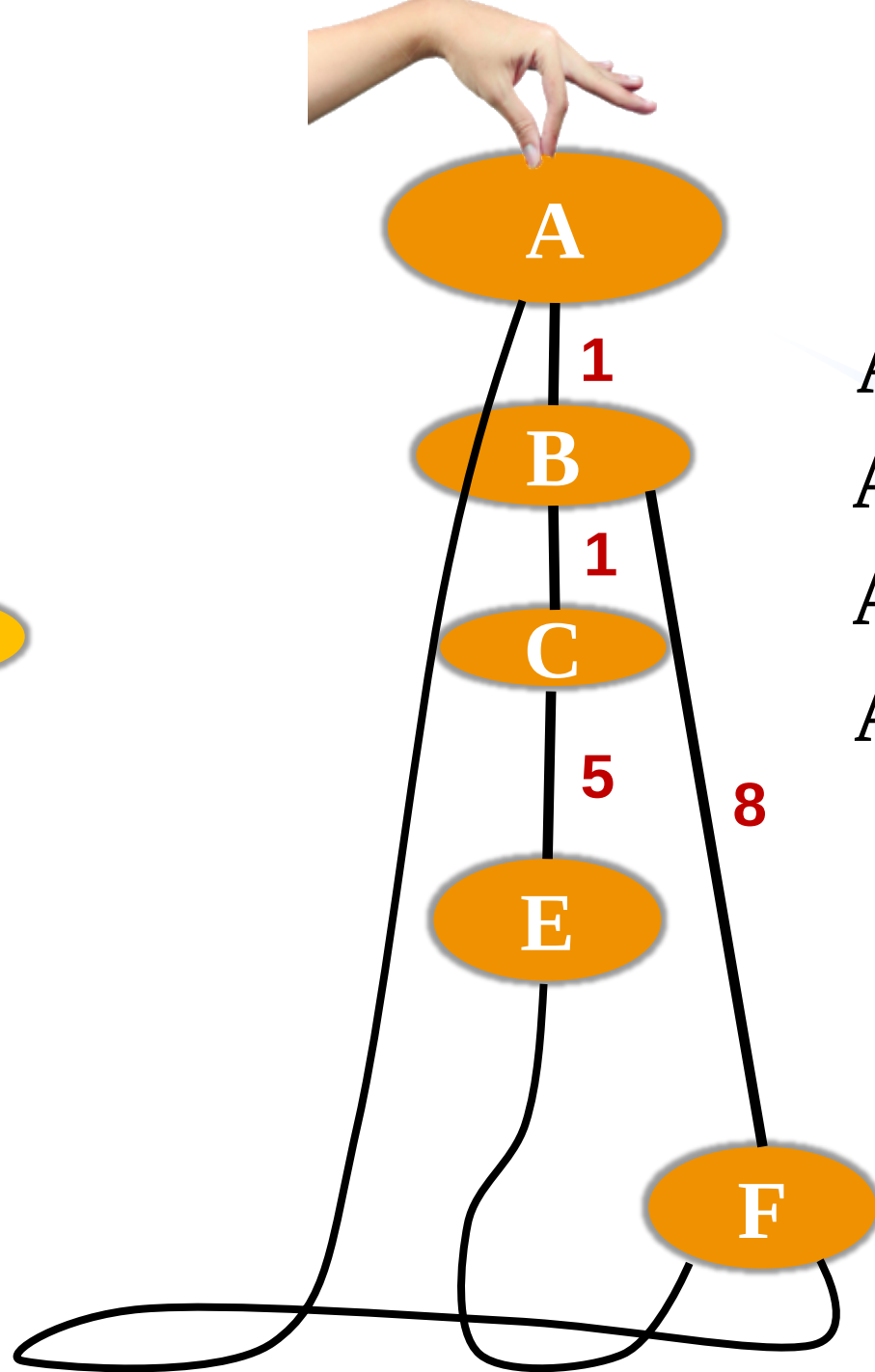
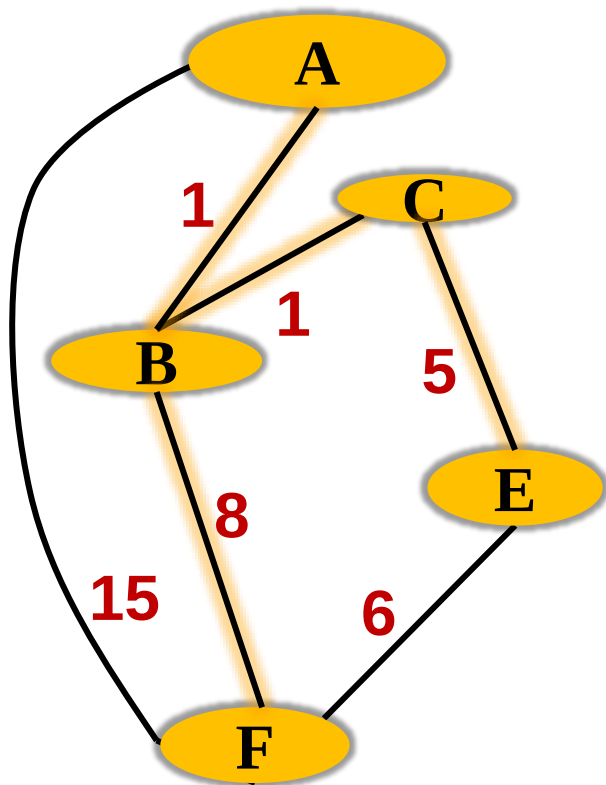




A到B的最短距离：1

A到C的最短距离：2

A到E的最短距离：7

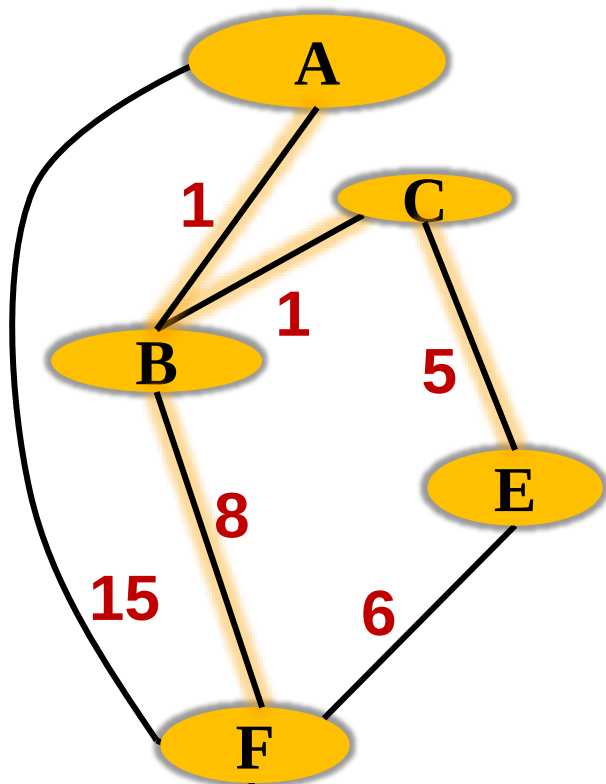


A到B的最短距离：1

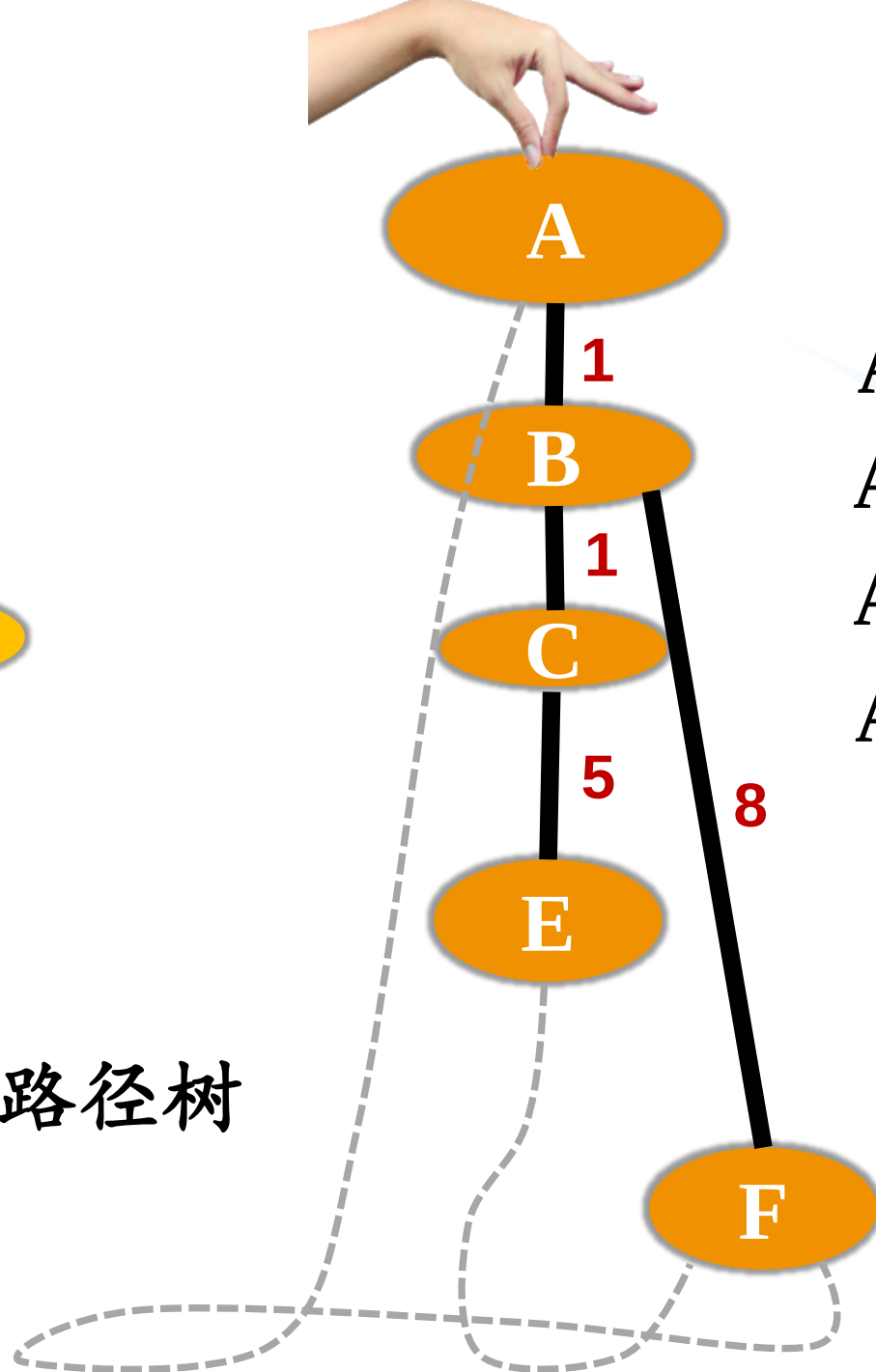
A到C的最短距离：2

A到E的最短距离：7

A到F的最短距离：9

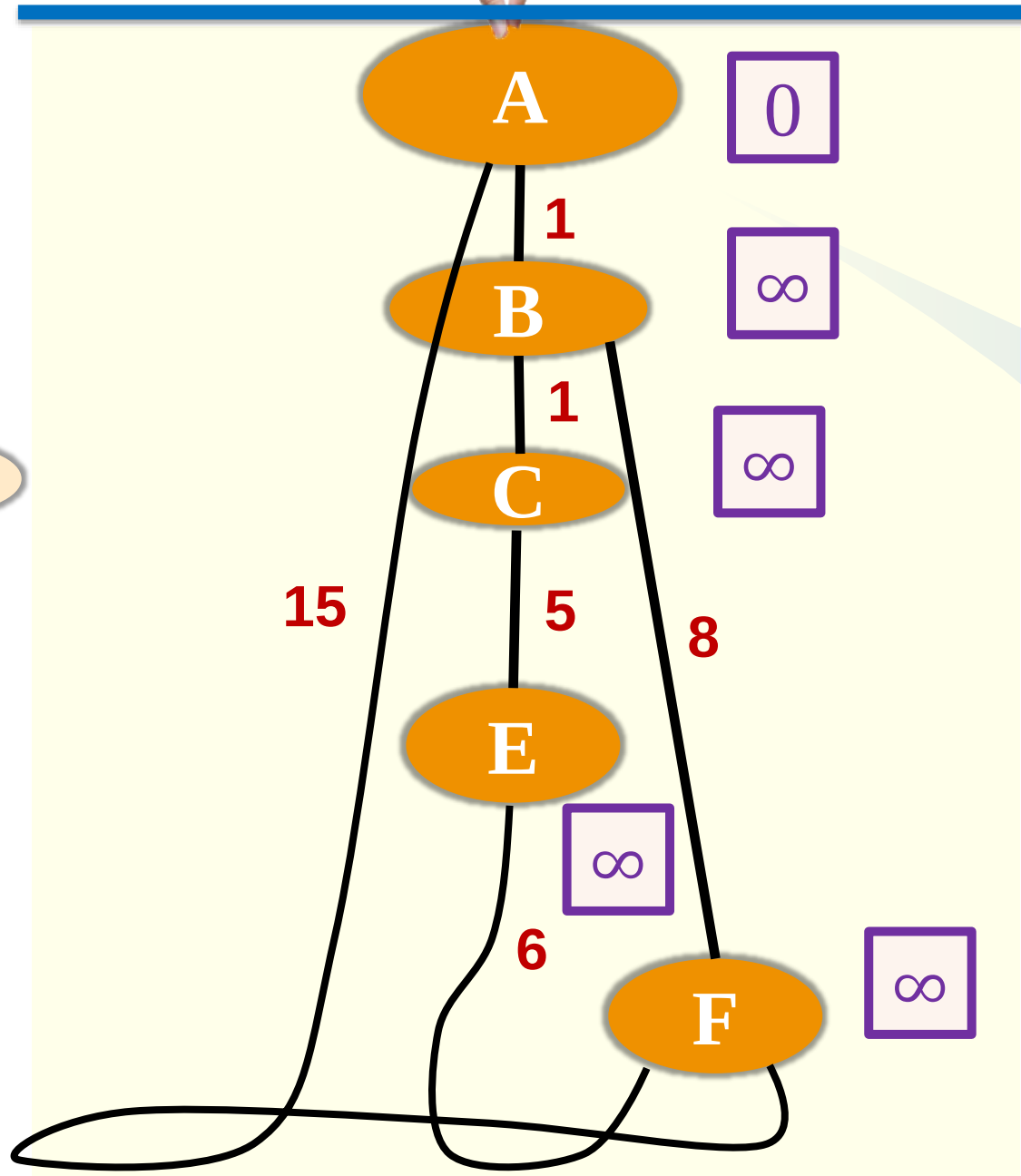
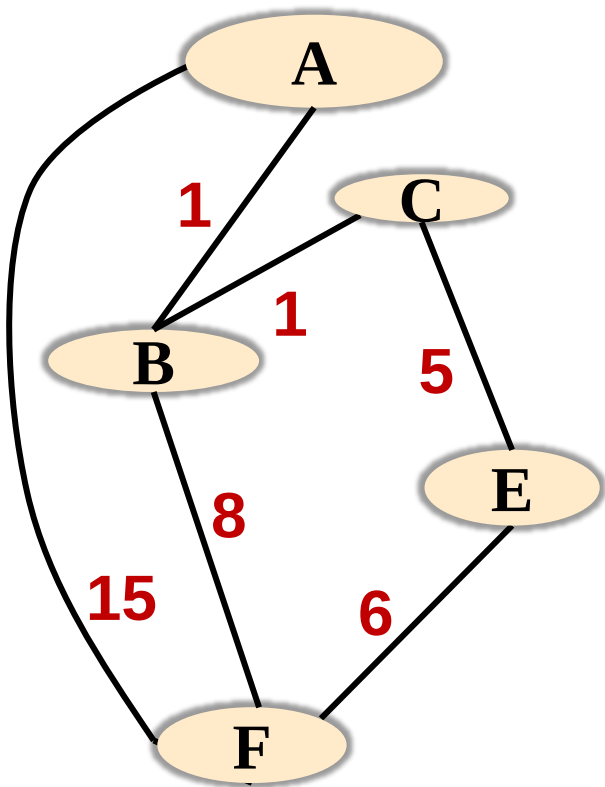


最短路径树



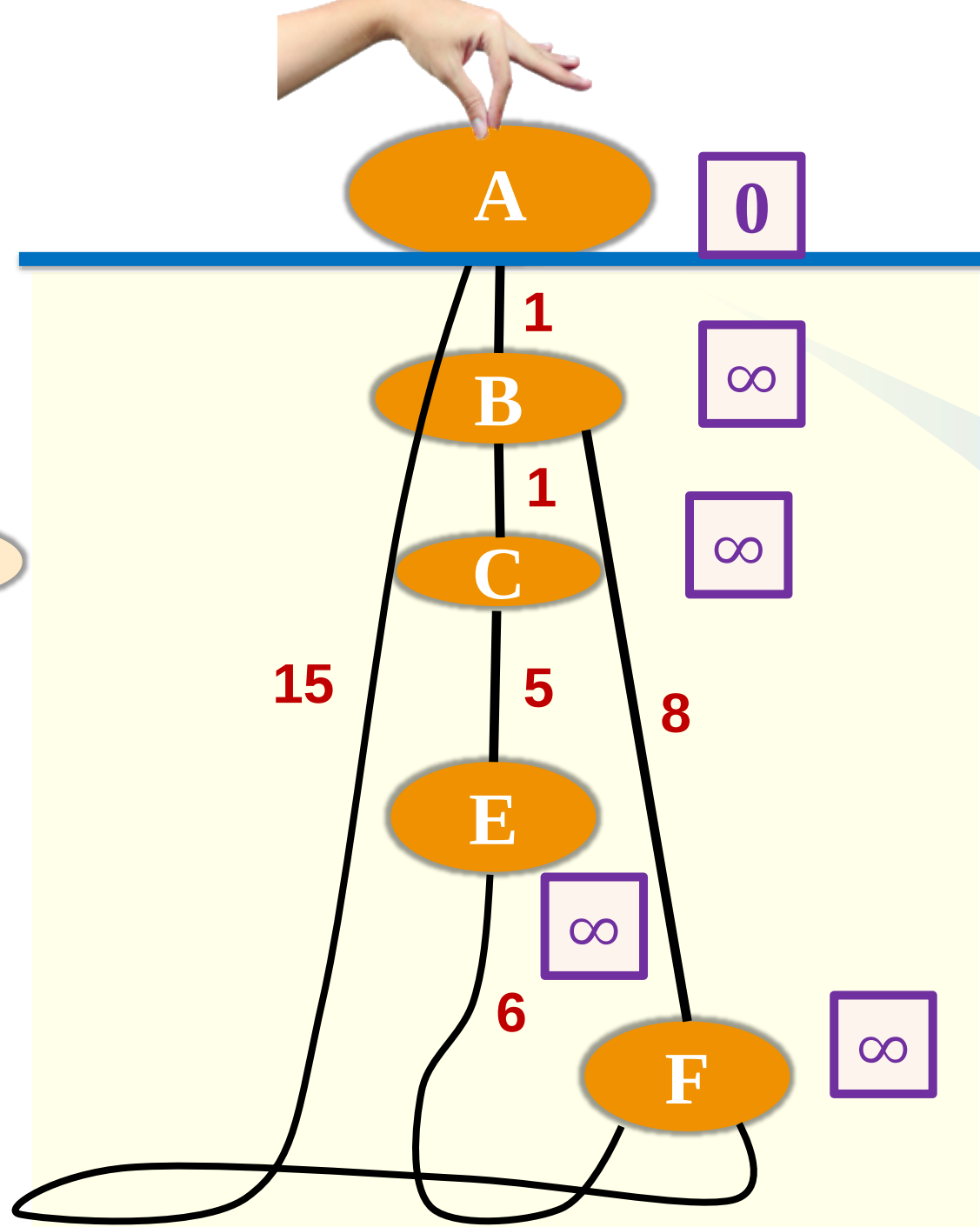
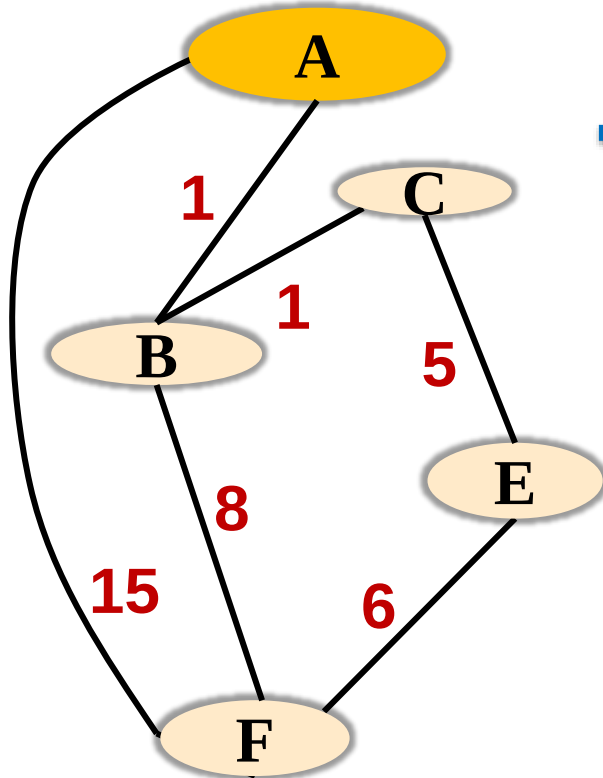
A到B的最短距离：1
A到C的最短距离：2
A到E的最短距离：7
A到F的最短距离：9

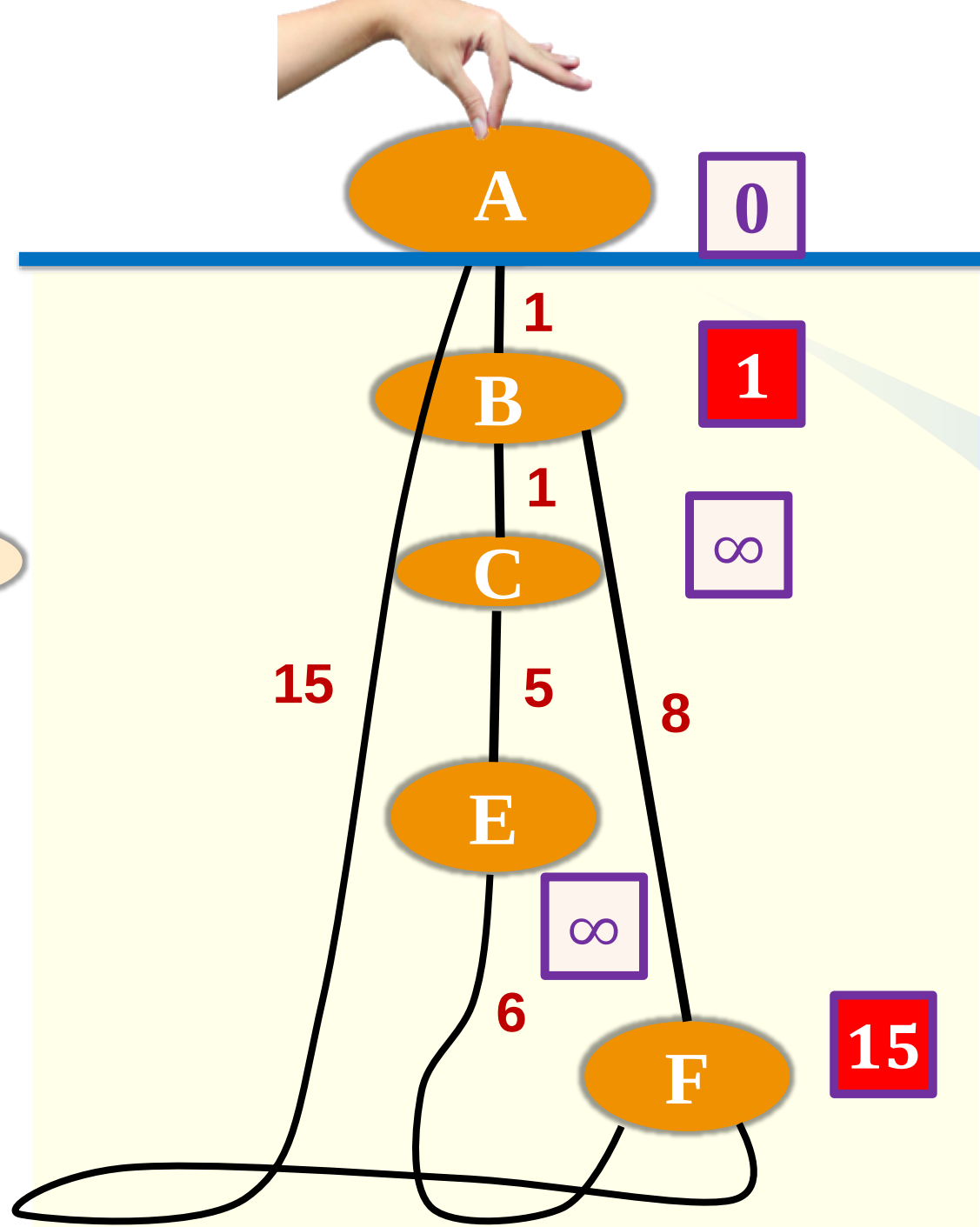
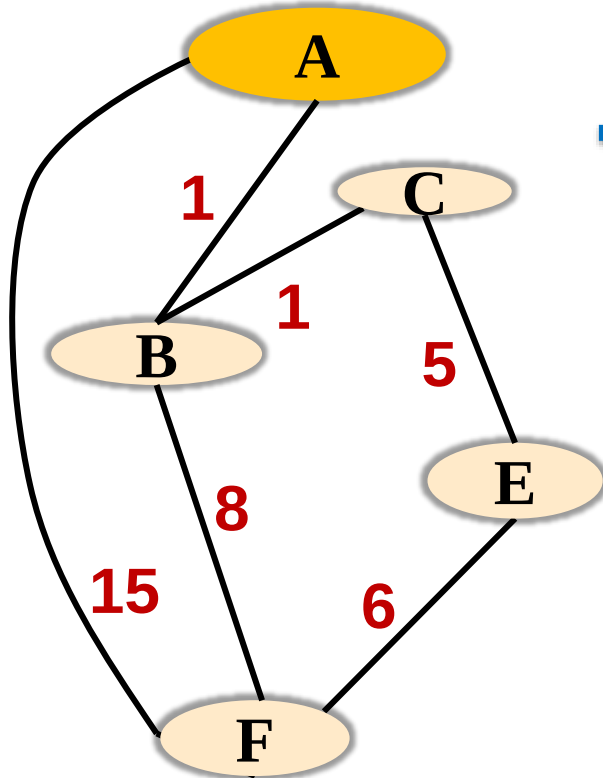
各顶点按照与
源点最短距离
的递增顺序依
次被拉起

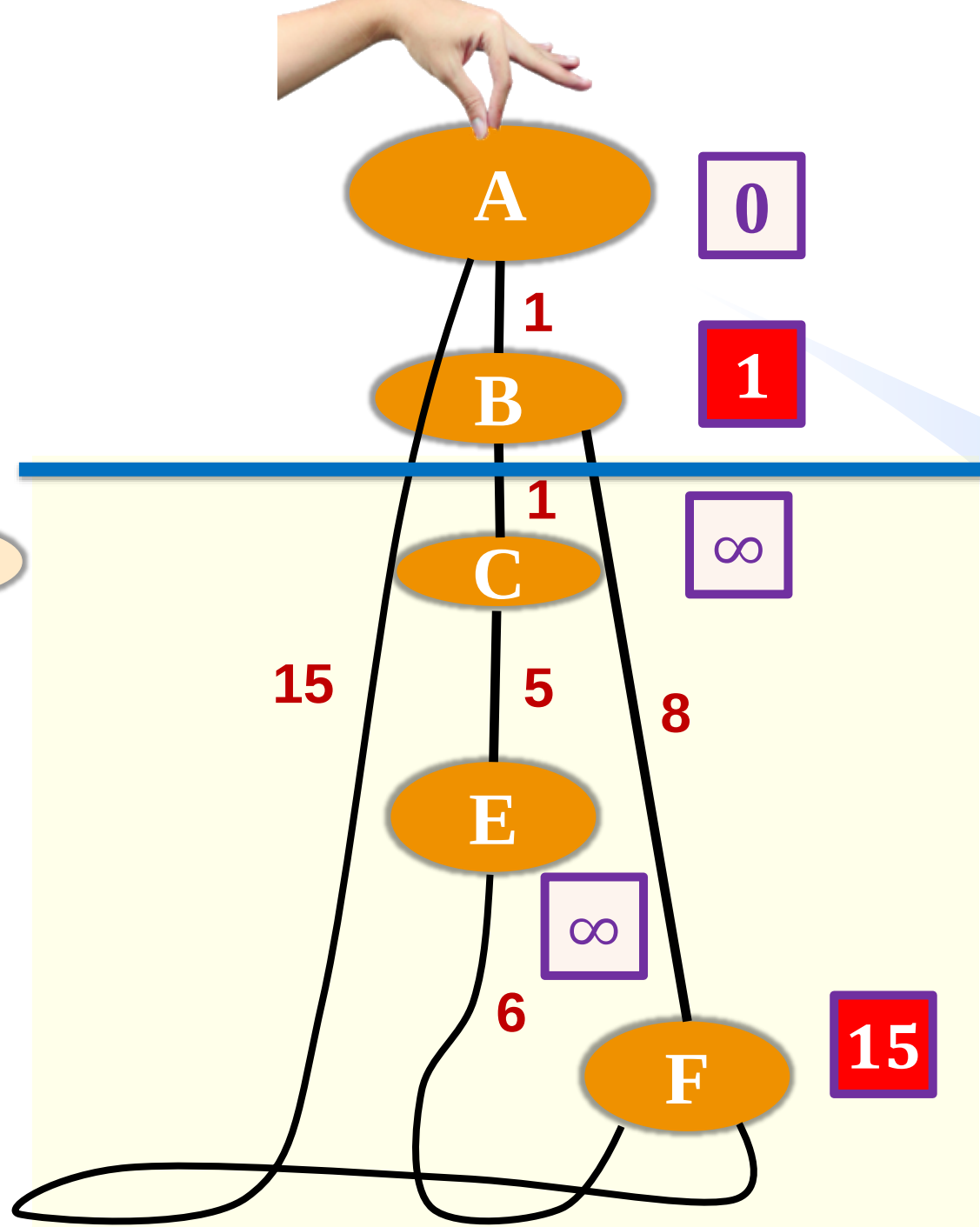
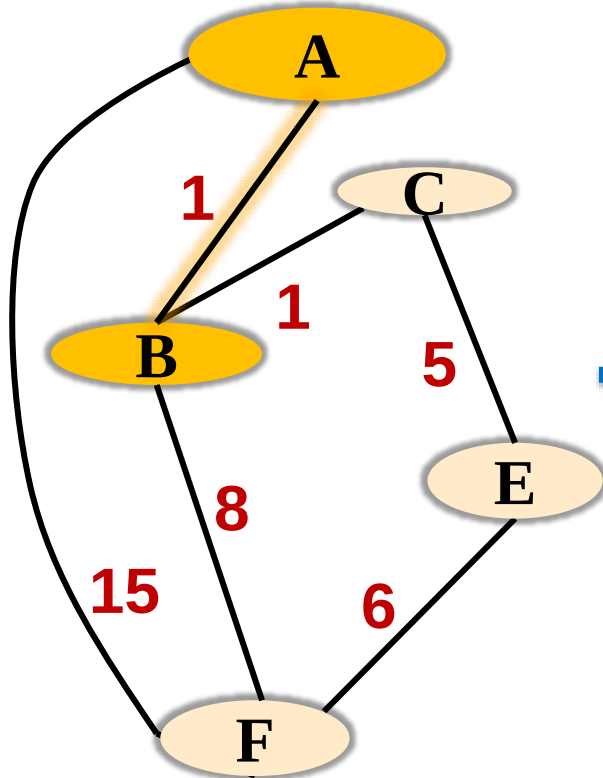


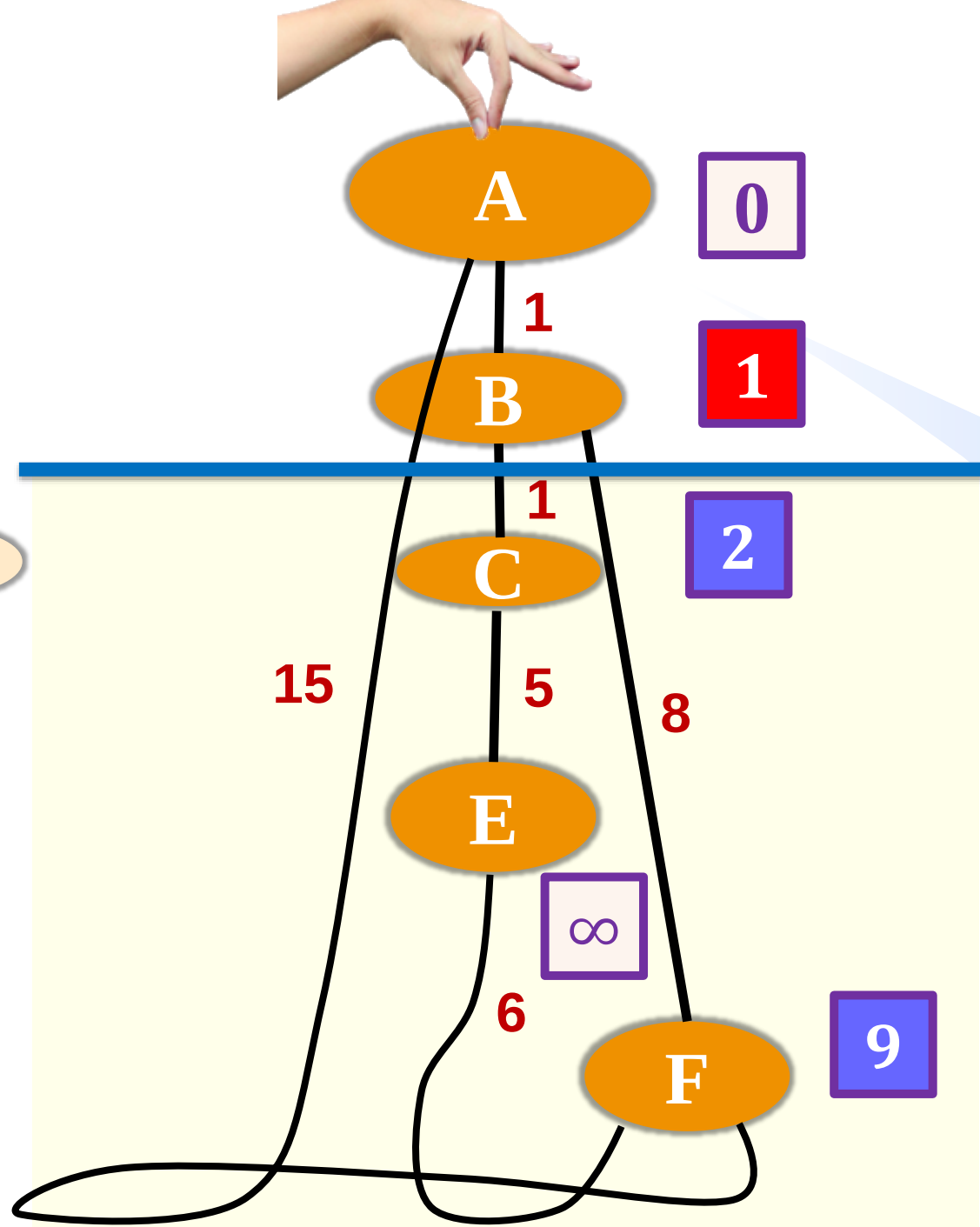
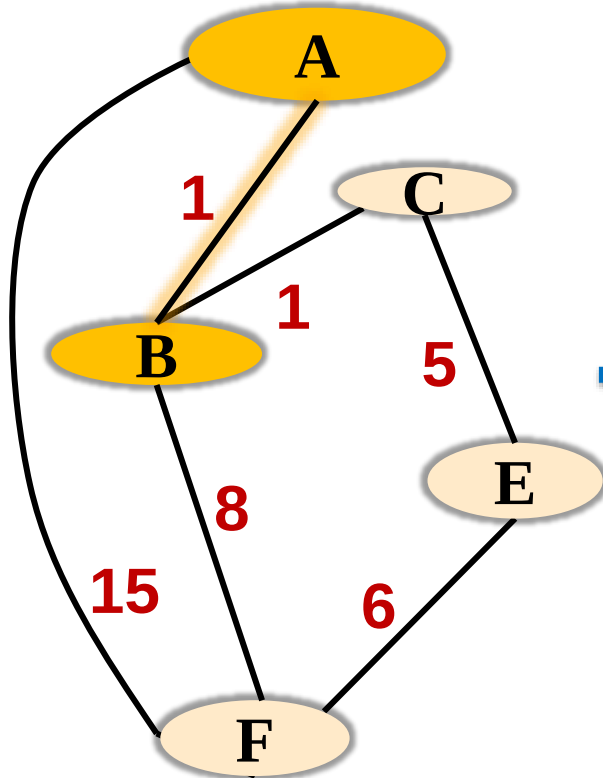
如何通过算法
实现这一过程？

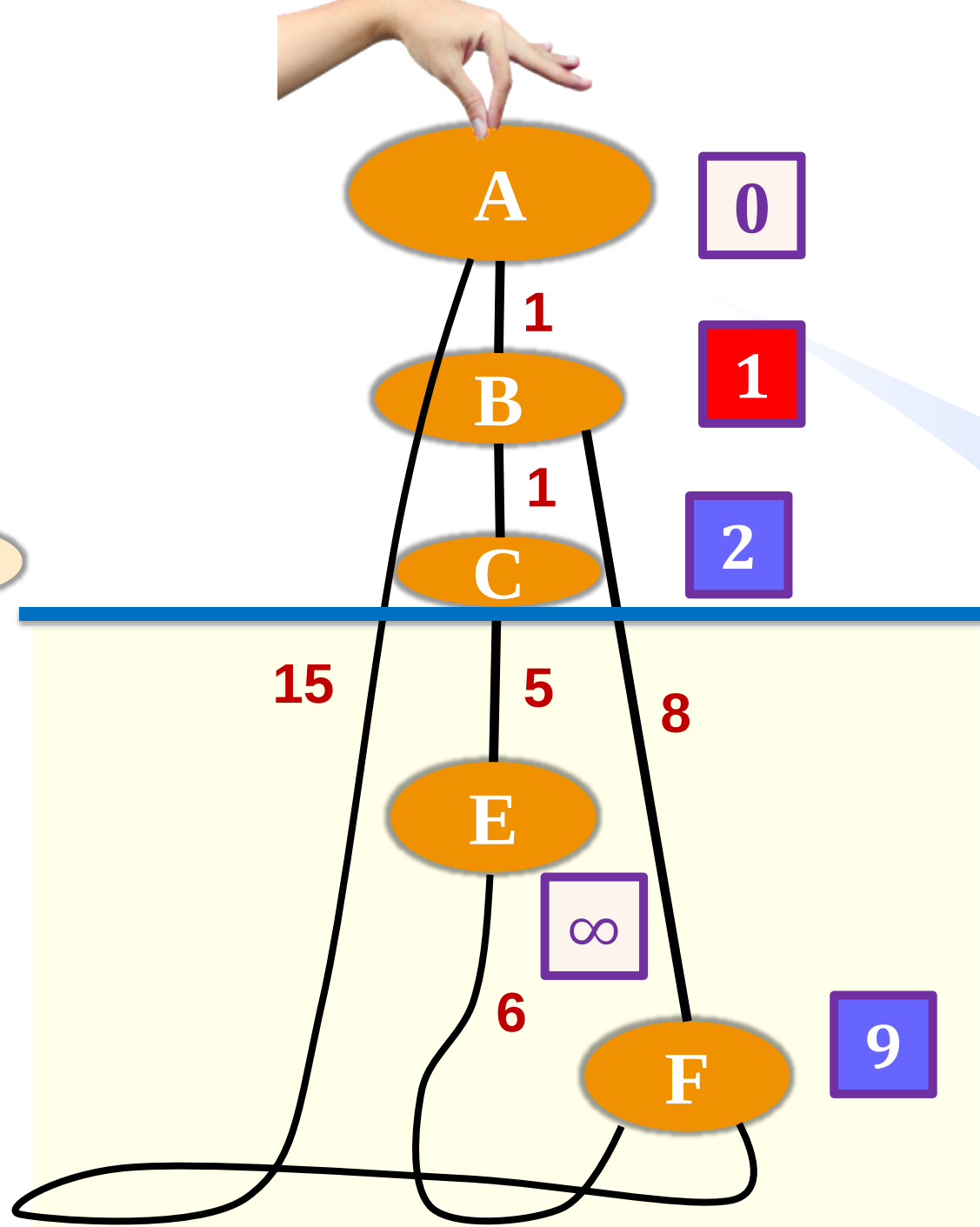
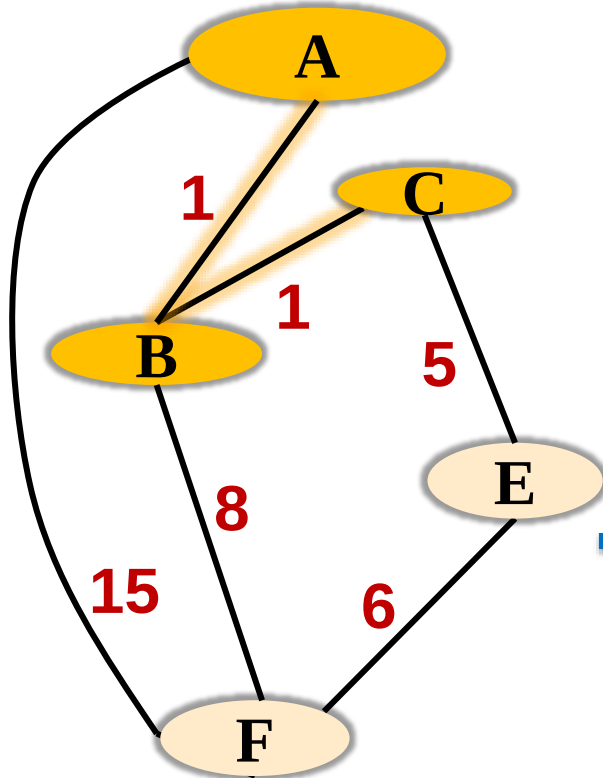
令 D_i 表示从源
点到顶点 i 的
最短距离

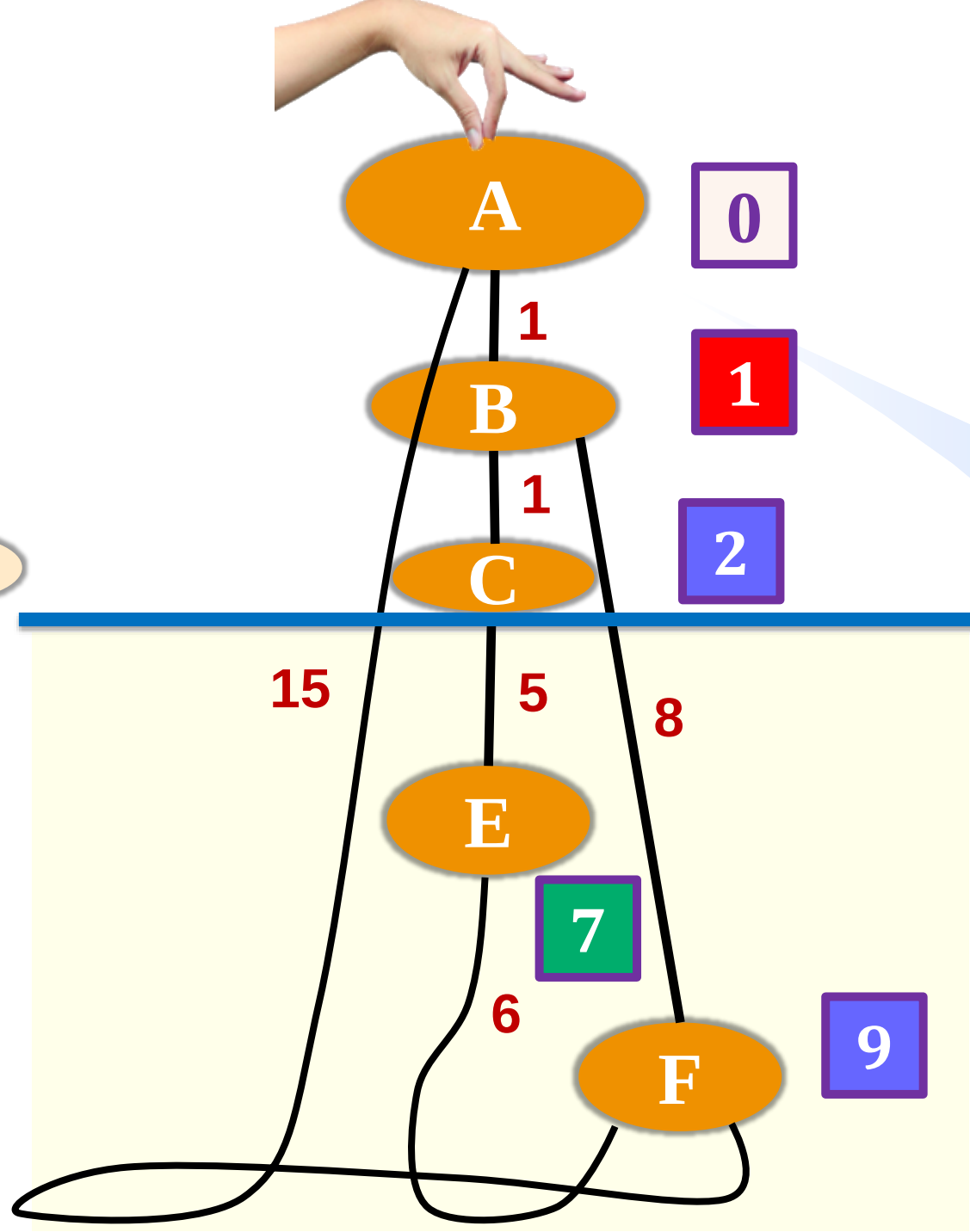
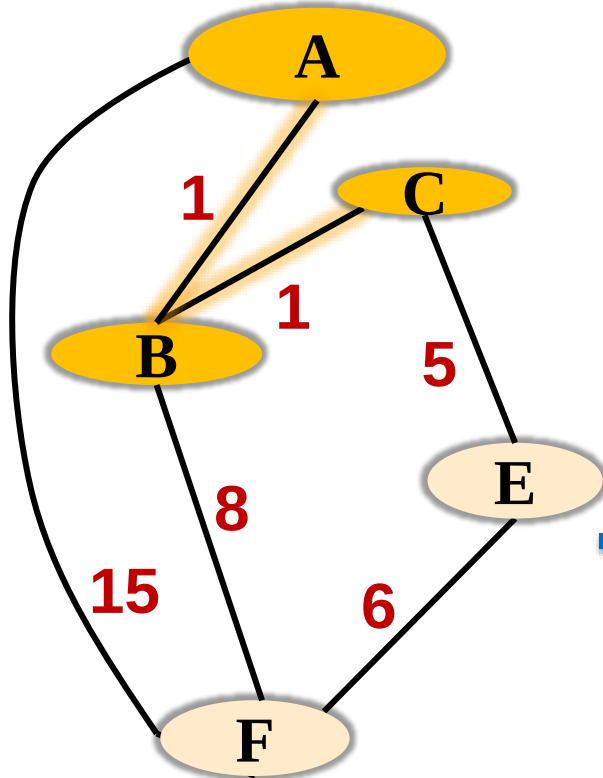


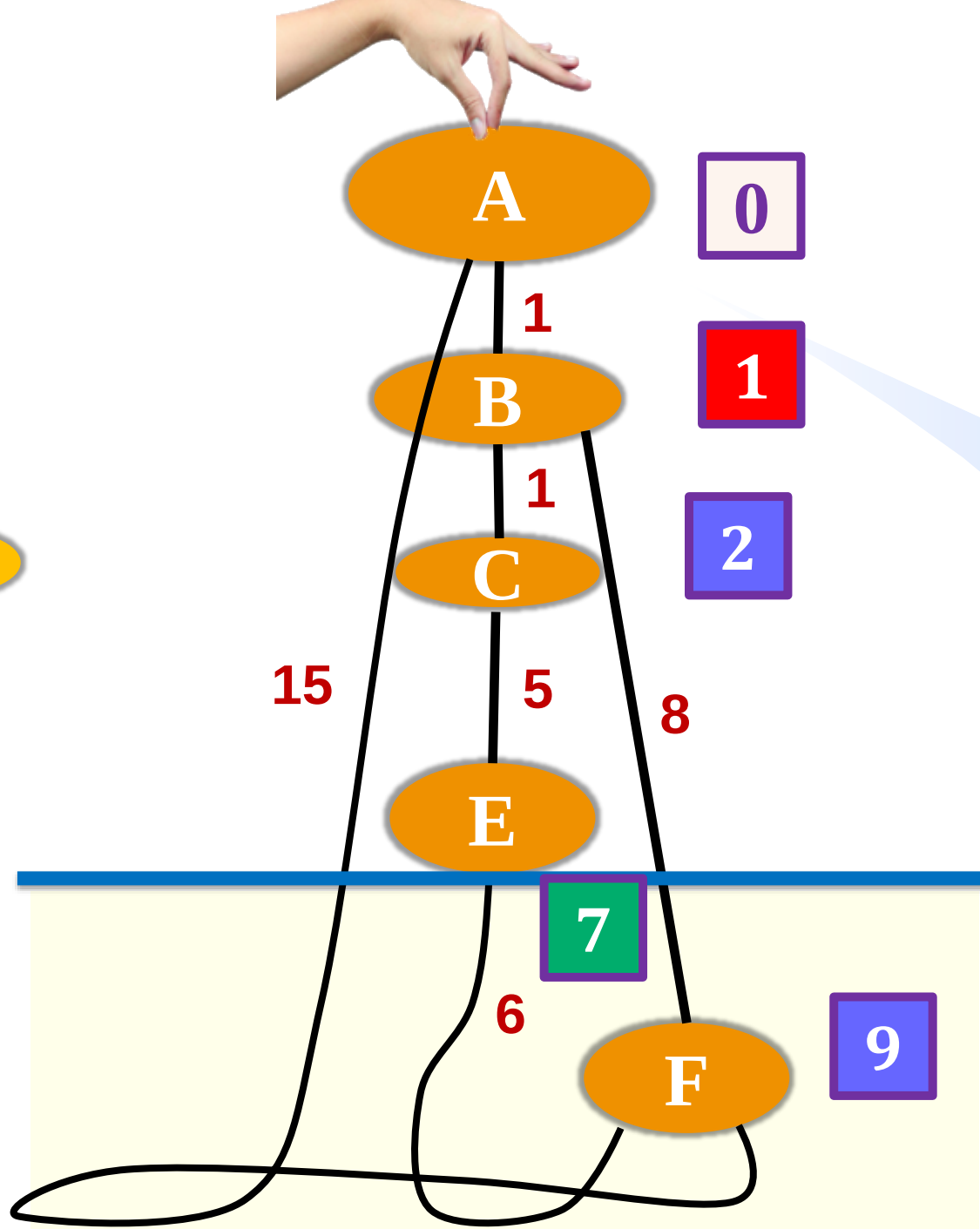
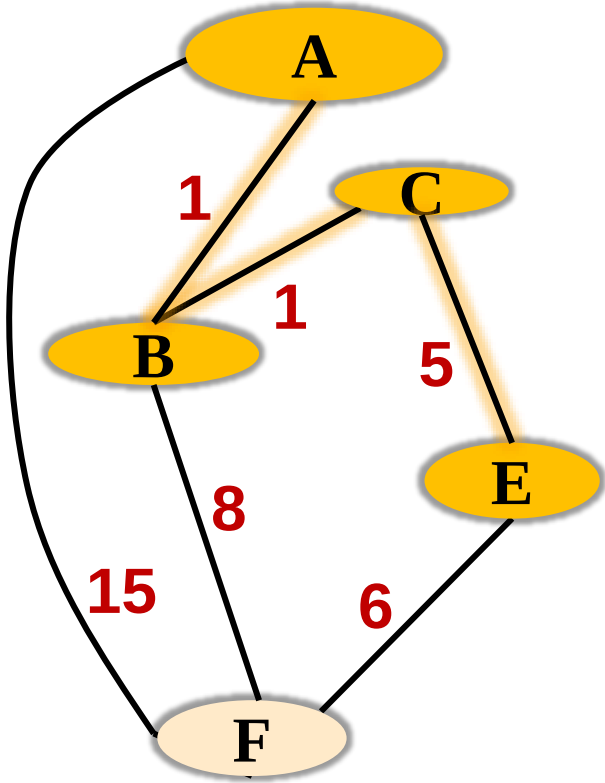












Dijkstra算法

基本思想：将图中所有顶点分成两个集合。

- **集合S**：包括已确定最短路径的顶点（拽起来的顶点）。
- **集合V-S**：包括尚未确定最短路径的顶点（没拽起来的点）。
- 按照与源点**最短距离递增的顺序**逐个把顶点加入集合S。

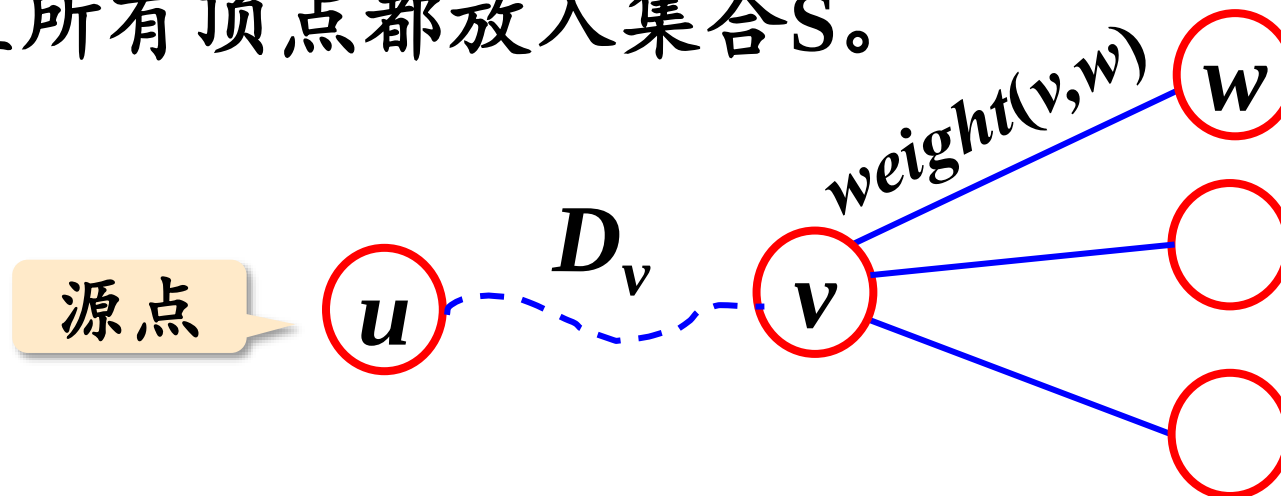
Dijkstra算法

令 D_i 表示从源点到顶点 i 的最短距离，假定顶点 u 为源点；初始时令 $D_u = 0$ 且 $D_i = \infty$ ($\forall i \neq u$)；

①从不在集合 S 中的顶点选择 D_v 最小的顶点 v ，将其放入集合 S （此时即已求出源点到 v 的最短距离，即 D_v ）；

②考察 v 的不在集合 S 中的邻接顶点 w ，若 $D_v + \text{weight}(v, w) < D_w$ ，则更新 D_w ，使 $D_w \leftarrow D_v + \text{weight}(v, w)$ ；

③重复①②，直至所有顶点都放入集合 S 。



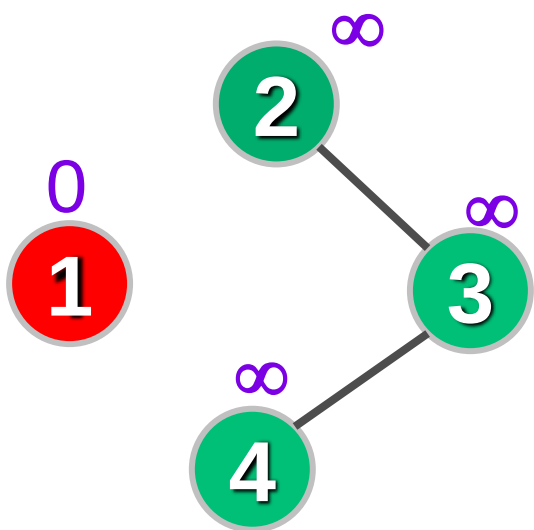
Dijkstra算法实现

- 引入3个辅助数组 $dist[]$ 、 $pre[]$ 、 $S[]$.
- ✓ $dist[i]$: 源点 u 到顶点 i 的最短距离, 初始时 $dist[u]=0$, $dist[i]=\infty$ ($\forall i \neq u$).
- ✓ $pre[i]$: u 到 i 最短路径上 i 的前驱顶点编号, 初始时 $pre[i]=-1$.
- ✓ $S[i]$: 顶点 i 是否已放入集合 S 中, 初始时 $S[i]=0$.

```
const int INF=0X3f3f3f3f, maxn=1e5+10;
```


Dijkstra算法实现

```
void Dijkstra(Vertex *Head, int n, int u, int dist[], int pre[]){
    for(int i=1; i<=n; i++) {pre[i]=-1; dist[i]=INF;} //初始化
    int S[maxn]={0};    dist[u]=0; //起点u的dist值置为0
    for(int i=1; i<=n; i++) {
        int v = findMin(S, dist, n); //从不在S中的顶点里选D值最小的顶点v
        if(v==-1) return; //图不连通, 源点u与剩余顶点不可及, 提前结束算法
        //未完待续...
    }
}
```



```
int findMin(int S[], int dist[], int n){
    int v=-1, min=INF;
    for(int i=1; i<=n; i++)
        if(S[i]==0 && dist[i]<min){
            min=dist[i]; v=i;
        }
    return v;
}
```

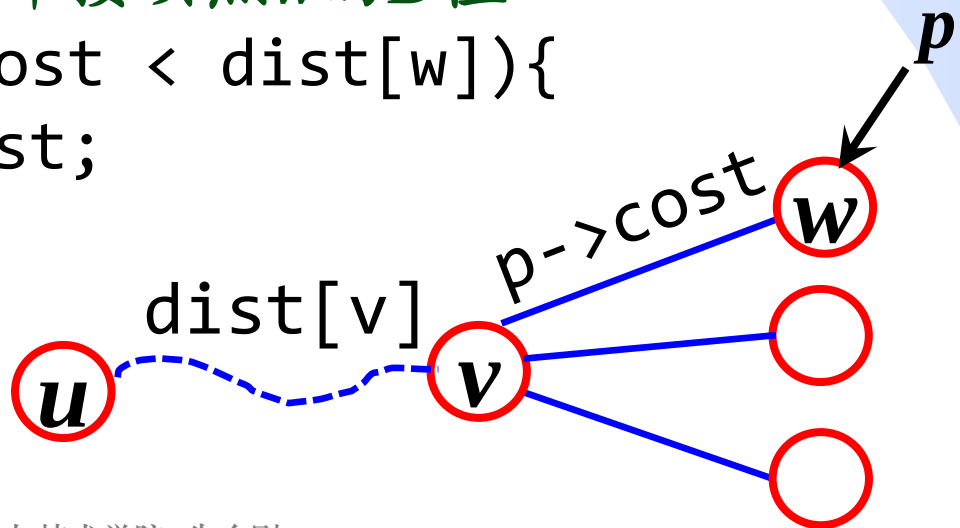
时间复杂度
 $O(n)$

Dijkstra算法实现

```

void Dijkstra(Vertex *Head, int n, int u, int dist[], int pre[]) {
    for(int i=1; i<=n; i++) {pre[i]=-1; dist[i]=INF;} //初始化
    int S[maxn]={0};    dist[u]=0; //起点u的dist值置为0
    for(int i=1; i<=n; i++) {
        int v = findMin(S, dist, n); //从不在S中的顶点里选D值最小的顶点v
        if(v==-1) return; //图不连通，源点u与剩余顶点不可及，提前结束算法
        S[v]=1; //将顶点v放入S集合
        for(Edge* p=Head[v].adjacent; p!=NULL; p=p->link){
            int w=p->VerAdj; //更新v的各邻接顶点w的D值
            if(S[w]==0 && dist[v]+p->cost < dist[w]){
                dist[w] = dist[v]+p->cost;
                pre[w] = v;
            }
        }
    }
}

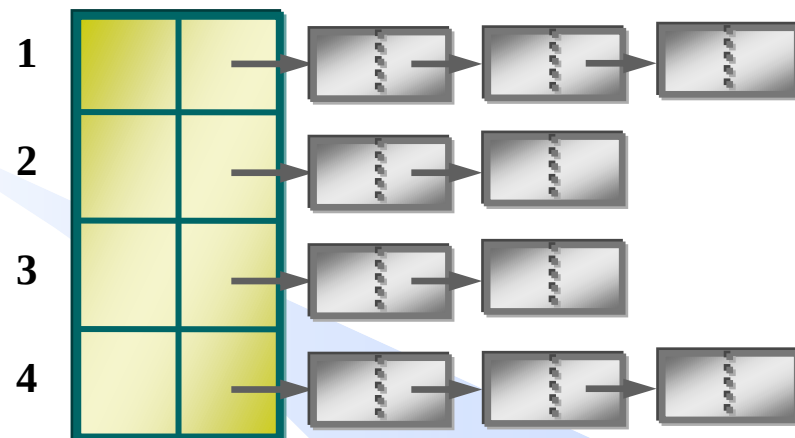
```



Dijkstra算法——时间复杂度

A

```
void Dijkstra(Vertex *Head,int n,int u,int dist[],int pre[]){
    for(int i=1; i<=n; i++) {pre[i]=-1; dist[i]=INF;}
    int S[maxn]={0};    dist[u]=0; //起点u的dist值置为0
    for(int i=1; i<=n; i++) {
        int v = findMin(S, dist, n);
        if(v==-1) return; //图不连通,提前结束算法
        S[v]=1;           //将顶点v放入S集合
        for(Edge* p=Head[v].adjacent; p!=NULL; p=p->link){
            int w=p->VerAdj; //更新v的各邻接顶点w的D值
            if(S[w]==0 && dist[v]+p->cost < dist[w]){
                dist[w] = dist[v]+p->cost;
                pre[w] = v;
            }
        }
    }
}
```



课下思考：若用邻接矩阵存图，时间复杂度是多少？

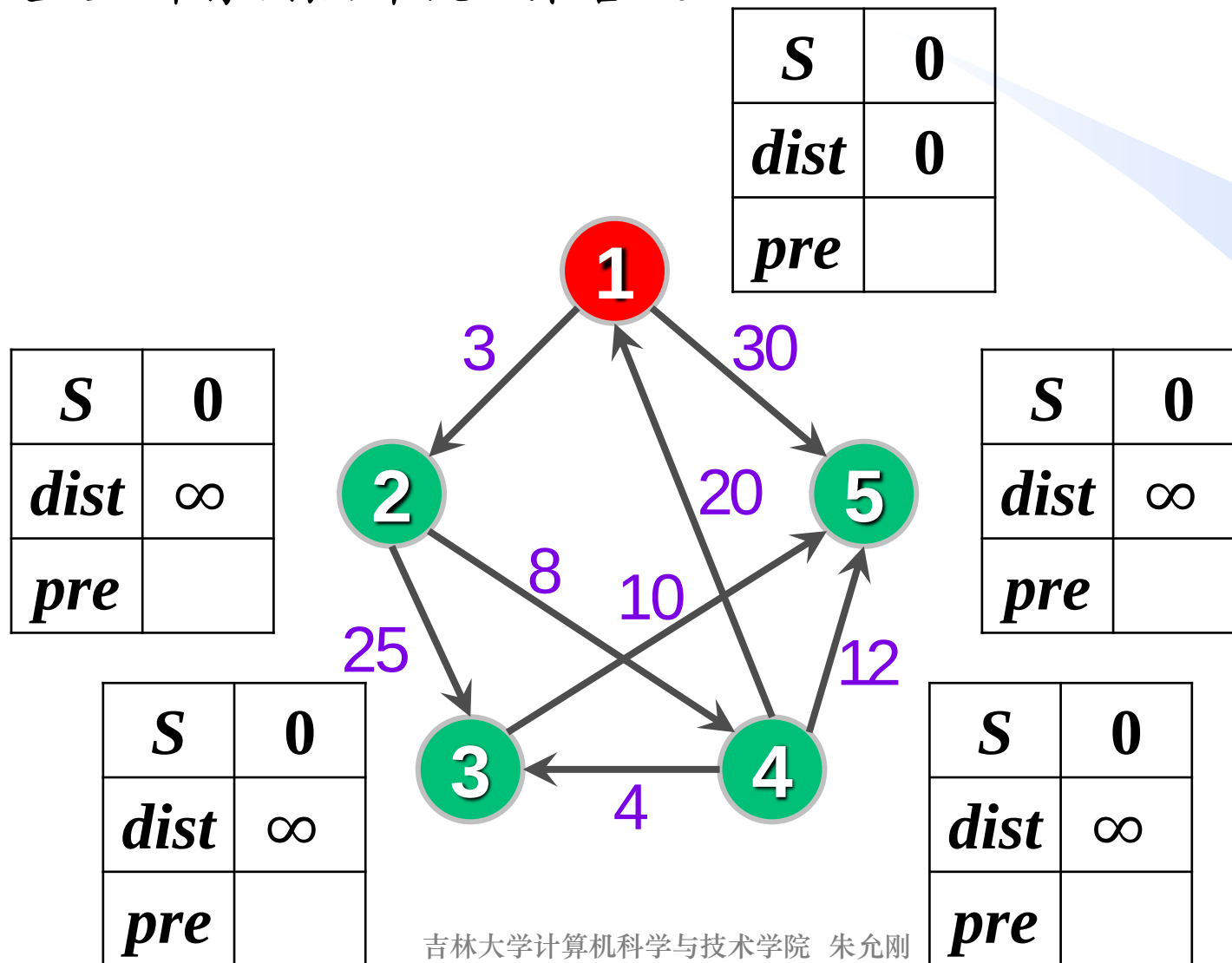
- ✓ 扫描了顶点表中的 n 个顶点，每个顶点花费的时间（findMin）为 $O(n)$ ，总时间 $O(n^2)$
- ✓ 扫描了所有的边结点 $O(e)$
- ✓ 时间复杂度 $O(n^2 + e)$

初始时 (u 为源点), 令 $D_u=0$ 且 $D_i=\infty$ ($\forall i \neq u$);

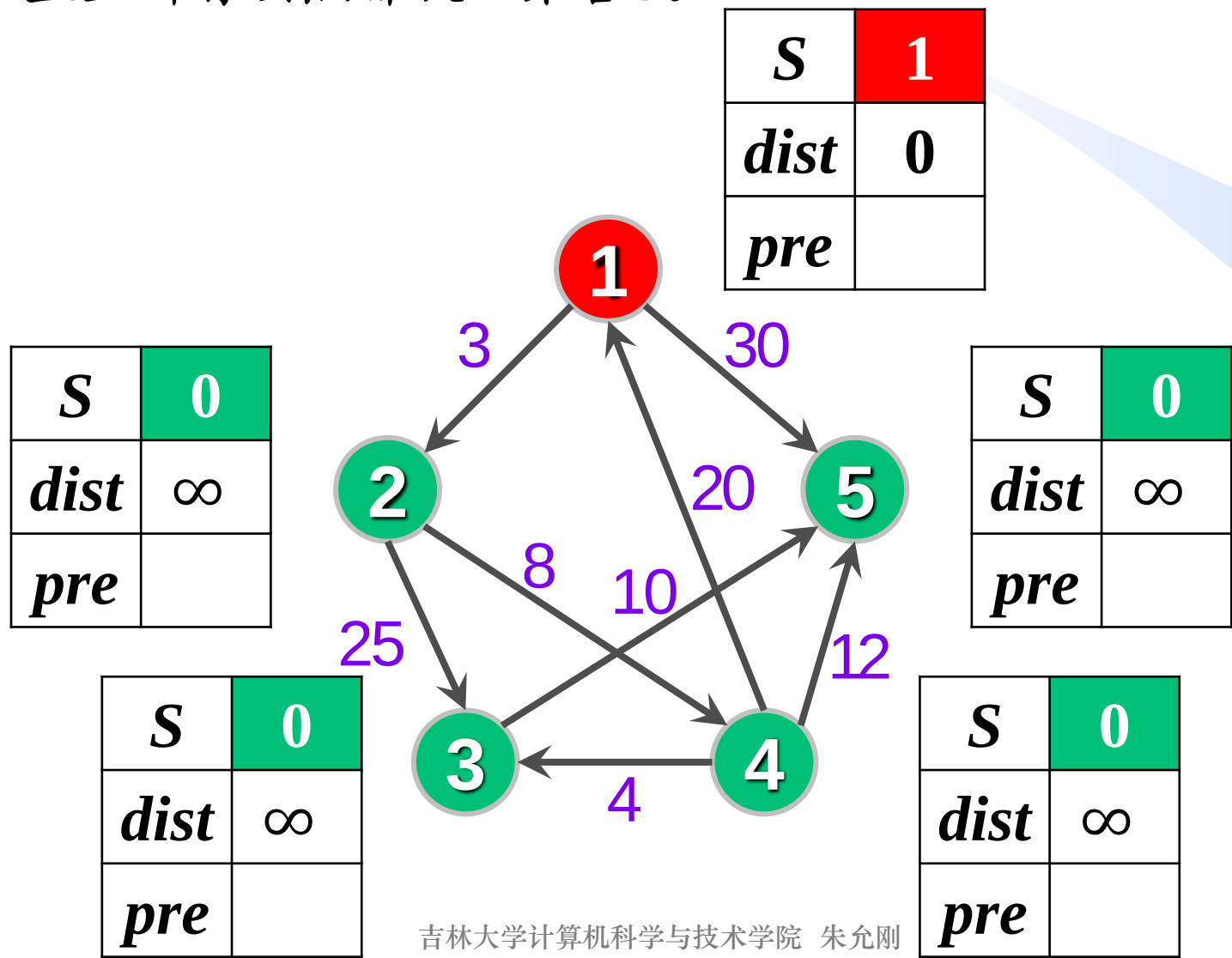
①从不在集合 S 中的顶点选择 D_v 最小的顶点 v , 放入集合 S ;

②更新 v 的不在 S 中的邻接顶点 w , 令 $D_w \leftarrow \min(D_w, D_v + \text{weight}(v, w))$.

③重复① ②, 直至所有顶点都放入集合 S 。



- 初始时 (u 为源点), 令 $D_u=0$ 且 $D_i=\infty$ ($\forall i \neq u$);
- ① 从不在集合 S 中的顶点选择 D_v 最小的顶点 v , 放入集合 S ;
 - ② 更新 v 的不在 S 中的邻接顶点 w , 令 $D_w \leftarrow \min(D_w, D_v + \text{weight}(v, w))$.
 - ③ 重复① ②, 直至所有顶点都放入集合 S 。

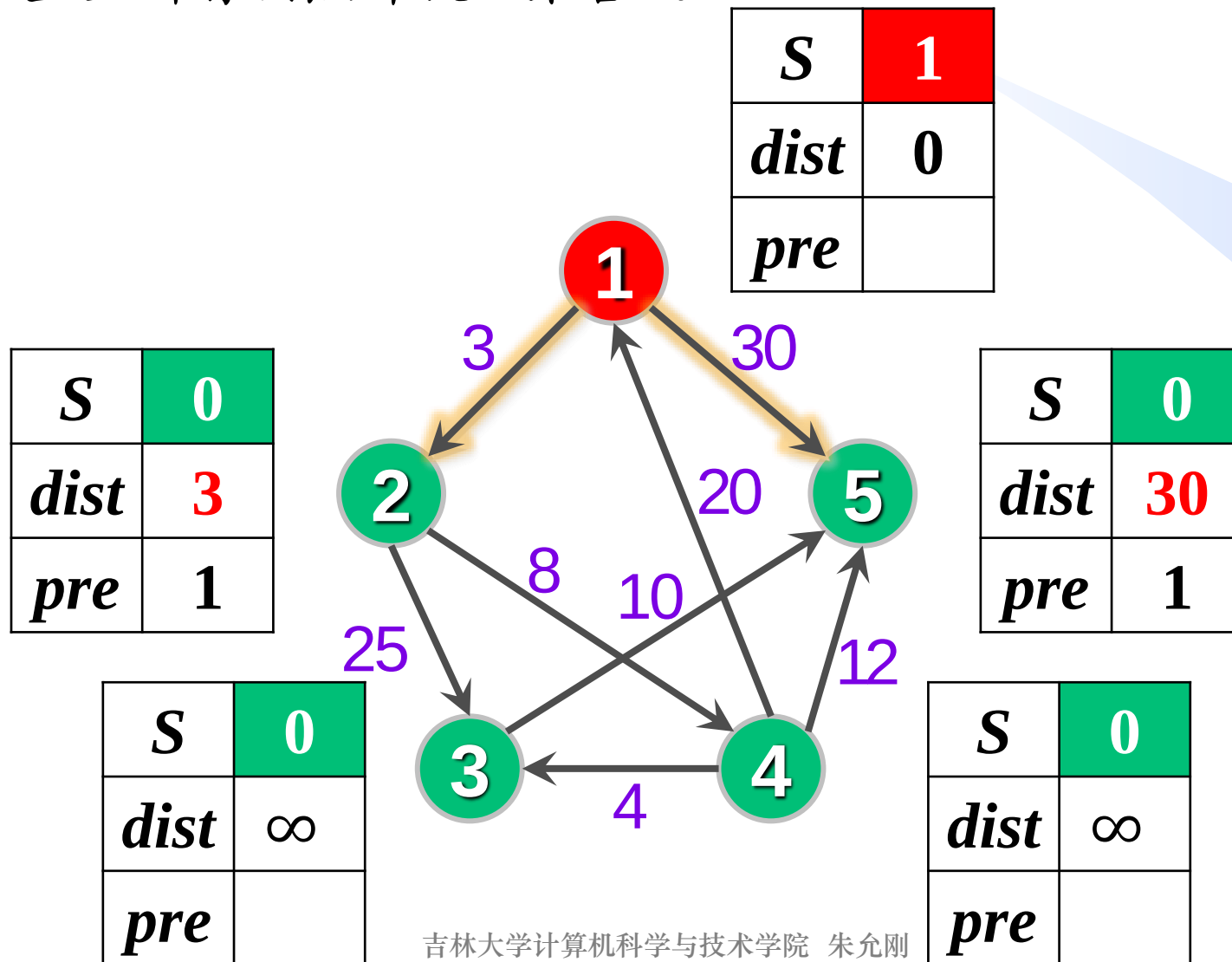


初始时 (u 为源点), 令 $D_u=0$ 且 $D_i=\infty$ ($\forall i \neq u$);

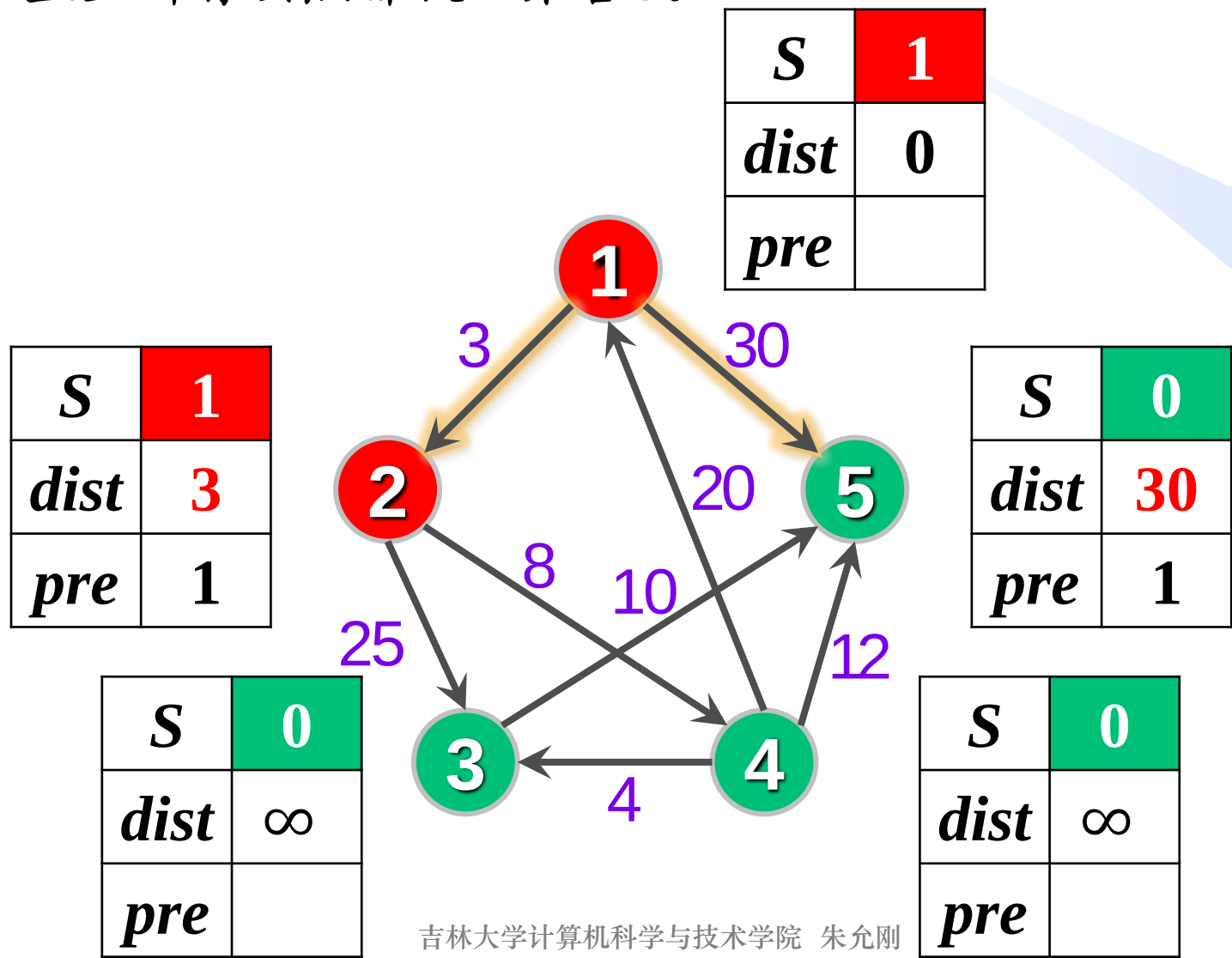
①从不在集合 S 中的顶点选择 D_v 最小的顶点 v , 放入集合 S ;

②更新 v 的不在 S 中的邻接顶点 w , 令 $D_w \leftarrow \min(D_w, D_v + \text{weight}(v, w))$.

③重复① ②, 直至所有顶点都放入集合 S 。



- 初始时 (u 为源点), 令 $D_u=0$ 且 $D_i=\infty$ ($\forall i \neq u$);
- ①从不在集合S中的顶点选择 D_v 最小的顶点 v , 放入集合S;
 - ②更新 v 的不在S中的邻接顶点 w , 令 $D_w \leftarrow \min(D_w, D_v + \text{weight}(v,w))$.
 - ③重复① ②, 直至所有顶点都放入集合S。

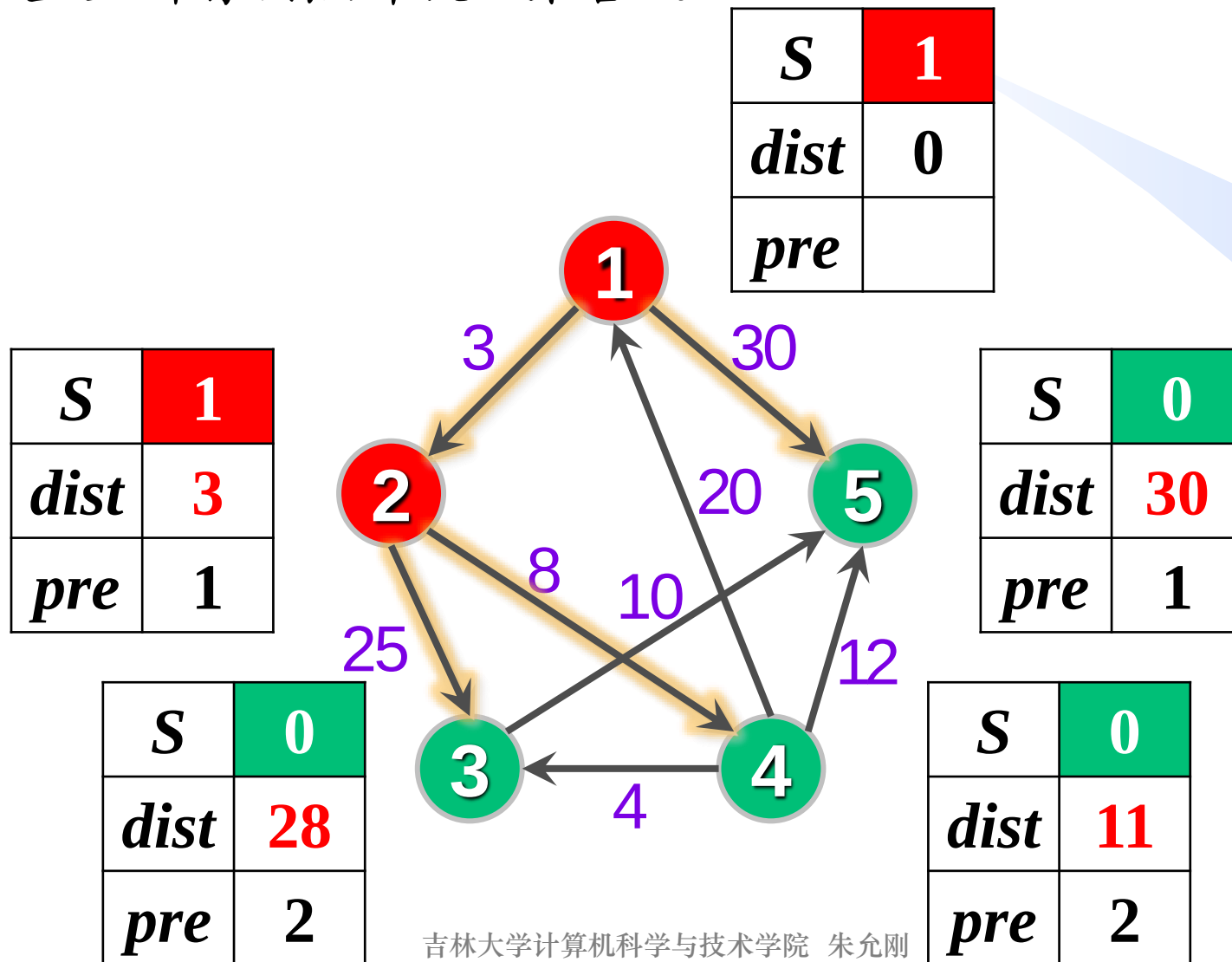


初始时 (u 为源点), 令 $D_u=0$ 且 $D_i=\infty$ ($\forall i \neq u$);

①从不在集合 S 中的顶点选择 D_v 最小的顶点 v , 放入集合 S ;

②更新 v 的不在 S 中的邻接顶点 w , 令 $D_w \leftarrow \min(D_w, D_v + \text{weight}(v, w))$.

③重复① ②, 直至所有顶点都放入集合 S 。

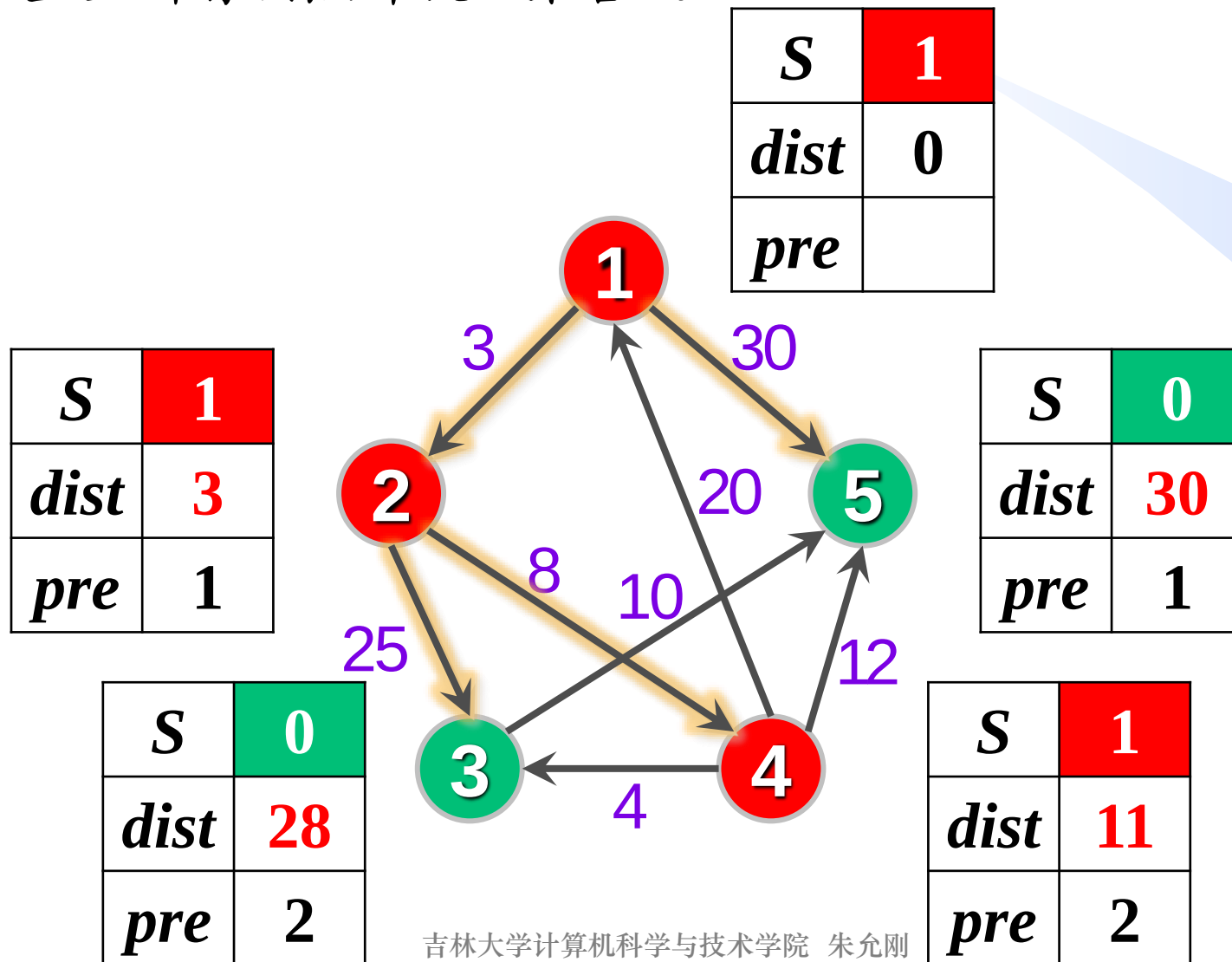


初始时 (u 为源点), 令 $D_u=0$ 且 $D_i=\infty$ ($\forall i \neq u$);

①从不在集合 S 中的顶点选择 D_v 最小的顶点 v , 放入集合 S ;

②更新 v 的不在 S 中的邻接顶点 w , 令 $D_w \leftarrow \min(D_w, D_v + \text{weight}(v, w))$.

③重复① ②, 直至所有顶点都放入集合 S 。

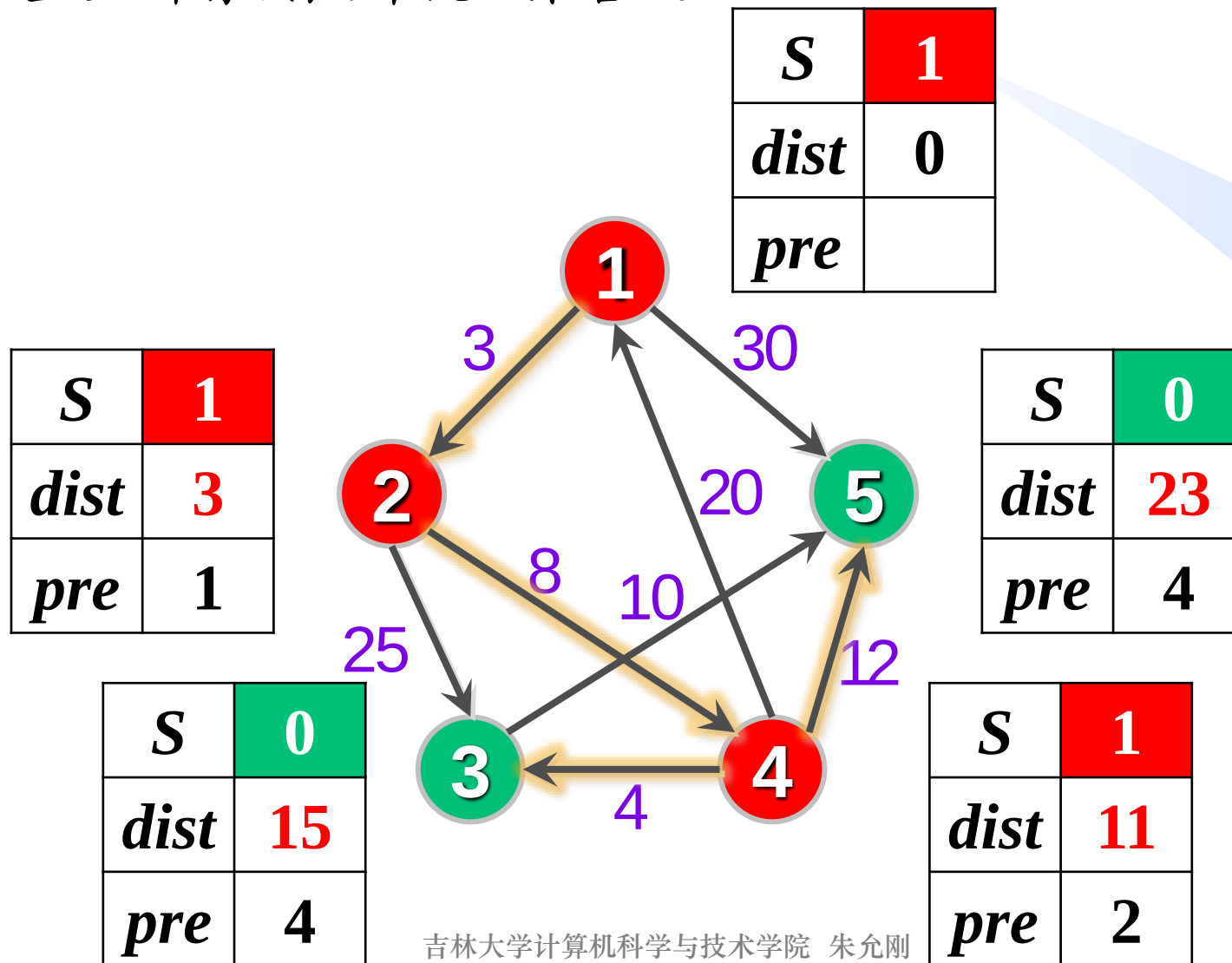


初始时 (u 为源点), 令 $D_u=0$ 且 $D_i=\infty$ ($\forall i \neq u$);

①从不在集合 S 中的顶点选择 D_v 最小的顶点 v , 放入集合 S ;

②更新 v 的不在 S 中的邻接顶点 w , 令 $D_w \leftarrow \min(D_w, D_v + \text{weight}(v, w))$.

③重复① ②, 直至所有顶点都放入集合 S 。

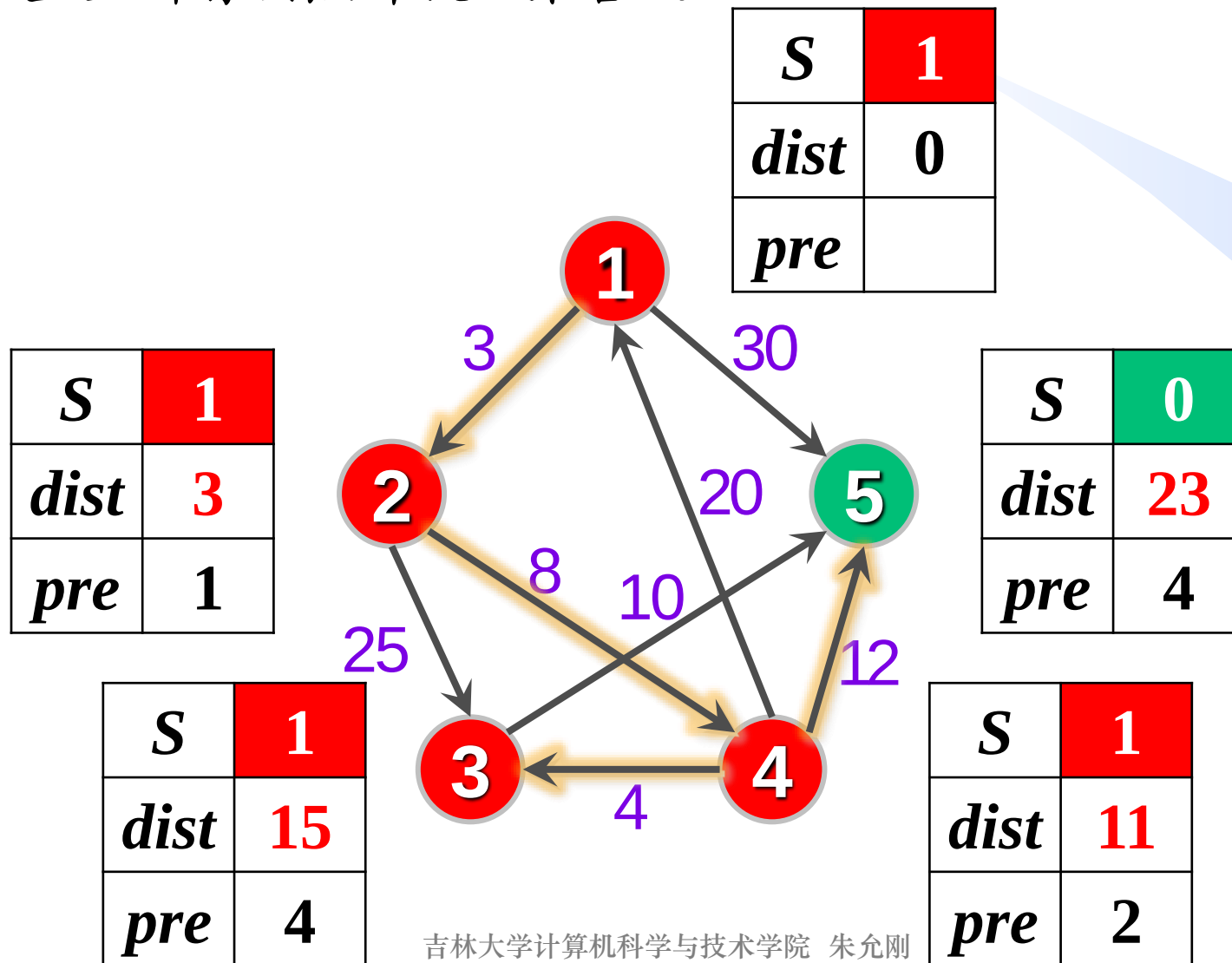


初始时 (u 为源点), 令 $D_u=0$ 且 $D_i=\infty$ ($\forall i \neq u$);

①从不在集合 S 中的顶点选择 D_v 最小的顶点 v , 放入集合 S ;

②更新 v 的不在 S 中的邻接顶点 w , 令 $D_w \leftarrow \min(D_w, D_v + \text{weight}(v, w))$.

③重复① ②, 直至所有顶点都放入集合 S 。

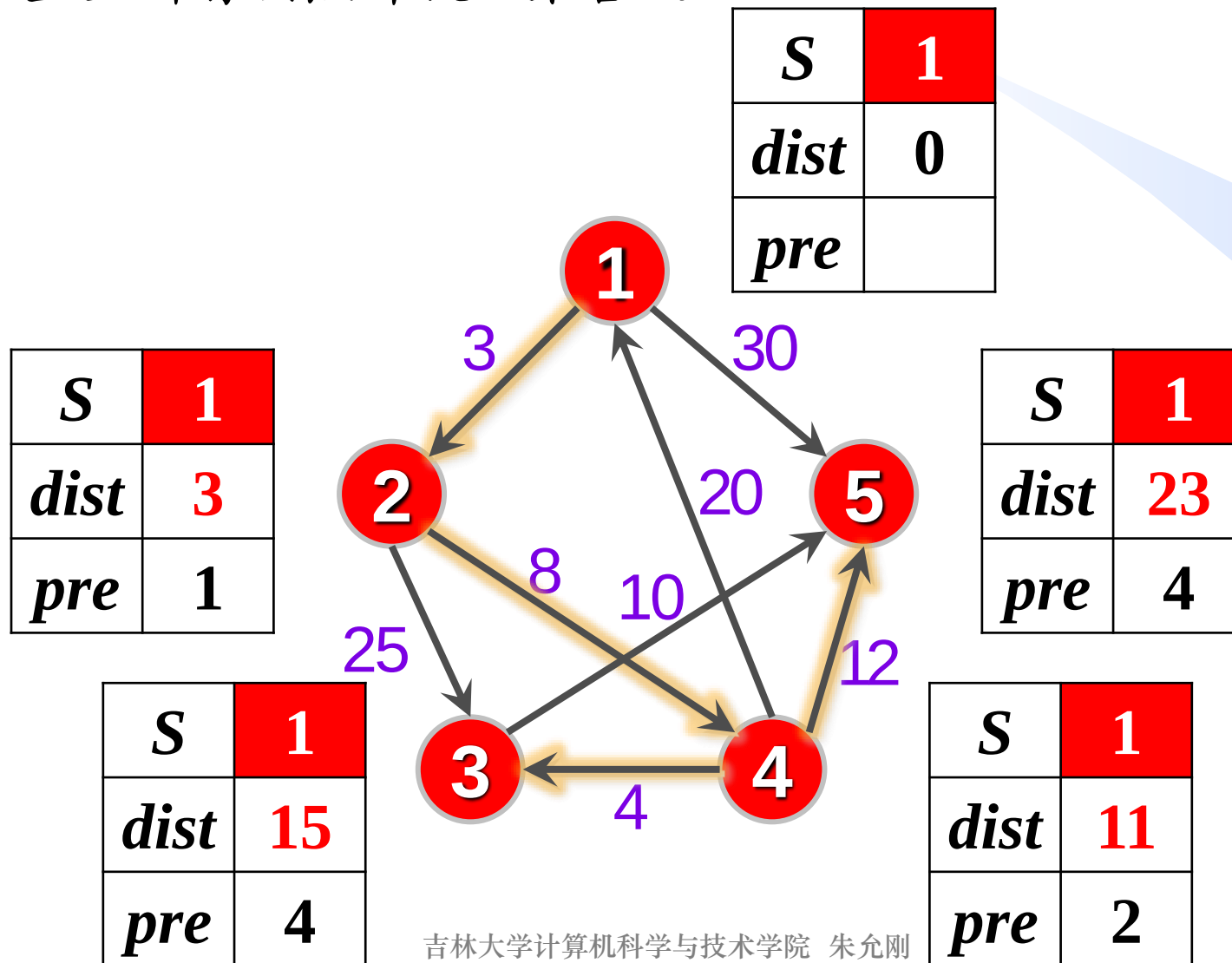


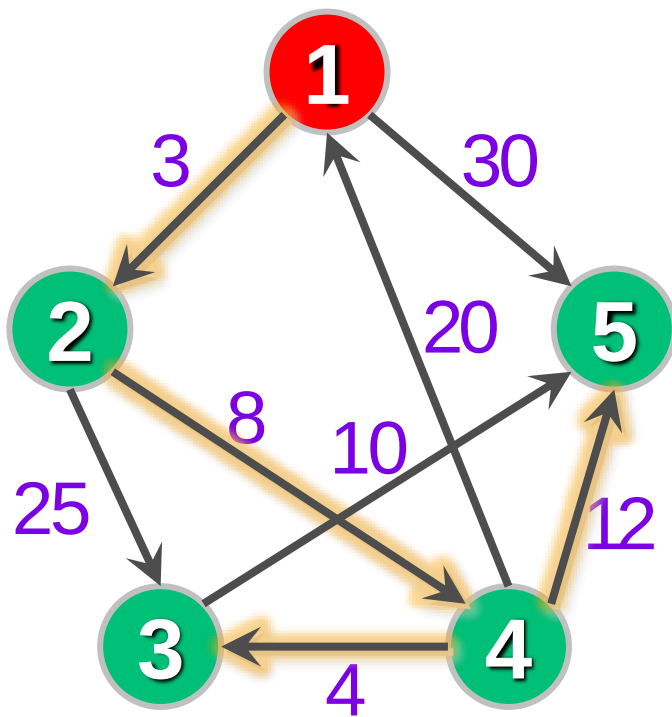
初始时 (u 为源点), 令 $D_u=0$ 且 $D_i=\infty$ ($\forall i \neq u$);

①从不在集合 S 中的顶点选择 D_v 最小的顶点 v , 放入集合 S ;

②更新 v 的不在 S 中的邻接顶点 w , 令 $D_w \leftarrow \min(D_w, D_v + \text{weight}(v, w))$.

③重复① ②, 直至所有顶点都放入集合 S 。



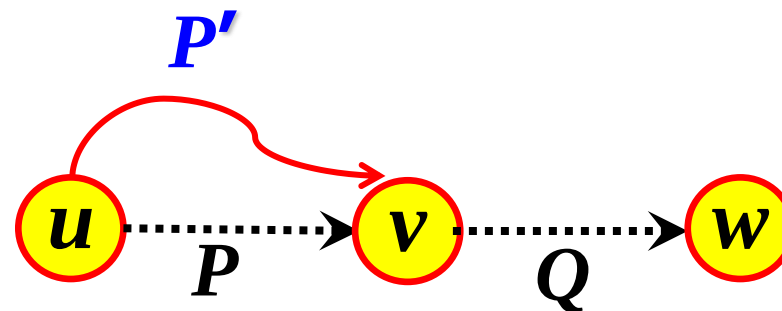


	1	2	3	4	5
S	1	1	1	1	1
$dist$	0	3	15	11	23
pre		1	4	2	4

源点到 v_5 的最短路

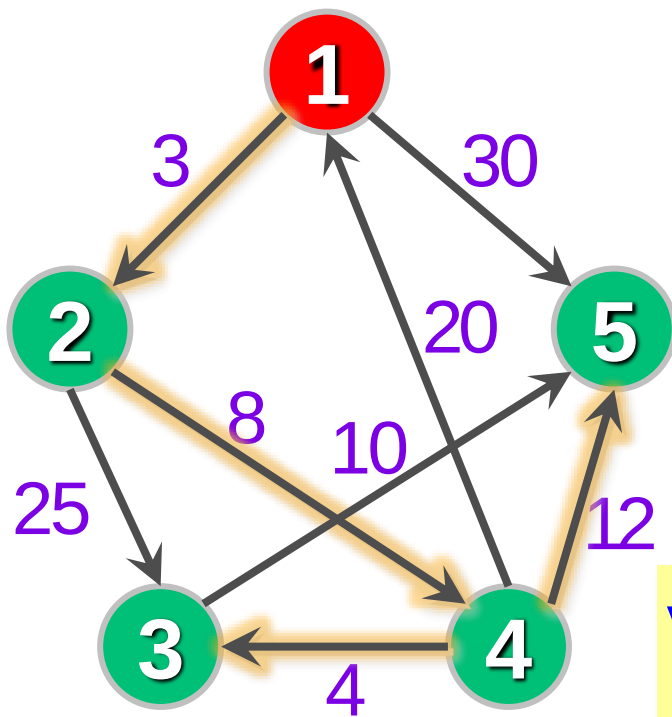
① ② ④ ⑤

pre 数组优势：可存源点到其他所有点的最短路



原理：任意最短路的前缀，也是一条最短路

反证法：若 u 到 v 有一条更短的路径 P'



	1	2	3	4	5
<i>S</i>	1	1	1	1	1
<i>dist</i>	0	3	15	11	23
<i>pre</i>		1	4	2	4

源点到 v_5 的最短路

① ② ④ ⑤

*pre*数组优势：可存源点到其他所有点的最短路

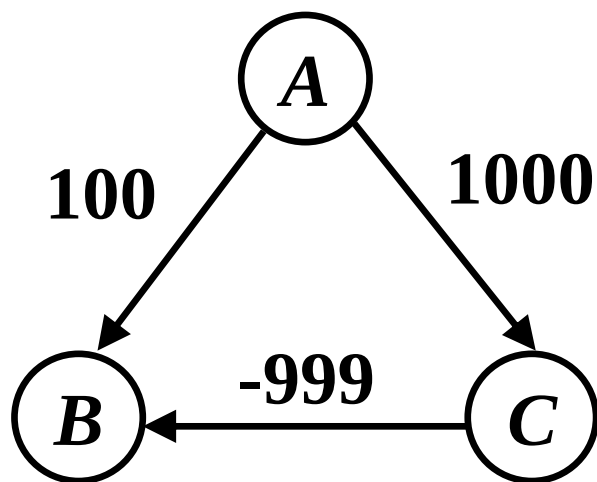
```
void PrintPath(int pre[], int s, int t){
    //输出s到t的最短路
    if(s==t) {printf("%d ",s); return;}
    PrintPath(pre,s,pre[t]); //输出s到t前驱的最短路
    printf("%d ",t);        //输出t
}
```

Dijkstra算法总结

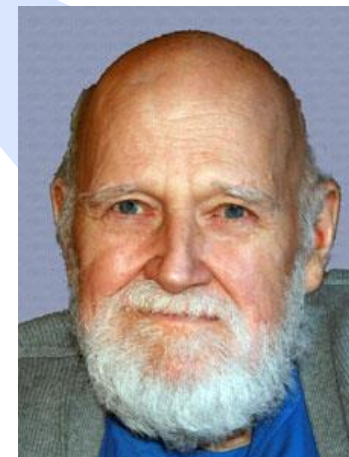
- 适用于有向图，也适用于无向图；
- 算法运行过程中，一旦顶点 v 加入集合 S ，源点到 v 的最短路径既已确定；
- 按照最短距离**递增**顺序依次把顶点加入集合 S ；
- 也可以求源点 s 到某一个顶点 t 的最短路；
- 用贪心策略的“形”，包裹了动态规划的“魂”；
- 使用**斐波那契堆**，算法时间复杂度理论上可达 $O(n\log n + e)$ 。

Dijkstra算法总结

- 能处理非负权图，不能处理负权图：在负权图中选出 D_v 值最小的顶点 v 加入集合 S 后，源点到 v 的最短距离仍不确定。
- 对于负权图的单源最短路径问题，可使用Bellman-Ford算法，时间复杂度 $O(ne)$ ，感兴趣的同学可查阅相关文献。



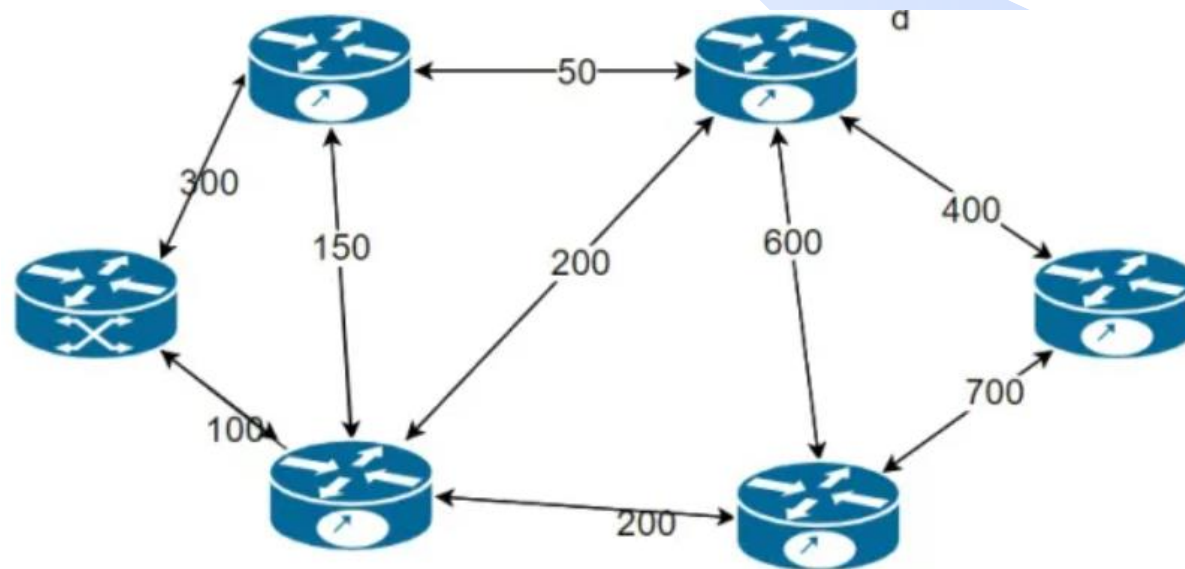
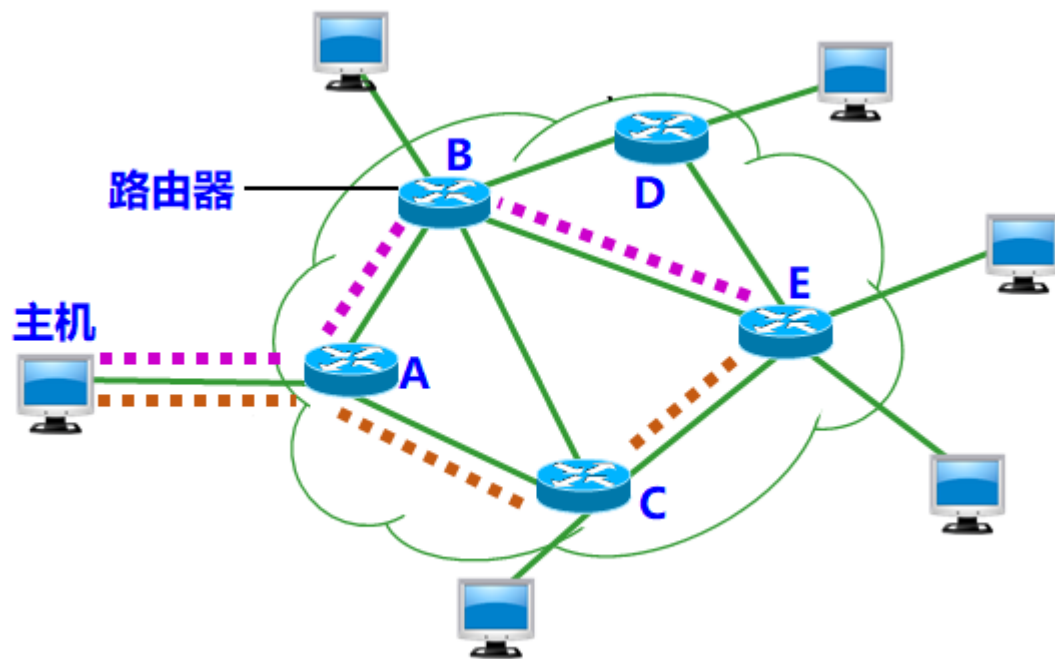
Richard Bellman
南加州大学教授
美国科学院/工程院院士
动态规划之父



Lester Ford
美国数学家

最短路径算法的应用举例——网络路由协议

- OSPF路由协议：基于Dijkstra算法
- RIP、BGP路由协议：基于Bellman-Ford算法



最新研究进展举例

D



段然研究组提出新的无向图单源最短路算法，首次在实数边权上突破了排序

时限

浏览量: 1653

2023年07月31日

近日，交叉信息院段然研究组提出新的无向图单源最短路算法，首次在实数边权上突破了Dijkstra算法的排序时限。该论文《A Randomized Algorithm for Single-Source Shortest Path on Undirected Real-Weighted Graphs》已被FOCS 2023接收。

最短路问题是图论领域最基础的问题之一，单源最短路问题需要找到从一点s到其他所有点的最短路。Dijkstra算法（利用斐波那契堆）的时间复杂度为 $O(m+n\log n)$ ，因为Dijkstra算法的副产品是所有点对从s的距离排序，而比较模型下排序需要 $\Omega(n\log n)$ 时间，所以要改进Dijkstra算法就要避免整体排序。在比较模型下，对于另一个图论基础问题——最小生成树，姚期智院士早在1975年就给出了比排序时间快的算法，而目前已有 $O(m)$ 时间的随机算法；但对于最短路问题却一直没有突破排序时间的算法。

Dijkstra算法每次从优先级队列（斐波那契堆）中取出目前距离最短的点，然后扫描这个点的边；而新的算法只随机取一部分点放到斐波那契堆中（称为点集R），而其他点v都绑定到离它最近的R点（称为 $b(v)$ ），并且找到距离v比 $b(v)$ 更近的所有点，称为 $Ball(v)$ 。如果R的大小为 n/k ，则 $Ball(v)$ 的平均大小为 $O(k)$ 。（这里m为图的边数、n为点数）

A Randomized Algorithm for Single-Source Shortest Path on Undirected Real-Weighted Graphs

Ran Duan ^{*1}, Jiayi Mao ^{†1}, Xinkai Shu ^{‡2}, and Longhui Yin ^{§1}

¹Institute for Interdisciplinary Information Sciences, Tsinghua University

²The University of Hong Kong

October 5, 2023

Abstract

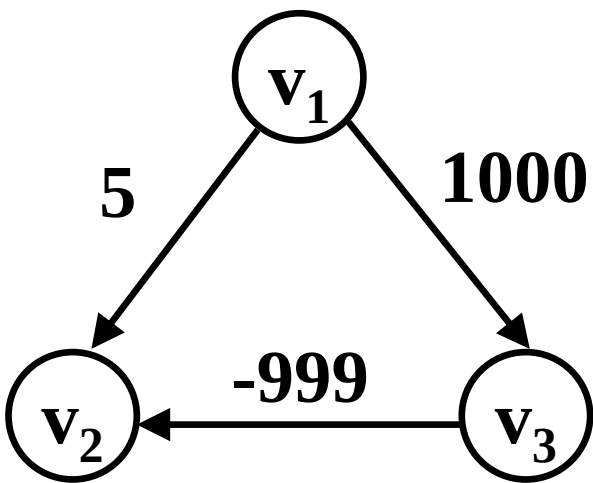
In undirected graphs with real non-negative weights, we give a new randomized algorithm for the single-source shortest path (SSSP) problem with running time $O(m\sqrt{\log n \cdot \log \log n})$ in the *comparison-addition* model. This is the first algorithm to break the $O(m + n \log n)$ time bound for real-weighted sparse graphs by Dijkstra's algorithm with Fibonacci heaps. Previous undirected non-negative SSSP algorithms give time bound of $O(m\alpha(m, n) + \min\{n \log n, n \log \log r\})$ in comparison-addition model, where α is the inverse-Ackermann function and r is the ratio of the maximum-to-minimum edge weight [Pettie & Ramachandran 2005], and linear time for integer edge weights in RAM model [Thorup 1999]. Note that there is a proposed complexity lower bound of $\Omega(m + \min\{n \log n, n \log \log r\})$ for hierarchy-based algorithms for undirected real-weighted SSSP [Pettie & Ramachandran 2005], but our algorithm does not obey the properties required for that lower bound. As a non-hierarchy-based approach, our algorithm shows great advantage with much simpler structure, and is much easier to implement.

FOCS(IEEE Annual Symposium on Foundations of Computer Science)是计算机领域最顶级的国际会议之一，并被公认为难度最高的会议之一。会议涵盖的领域包括算法和数据结构、计算复杂性等。

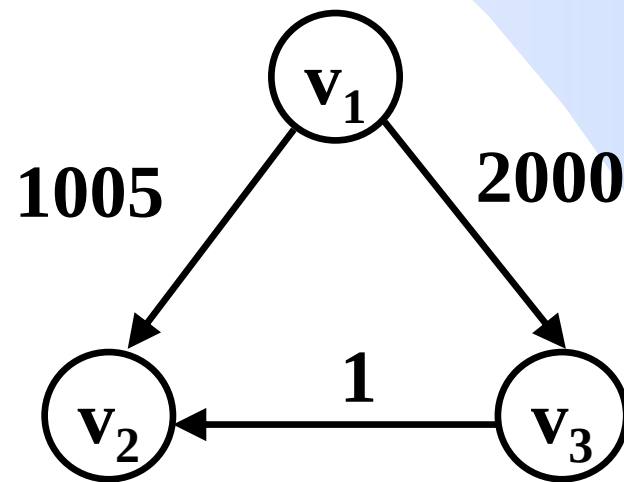
<https://iiis.tsinghua.edu.cn/show-10428-1.html>

课下思考

对于负权图最短路径问题，可否采用如下方案：若图中最小边权是 $-x$ ($x>0$)，做一个变换，将所有边的权值加上 $x+1$ ，使所有边权值变为正数，再调用Dijkstra求最短路径，该最短路径就是原图的最短路。

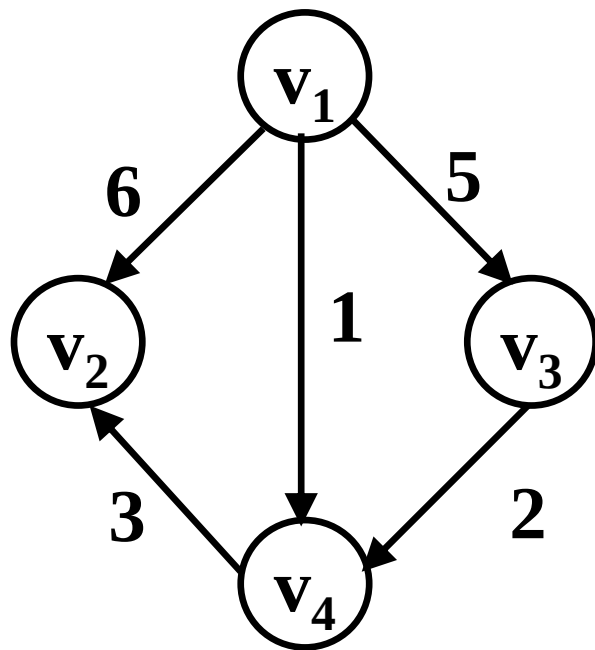


权值加1000



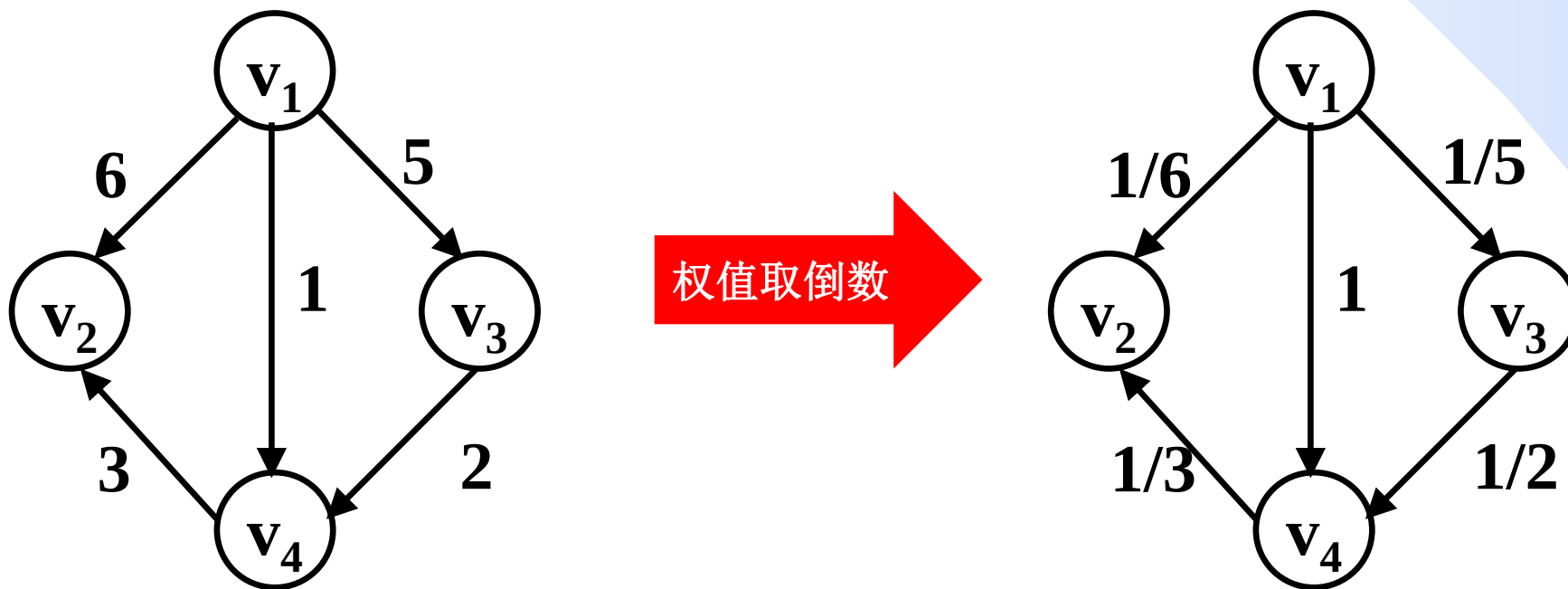
课下思考

求图的最长路径，可否采用如下方案：对Dijkstra算法稍加修改，将算法中 $D_v + \text{weight}(v, w) < D_w$ 改为 $D_v + \text{weight}(v, w) > D_w$ 。



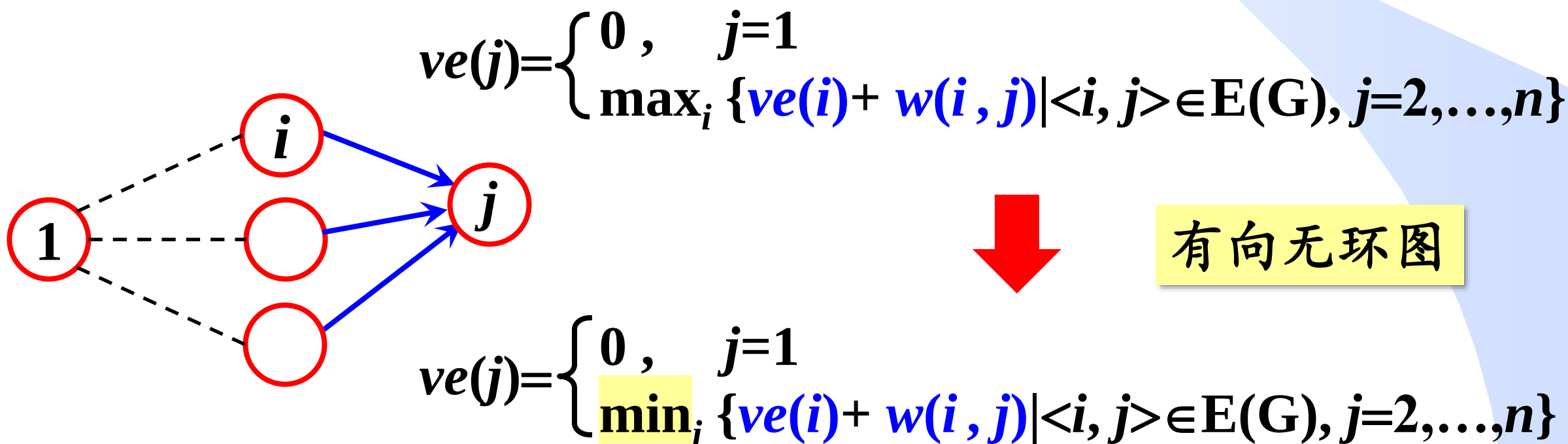
课下思考

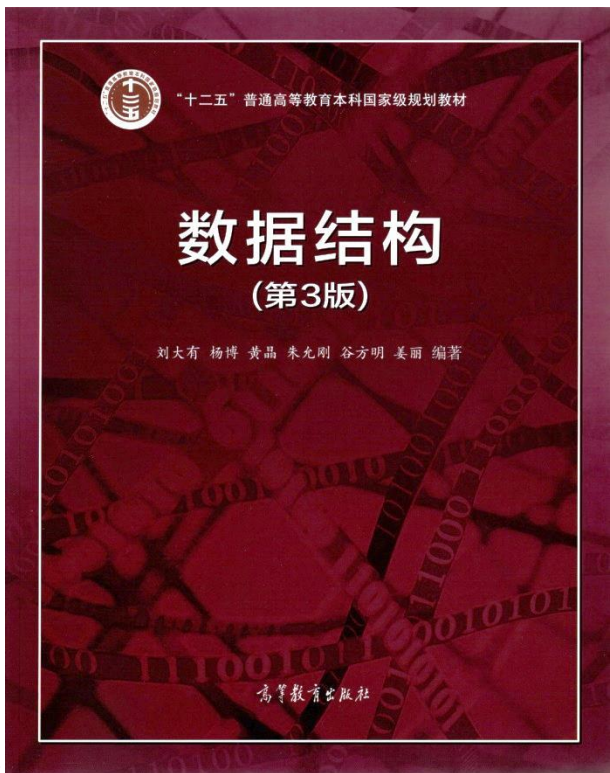
求图的最长路径，可否采用如下方案：若权值都为正，则对所求权值取倒数，用Dijkstra算法求最短路径，该路径就是原图的最长路径。



课下思考

求图的单源最短路径，可否采用如下方案：将关键路径算法中求 ve 值的算法中 $\max$ 修改为 $\min$ ，则 $ve(j)$ 即源点到顶点 j 的最短路径长度，时间复杂度 $O(n+e)$ 。若该方案可行，那么 $O(n^2+e)$ 的Dijkstra算法的意义何在？





图的最短路径

- 无权图最短路径
- Dijkstra算法
- **A*算法初探**
- Floyd算法
- 满足约束的最短路径

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

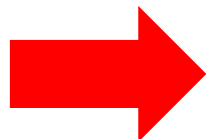
游戏中的自动寻路



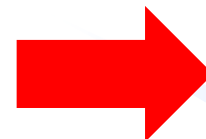
起点 s 到目标点 t 的最短路径是该问题的最优解。

最短路径问题

无权图的单
源最短路径
问题



正权图的单
源最短路径
问题



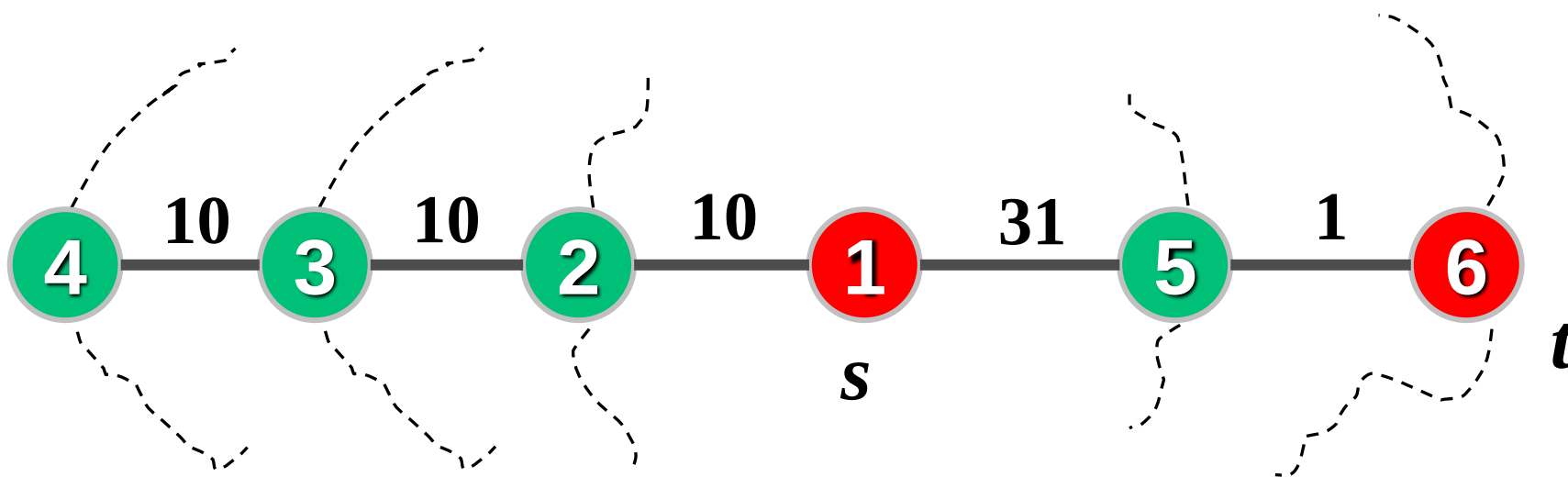
正权图单源
单目标点最
短路径问题



找北京到长春
的最短路
若采用Dijkstra
算法.....

例：找顶点1到顶点6的最短路

- 按照Dijkstra算法，则搜索顺序2、3、4.....
- 原因：Dijkstra只考虑顶点与起点的距离，未考虑顶点与目标点的距离，搜索过程中略显盲目。
- 如何避免跑偏？把顶点到目标点的距离也考虑进去。



A*算法

$$f(v) = g(v) + h(v)$$

估价函数

启发函数

- $g(v)$: 起点到顶点 v 的最短距离, 即 $\text{dist}[v]$ 。
- $h(v)$: 启发函数, 顶点 v 到目标点距离的**估计值**。
- 算法运行过程中, 每次**选 f 值最小**的顶点 v 。
- A*算法属于启发式搜索算法, 将其用于图搜索问题, 相当于是对Dijkstra算法的改进。

A*算法

$$f(v) = g(v) + h(v)$$

估价函数

启发函数

- 若 $h(v) \leq h^*(v)$ ，则可获得最优解，否则无法确保获得最优解，只能获得近似解，其中 $h^*(v)$ 为 v 到目标点的实际最优距离。
- 实际面对的往往是非常大的地图和海量的寻路请求，对算法时间效率要求高。并不一定非要求最优解（最短路），在结果质量和时间效率做一个折中，选择次优解，在实际开发中的应用更加广泛。

地图搜索

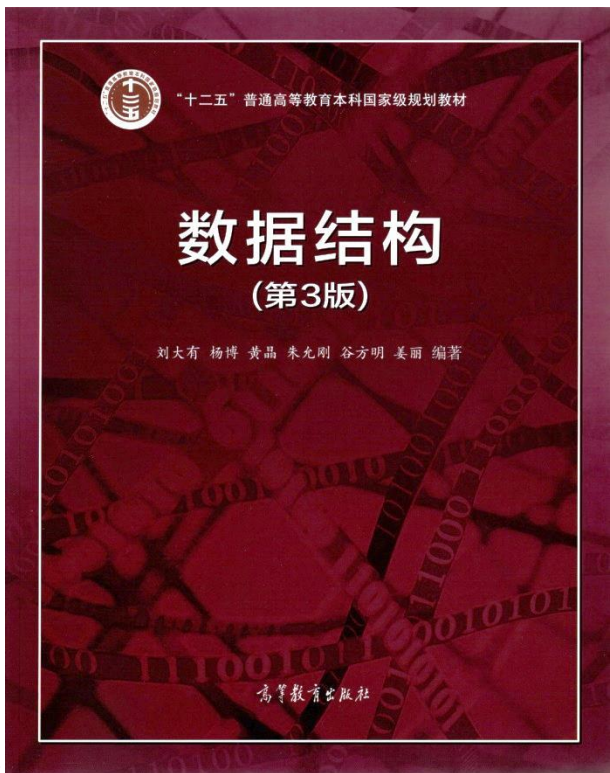
- 将地图抽象为一个有向带权图：
- 把每个岔路口看作一个顶点，岔路口与岔路口之间的道路看作一条边，路的长度就是边的权值。
- 如果路是单行道，可以在两个顶点之间画一条有向边，如果是双行道，就在两个顶点之间画两条方向不同的边。
- 执行最短路径算法。
- 若需规划“所需时间”最少的路径，算法不变，只需把边的权值设置为通行这段道路所需要的时间（会根据拥堵情况时刻变化）。



地图搜索

- 对于一个大地图来说，岔路口、道路都非常多，即顶点和边很多。
- 两点之间的最短路径或者说较好的出行线路，一般只会出现在两点之间和两点附近的区域内。
- 可以在整个大地图上，划出一个小的区域，这个小区块恰好可以覆盖住两个点，但又不会很大。
- 在这个小区域内部运行最短路径算法，避免遍历整个大图，提高效率。





图的最短路径

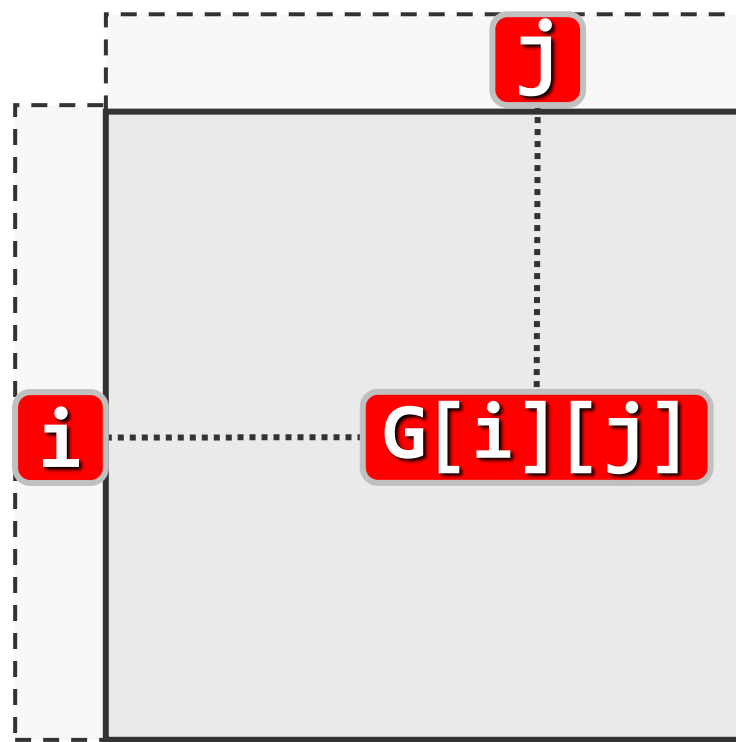
- 无权图最短路径
- Dijkstra算法
- A*算法
- **Floyd算法**
- 满足约束的最短路径

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

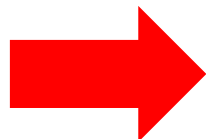
每对顶点间的最短路径

- **问题：** G 是带权图，对每一对顶点 $v_i \neq v_j$ ，求 v_i 与 v_j 间的最短路径。
- **方案一：**若权值为正，可依次把每个顶点作为源点，执行 Dijkstra 算法。
- **方案二：**Floyd 算法

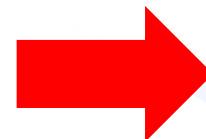


问题

无权图的单
源最短路径
问题



正权图的单
源最短路径
问题



正权图单源
单目标点最
短路径问题



任意两点间
的最短路径
问题

Floyd算法



Robert W. Floyd（弗洛伊德）

斯坦福大学教授

图灵奖获得者

**文学转专业计算机
成功逆袭终获图灵奖**

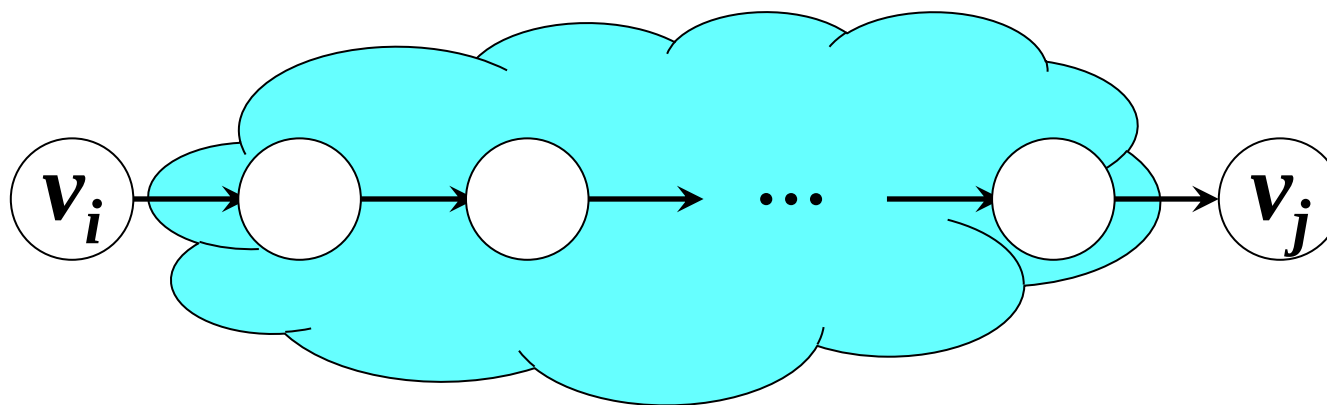
Floyd算法

可以把 v_i 到 v_j 的最短
路径分为3部分

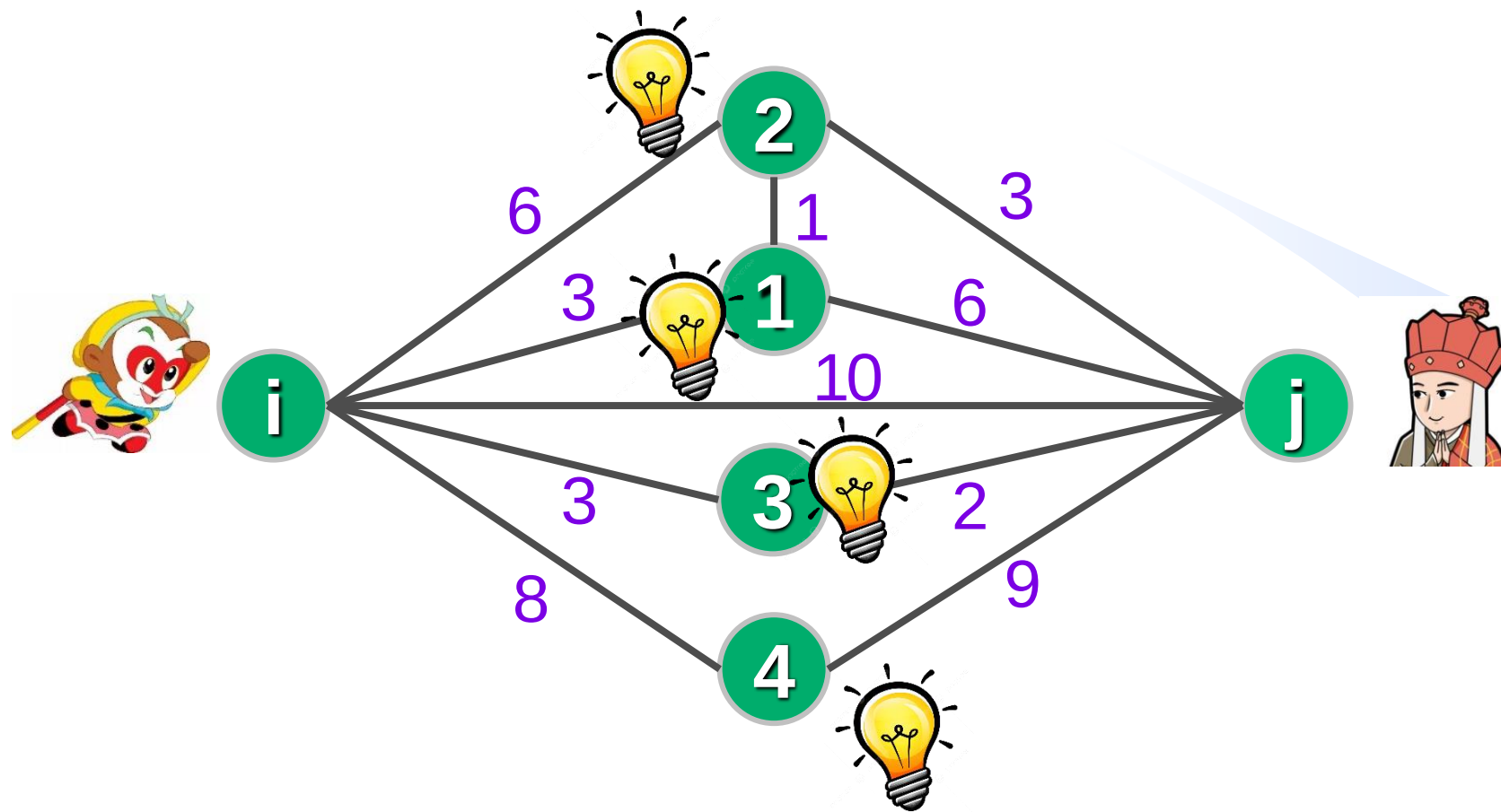
v_i

中间顶点

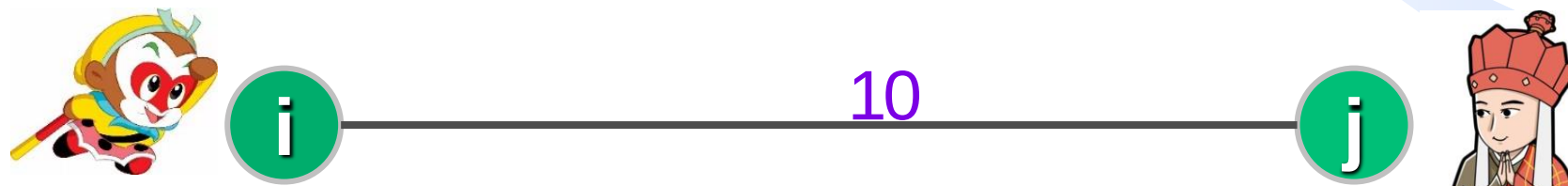
v_j



Floyd算法



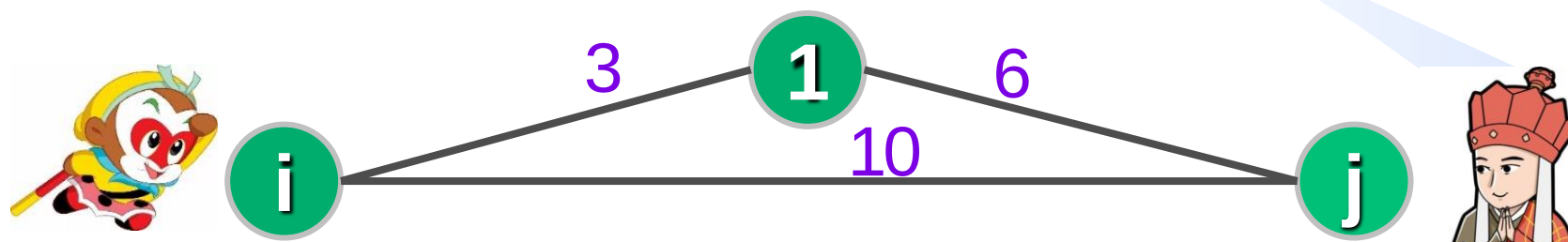
Floyd算法



顶点 i 经过顶点集合 $\{\emptyset\}$ 中的顶点到达 j 的最短距离

$$D^0(i, j) = G[i][j] = 10$$

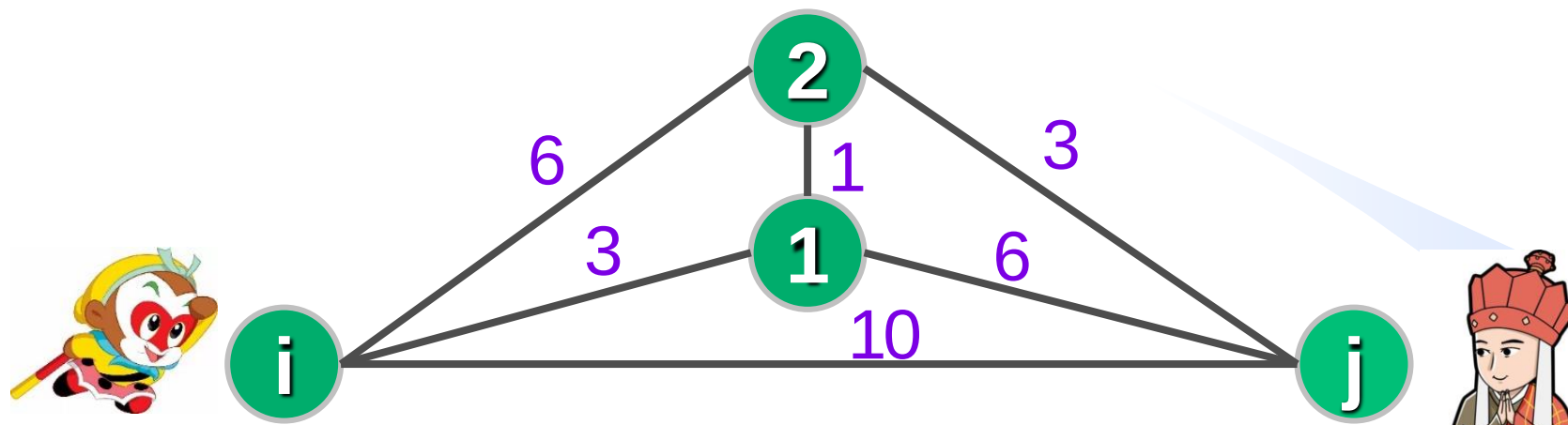
Floyd算法



顶点 i 经过顶点集合 $\{1\}$ 中的顶点到达 j 的最短距离

$$D^1(i, j) = \min(9, 10) = 9$$

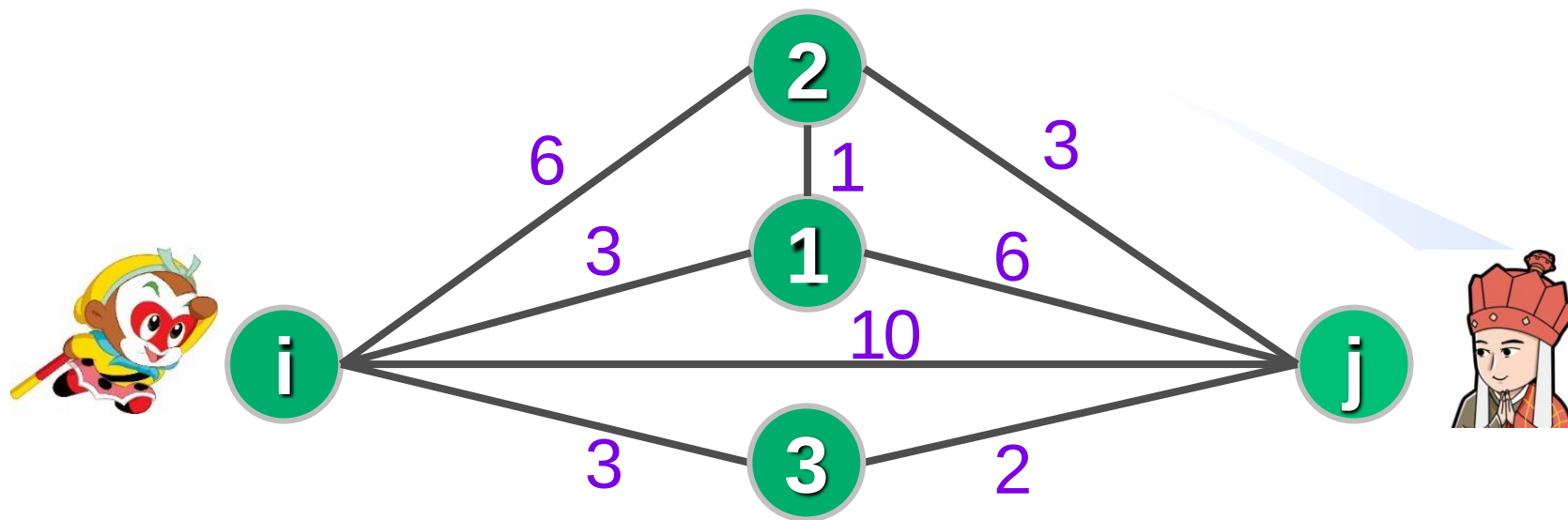
Floyd算法



顶点 i 经过顶点集合 $\{1, 2\}$ 中的顶点到达 j 的最短距离

$$D^2(i, j) = \min(9, 7) = 7$$

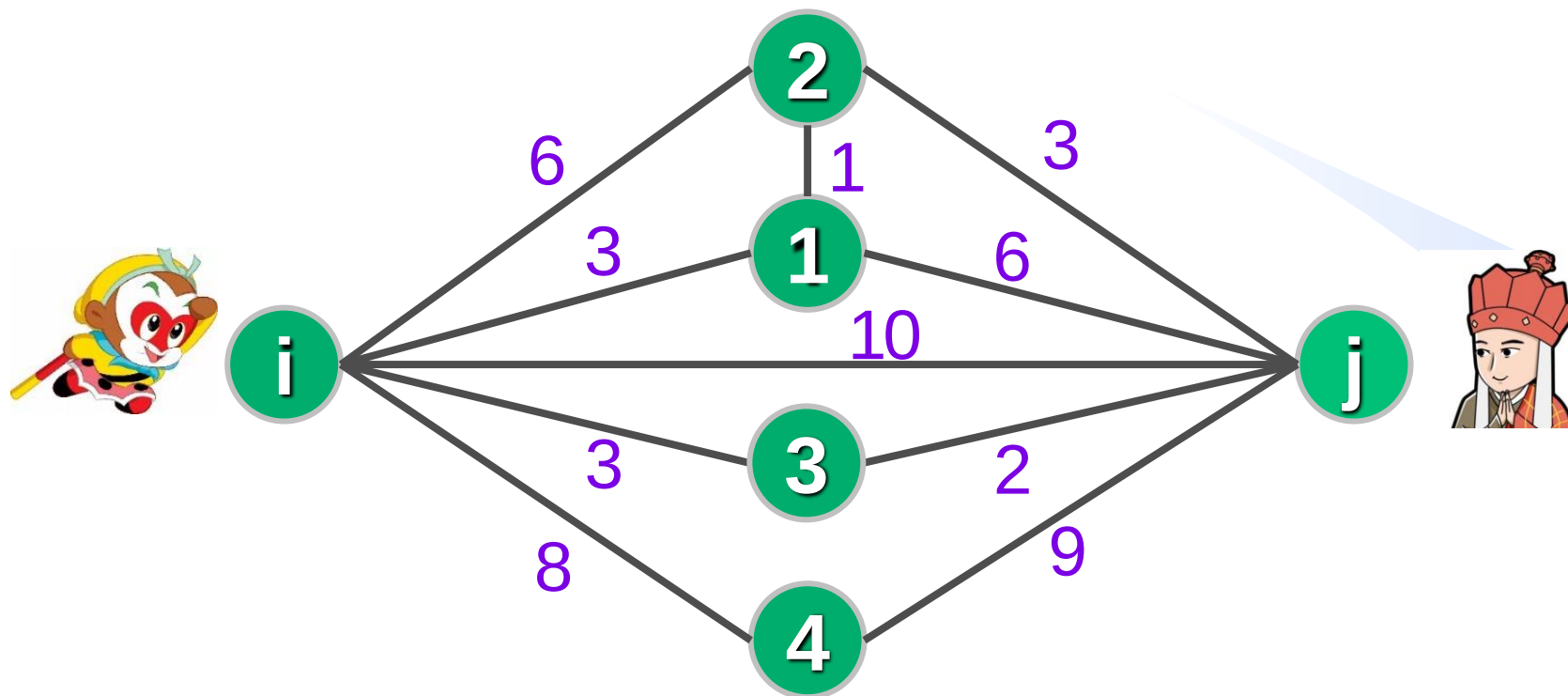
Floyd算法



顶点 i 经过顶点集合 $\{1, 2, 3\}$ 中的顶点到达 j 的最短距离

$$D^3(i, j) = \min(7, 5) = 5$$

Floyd算法



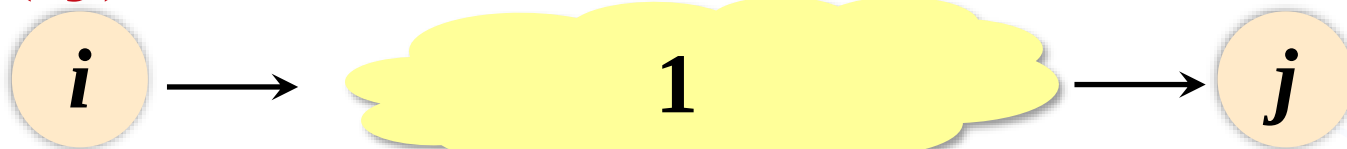
顶点 i 经过顶点集合 $\{1, 2, 3, 4\}$ 中的顶点到达 j 的最短距离

$$D^4(i, j) = \min(5, 17) = 5$$

Floyd算法

A

1. $D^1(i, j)$ 是顶点 i 经过顶点集合 $\{1\}$ 中的顶点到达顶点 j 的最短距离;



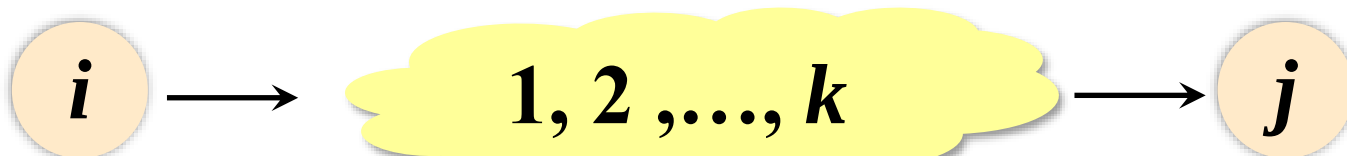
2. $D^2(i, j)$ 是顶点 i 经过顶点集合 $\{1, 2\}$ 中的顶点到达顶点 j 的最短距离;



$D^k[i][j]$: 顶点 i 至 j 在包含前 k 个顶点的子图上的最短距离

...

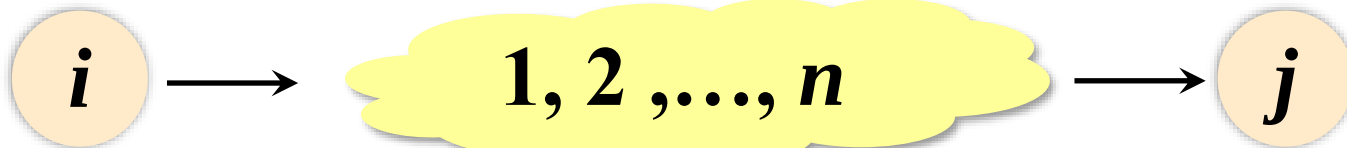
k . $D^k(i, j)$ 是顶点 i 经过集合 $\{1, \dots, k\}$ 中的顶点到达顶点 j 的最短距离;



$D^k[i][j]$: 顶点 i 至 j 的中间顶点序号小于等于 k 的最短距离

...

n . $D^n(i, j)$ 是顶点 i 经过集合 $\{1, \dots, n\}$ 中的点到达顶点 j 的最短距离;

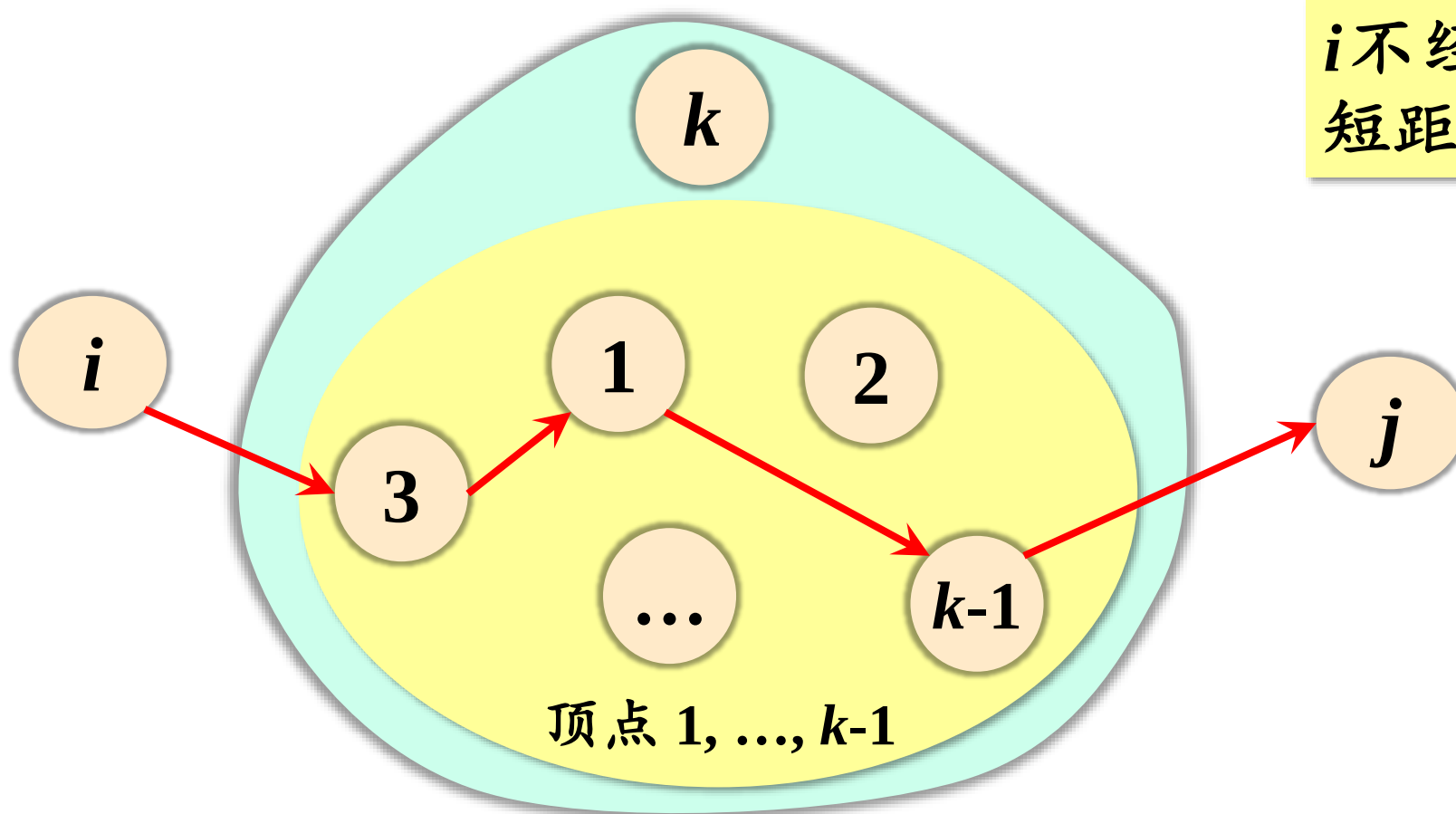


则 $D^n(i, j)$ 即为所求。

由 $D^{k-1}(i, j)$ 递推求出 $D^k(i, j)$

$D^{k-1}(i, j)$: 顶点 i 经过 $\{1, \dots, k-1\}$ 中的某些点到达顶点 j 的最短距离;

$D^k(i, j)$: 顶点 i 经过 $\{1, \dots, k-1, k\}$ 中的某些点到达顶点 j 的最短距离;



i 不经过点 k 到达 j 的最短距离是 $D^{k-1}(i, j)$

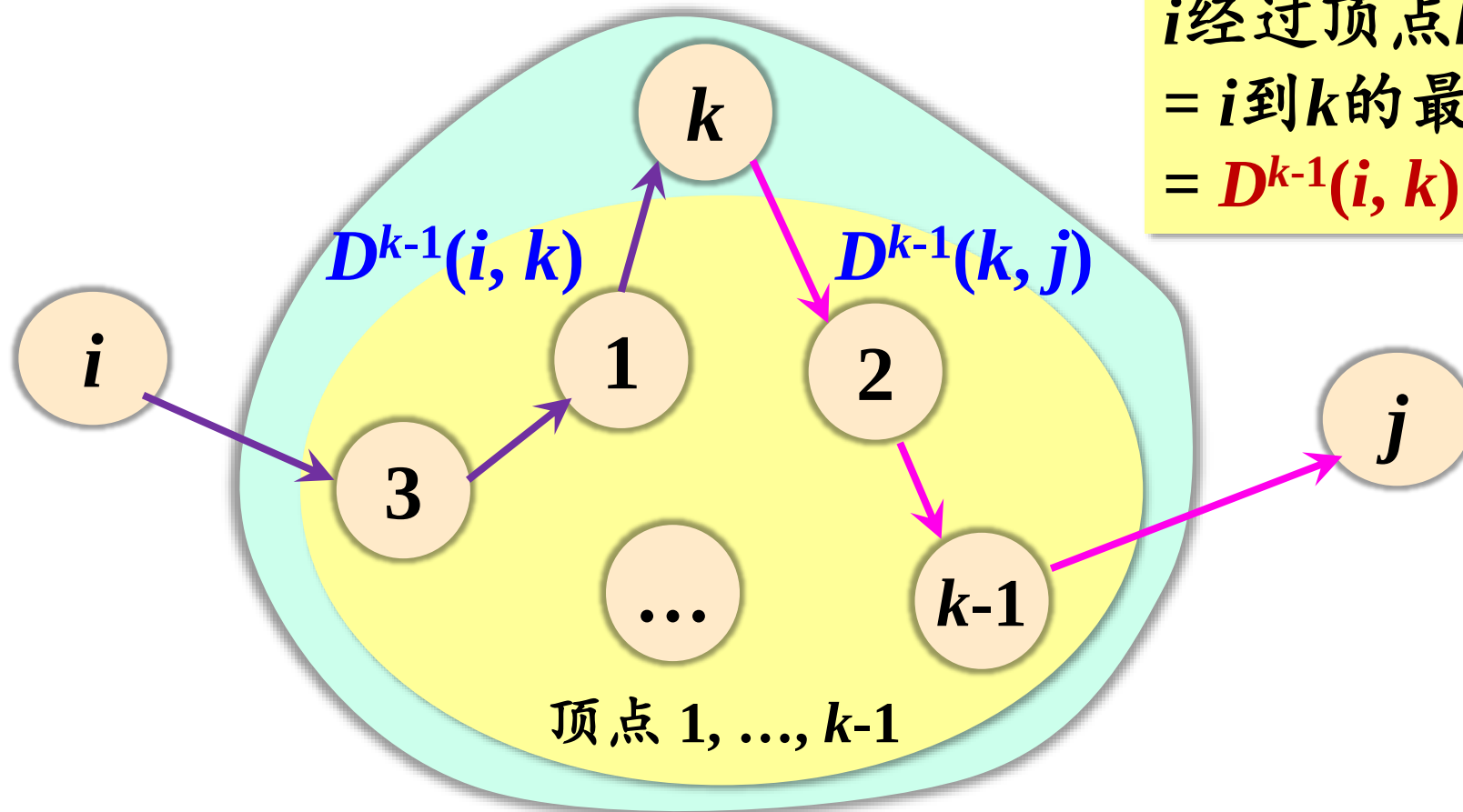
现在加上顶点 k 后, 最短距离会不会更新? 取决于顶点 i 经过 k 到 j 的最短距离会不会更短

由 $D^{k-1}(i, j)$ 递推求出 $D^k(i, j)$

$D^{k-1}(i, j)$: 顶点 i 经过 $\{1, \dots, k-1\}$ 中的某些点到达顶点 j 的最短距离;

$D^k(i, j)$: 顶点 i 经过 $\{1, \dots, k-1, k\}$ 中的某些点到达顶点 j 的最短距离;

i 经过顶点 k 到 j 的最短距离
= i 到 k 的最短距离 + k 到 j 的最短距离
= $D^{k-1}(i, k) + D^{k-1}(k, j)$

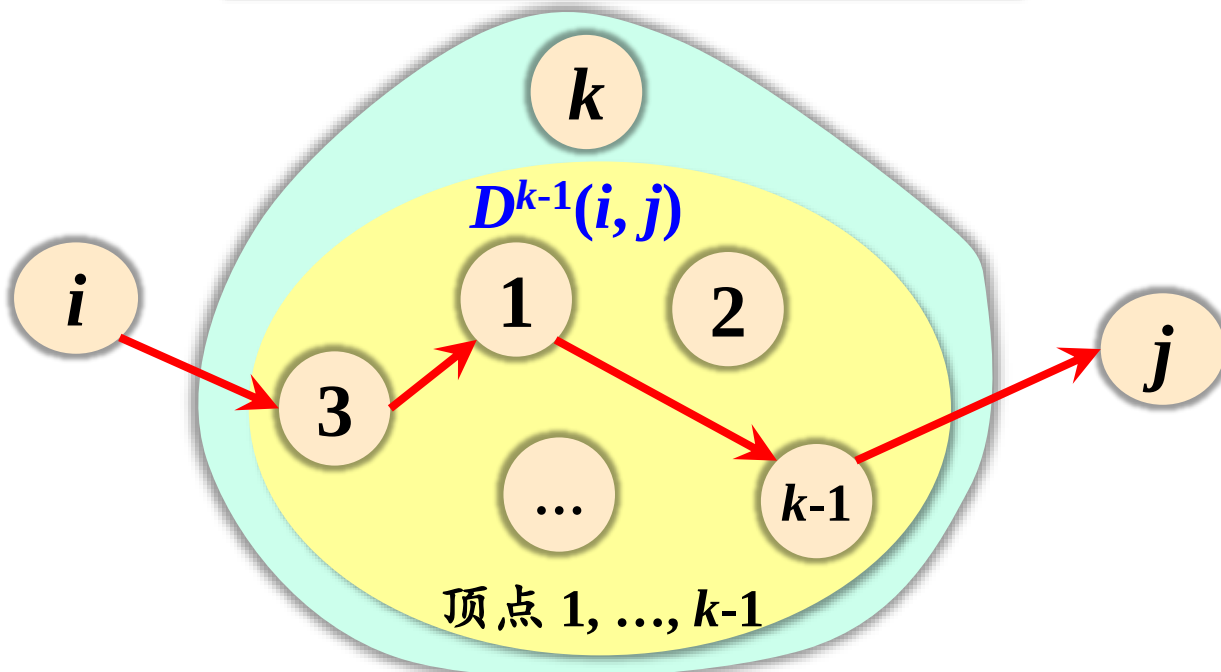


由 $D^{k-1}(i, j)$ 递推求出 $D^k(i, j)$

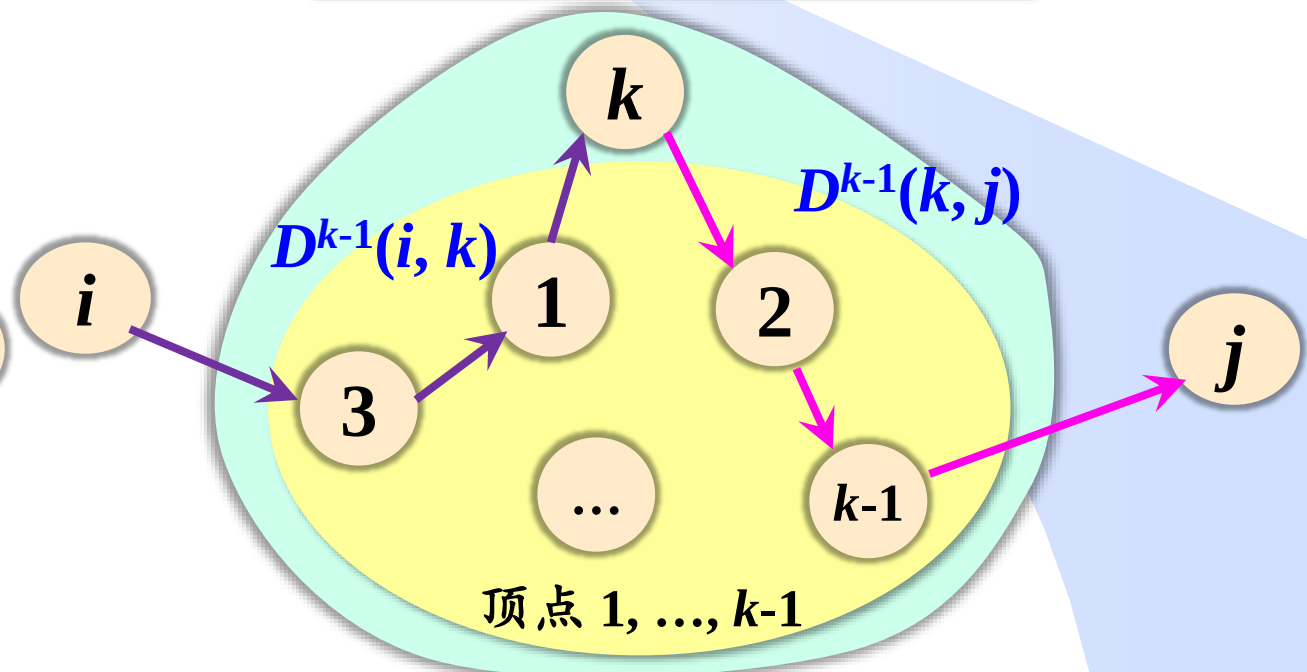
$D^{k-1}(i, j)$: 顶点 i 经过 $\{1, \dots, k-1\}$ 中的某些点到达顶点 j 的最短距离;

$D^k(i, j)$: 顶点 i 经过 $\{1, \dots, k-1, k\}$ 中的某些点到达顶点 j 的最短距离;

i 不经过点 k 到 j 的最短路



i 经过点 k 到 j 的最短路



$$D^k(i, j) = \min\{ D^{k-1}(i, j)$$

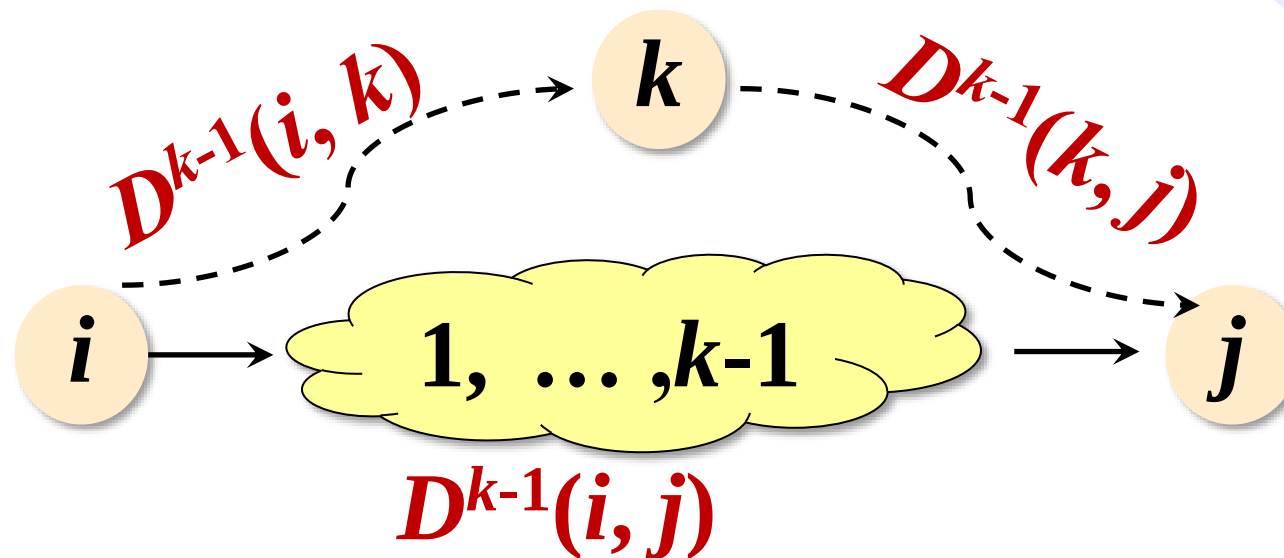
,

$$D^{k-1}(i, k) + D^{k-1}(k, j) \}$$

由 $D^{k-1}(i, j)$ 递推求出 $D^k(i, j)$

$D^{k-1}(i, j)$: 顶点 i 经过 $\{1, \dots, k-1\}$ 中的某些点到达顶点 j 的最短距离;

$D^k(i, j)$: 顶点 i 经过 $\{1, \dots, k-1, k\}$ 中的某些点到达顶点 j 的最短距离;



$$D^k(i, j) = \min\{ D^{k-1}(i, j), D^{k-1}(i, k) + D^{k-1}(k, j) \}$$

Floyd算法的实现——准备工作

➤ 算法输入:

✓ **G** : 图的邻接矩阵。

✓ **n** : 图的顶点个数。

➤ 算法结果保存在 **D** 和 **$path$** 两个二维数组里:

✓ **$D[i][j]$** : 保存 **$D(i, j)$** 的值, 即**顶点 i** 到**顶点 j** 的最短距离, 初值等于邻接矩阵。

✓ **$path[i][j]$** : i 到 j 最短路径上 i 的下一个顶点的编号。

Floyd算法的实现——准备工作

$$D^k[i][j] = \min\{D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j]\}$$

j

$$D^0 \rightarrow D^1 \rightarrow D^2 \rightarrow \dots D^{n-1} \rightarrow D^n$$

邻接矩阵 G

```
for(int k=1; k<=n; k++) //构造 $D^k$ 
    for(int i=1; i<=n; i++) //填写 $D^k[i][j]$ 
        for(int j=1; j<=n; j++)
            D[i][j]=min(D[i][j], D[i][k]+D[k][j]);
```

Floyd算法

A

```
#define N 400
```

```
void Floyd(int G[N][N], int n, int D[N][N], int path[N][N]){
```

```
    for(int i=1; i<=n; i++) //初始化, G为图的邻接矩阵
```

```
        for(int j=1; j<=n; j++) {
```

```
            D[i][j] = G[i][j];
```

```
            path[i][j] = (i==j || G[i][j]==INF)? -1 : j;
```

```
        }
```

```
    for(int k=1; k<=n; k++) //递推构造 $D^k$ 
```

```
        for(int i=1; i<=n; i++)
```

```
            for(int j=1; j<=n; j++)
```

```
                if( $D[i][k] + D[k][j] < D[i][j]$ ){
```

```
                     $D[i][j] = D[i][k] + D[k][j];$ 
```

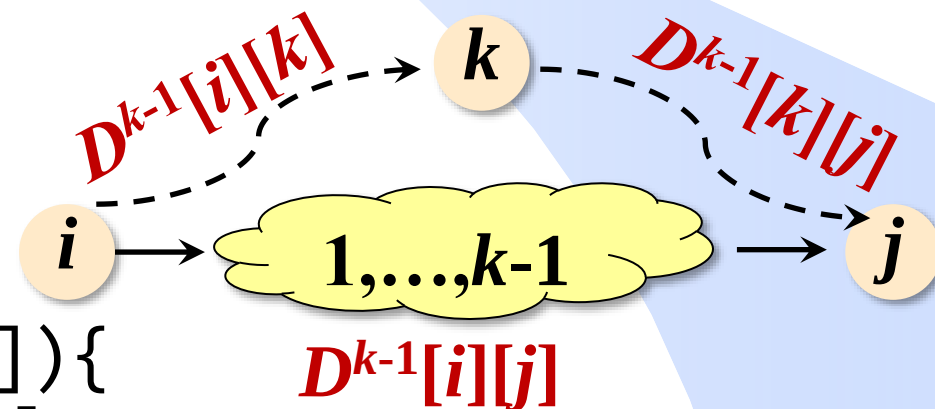
```
                    path[i][j] = path[i][k];
```

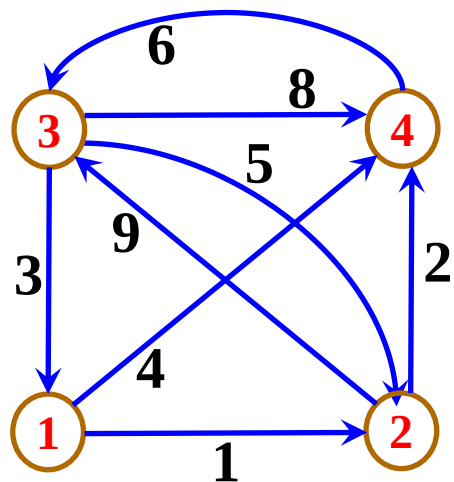
```
                }
```

```
    }
```

$O(n^3)$

$D^k[i][j] = \min\{D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j]\}$





	1	2	3	4
1	0	1	∞	4
2	∞	0	9	2
3	3	5	0	8
4	∞	∞	6	0

从 $D^{(4)}$ 知，顶点 2 到 1 的最短距离为 $D[2][1]=11$ ，其最短路径：

$path[2][1]=4$ ，表示顶点 2→顶点 4；

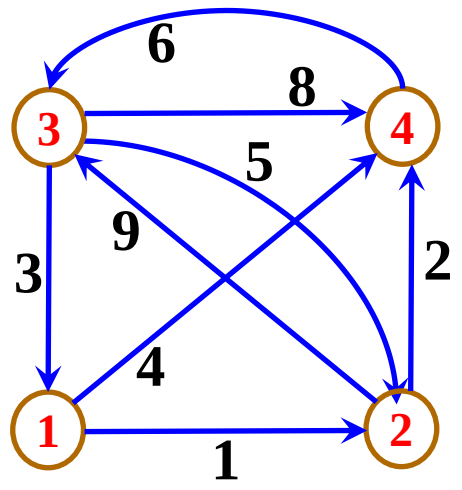
$path[4][1]=3$ ，表示顶点 4→顶点 3；

$path[3][1]=1$ ，表示顶点 3→顶点 1；

综上：2→4→3→1

	D^0				D^1				D^2				D^3				D^4			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	0	1	∞	4	0	1	∞	4	0	1	10	3	0	1	10	3	0	1	9	3
2	∞	0	9	2	∞	0	9	2	∞	0	9	2	12	0	9	2	11	0	8	2
3	3	5	0	8	3	4	0	7	3	4	0	6	3	4	0	6	3	4	0	6
4	∞	∞	6	0	∞	∞	6	0	∞	∞	6	0	9	10	6	0	9	10	6	0

	$path^0$				$path^1$				$path^2$				$path^3$				$path^4$			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	-1	2	-1	4	-1	2	-1	4	-1	2	2	2	-1	2	2	2	-1	2	2	2
2	-1	-1	3	4	-1	-1	3	4	-1	-1	3	4	3	-1	3	4	4	-1	4	4
3	1	2	-1	4	1	1	-1	1	1	1	-1	1	1	1	-1	1	1	1	-1	1
4	-1	-1	3	-1	-1	-1	3	-1	-1	-1	3	-1	3	3	3	-1	3	3	3	-1



	1	2	3	4
1	0	1	∞	4
2	∞	0	9	2
3	3	5	0	8
4	∞	∞	6	0

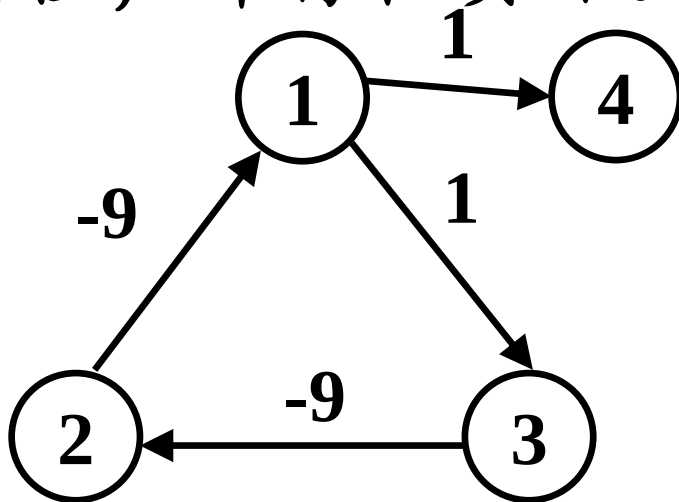
```
void PrintPath(int s, int t){
    printf("%d ", s);
    for(;s!=t; s=path[s][t])
        printf("%d ", path[s][t]);
}
```

	D^0				D^1				D^2				D^3				D^4			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	0	1	∞	4	0	1	∞	4	0	1	10	3	0	1	10	3	0	1	9	3
2	∞	0	9	2	∞	0	9	2	∞	0	9	2	12	0	9	2	11	0	8	2
3	3	5	0	8	3	4	0	7	3	4	0	6	3	4	0	6	3	4	0	6
4	∞	∞	6	0	∞	∞	6	0	∞	∞	6	0	9	10	6	0	9	10	6	0

	$path^0$				$path^1$				$path^2$				$path^3$				$path^4$			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	-1	2	-1	4	-1	2	-1	4	-1	2	2	2	-1	2	2	2	-1	2	2	2
2	-1	-1	3	4	-1	-1	3	4	-1	-1	3	4	3	-1	3	4	4	-1	4	4
3	1	2	-1	4	1	1	-1	1	1	1	-1	1	1	1	-1	1	1	1	-1	1
4	-1	-1	3	-1	-1	-1	3	-1	-1	-1	3	-1	3	3	3	-1	3	3	3	-1

Floyd算法总结

- Floyd算法的时间复杂度为 $O(n^3)$ ，与调用 n 次Dijkstra算法求每对顶点的最短路的时间基本相当，但Floyd算法允许负权边，形式简单、便于实现。
- 适用于有向图，也适用于无向图。
- 不允许存在负环（环上所有边的权值和为负数）。Floyd算法可以判负环，算法执行过程中若 $D[i][i] < 0$ ，表示顶点 i 到自己的最短距离为负值，即存在负环。



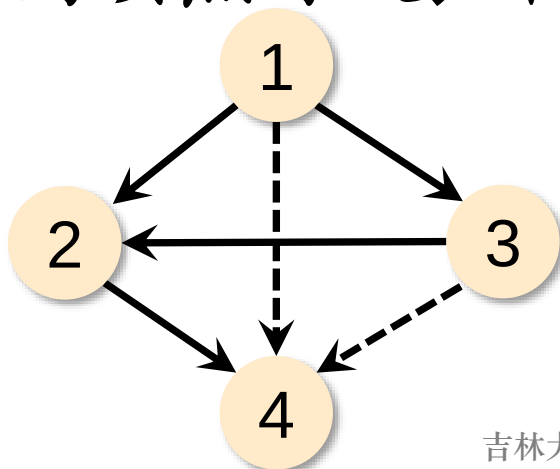
任意两点间最短路径问题研究进展

时间复杂度	提出者	时间
$O(n^3)$	Floyd	1962
$O(n^3/\log^{1/3}n)$	Fredman	1975
$O(n^3/\log^{1/2}n)$	Dobosiewicz	1990
$O(n^3/\log^{5/7}n)$	Han	2004
$O(n^3/\log n)$	Takaoka	2005
$O(n^3/\log^{5/4}n)$	Han	2006
$O(n^3/\log^2n)$	Chan/Han-Takaoka	2007/2012
$O(n^3/2^{\Omega(\sqrt{\log n})})$	Williams/Chan	2014/2020

T Chan, R Williams. Deterministic APSP, Orthogonal Vectors, and More: Quickly Derandomizing Razborov-Smolensky. *ACM Transactions on Algorithms*, 17(1), Article 2, 2020.

有向图的传递闭包问题

- 不关注路径长度，而仅关注是否存在路径。
- v_i 和 v_j 是有向图 G 的两个顶点，若从 v_i 到 v_j 存在一条路径，则称 v_i 到 v_j 可达；
- 传递性：若 v_i 到 v_j ， v_j 到 v_k 可达，则 v_i 到 v_k 可达。
- 描述有向图顶点间可达关系的 n 阶方阵 R 称为可达矩阵：若顶点 v_i 到 v_j 可达，则 $R_{ij}=1$ ，否则 $R_{ij}=0$ 。
- 传递闭包：由有向图 G 的顶点集 V 、边集 E ，以及新添加的虚边（表示两顶点可达）构成的图被称为 G 的传递闭包。



$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

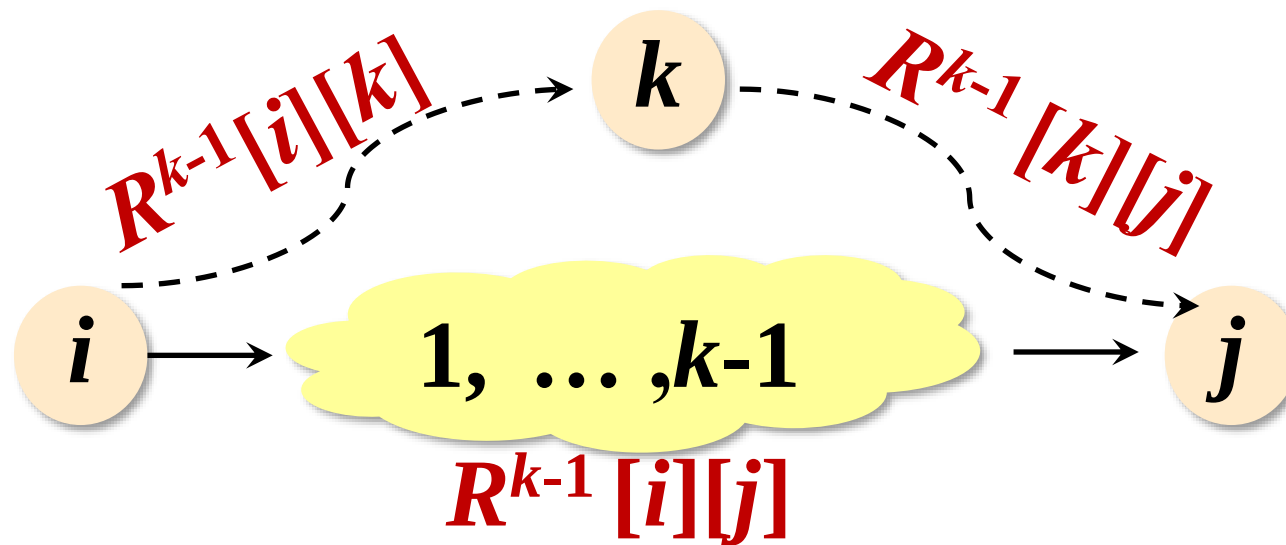
可达矩阵

由 R^{k-1} 递推求出 R^k

$R^{k-1}(i, j)$: 顶点 i 经过 $\{1, \dots, k-1\}$ 中的某些顶点到达 j 的可及性。

$R^k(i, j)$: 顶点 i 经过 $\{1, \dots, k-1, k\}$ 中的某些顶点到达 j 的可及性。

$$R^k[i][j] = R^{k-1}[i][j] \text{ OR } (R^{k-1}[i][k] \text{ AND } R^{k-1}[k][j])$$



Warshall算法

```
void Warshall(int G[N][N], int n, bool R[N][N]) {
    for (int i=1; i<=n; i++) //假定G是无权图
        for (int j=1; j<=n; j++)
            R[i][j] = G[i][j];
    for (int k=1; k<=n; k++) //递推构造 $R^k$ 
        for (int i=1; i<=n; i++)
            for (int j=1; j<=n; j++)
                R[i][j] = R[i][j] || (R[i][k] && R[k][j]);
}
```

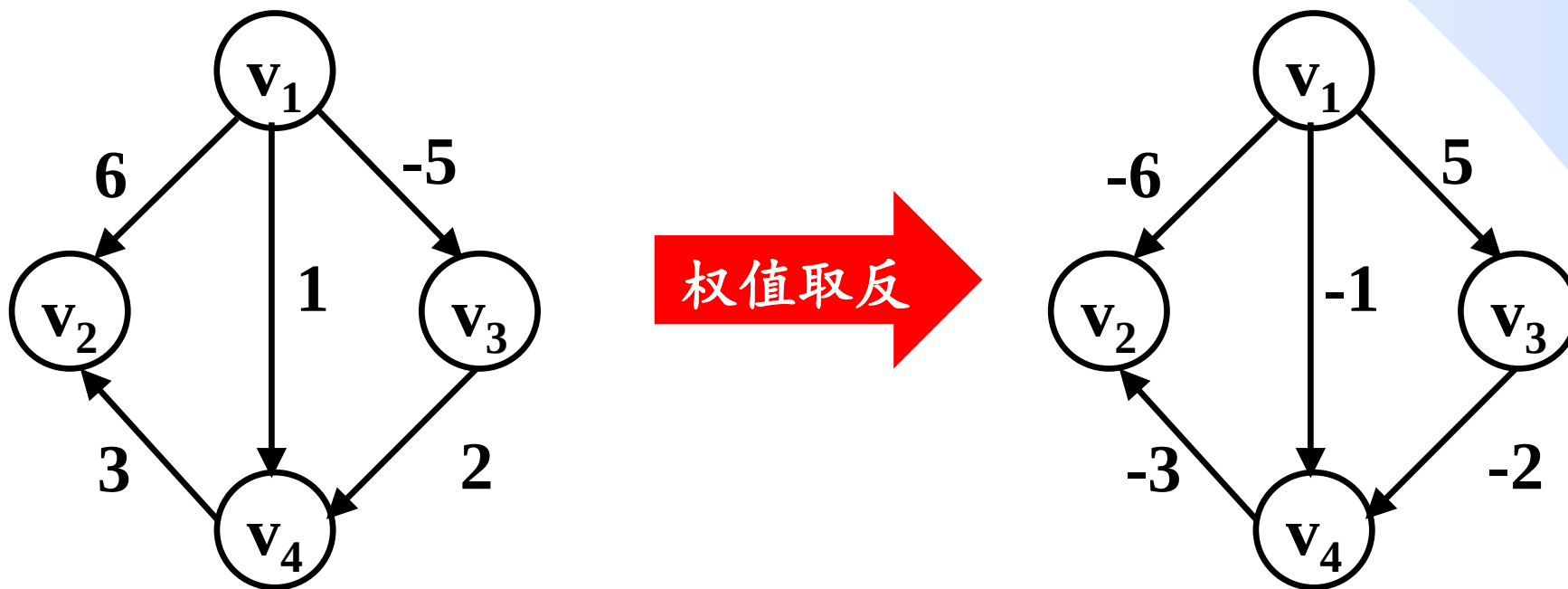
若G为带权图则此处为
 if($i \neq j$ && $G[i][j] < INF$)
 $R[i][j] = \text{true};$
 else $R[i][j] = \text{false};$

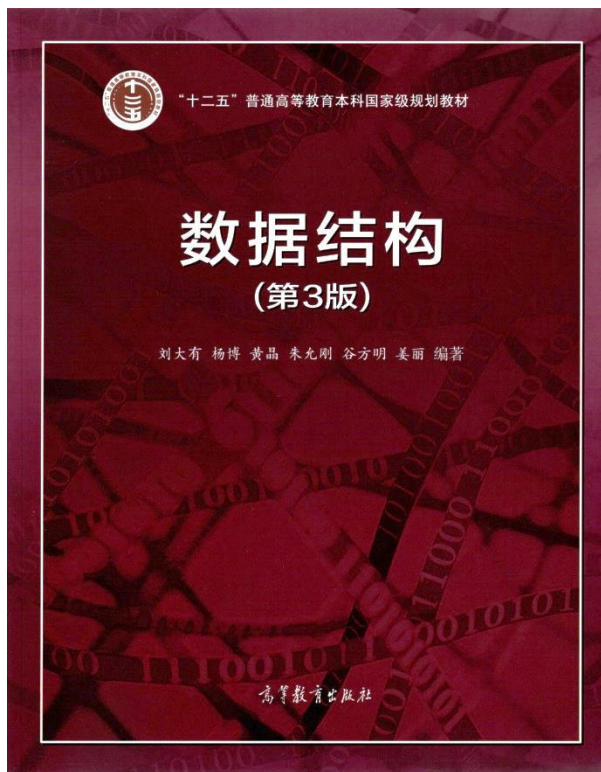
$$R^k[i][j] = R^{k-1}[i][j] \text{ OR } (R^{k-1}[i][k] \text{ AND } R^{k-1}[k][j])$$

时间复杂度 $O(n^3)$

课下思考

求图的最长路径，可否采用如下方案：若图中不存在正环，对所有边的权值取相反数，执行Floyd算法求最短路径，该路径就是原图的最长路。





图的最短路径

- 无权图最短路径
- Dijkstra算法
- A*算法
- Floyd算法
- **满足约束的最短路径**

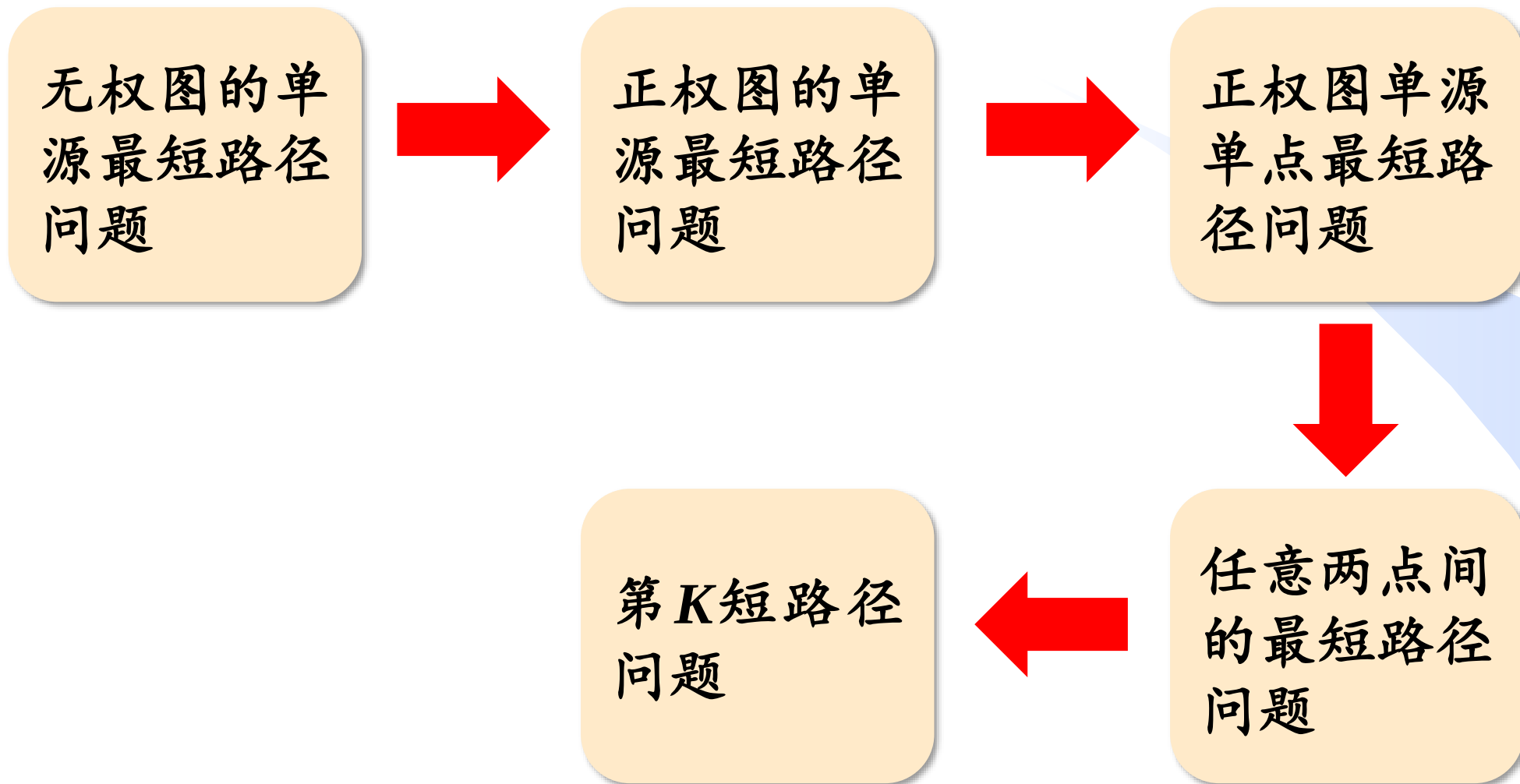
数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

第 k 短路径



最短路径问题



第 k 短路径

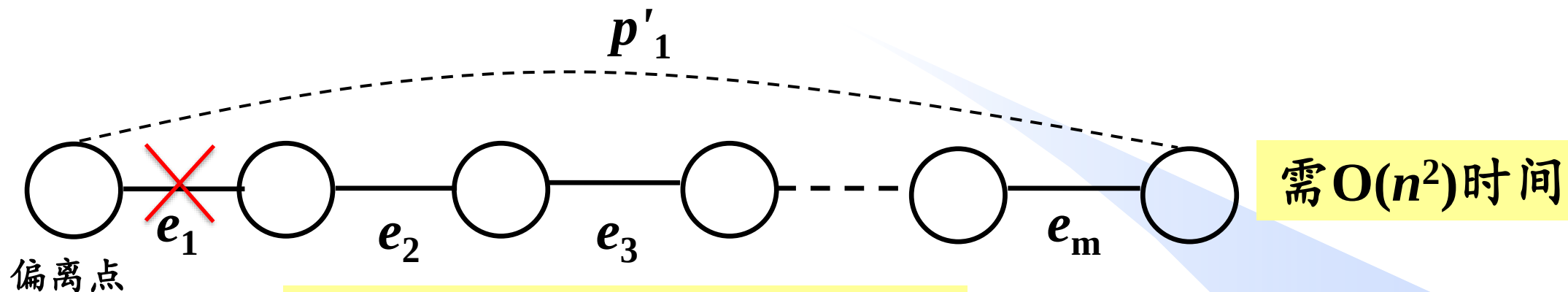
假设 $G = (V, E)$ 是一个包含 n 个顶点的有向图。 s 是初始顶点， t 是目标顶点，每条边的权值都为正，要求给出 s 到 t 的最短路径、第2短路径、第3短路径、.....、第 k 短路径。

Yen 算法

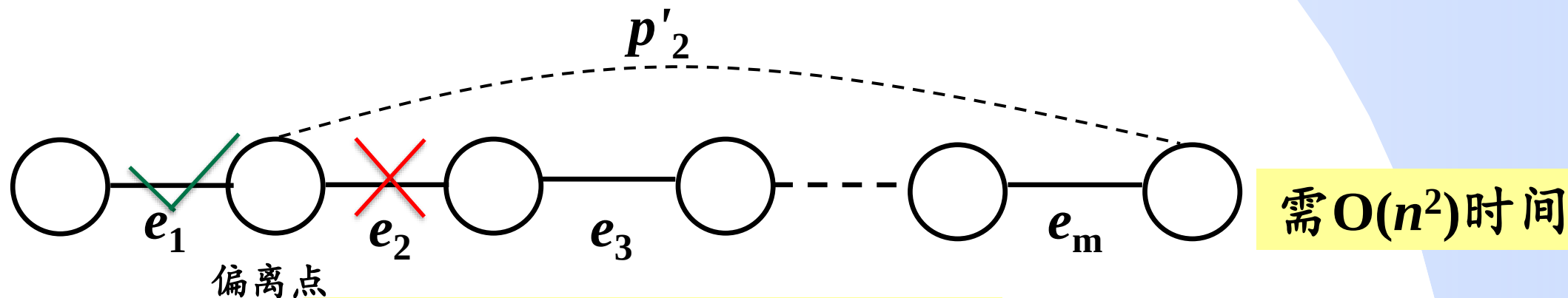
B

(1) 先求得 s 到 t 的最短路径, 假设其经过的边为 $e_1, e_2, \dots, e_{m-2}, e_{m-1}, e_m$;

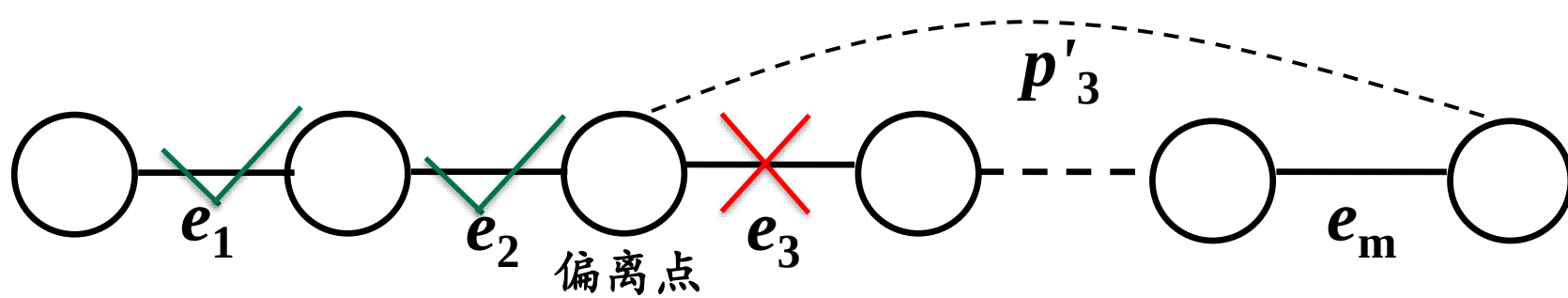
(2)



$p_1 = p'_1$ 送入候选集合



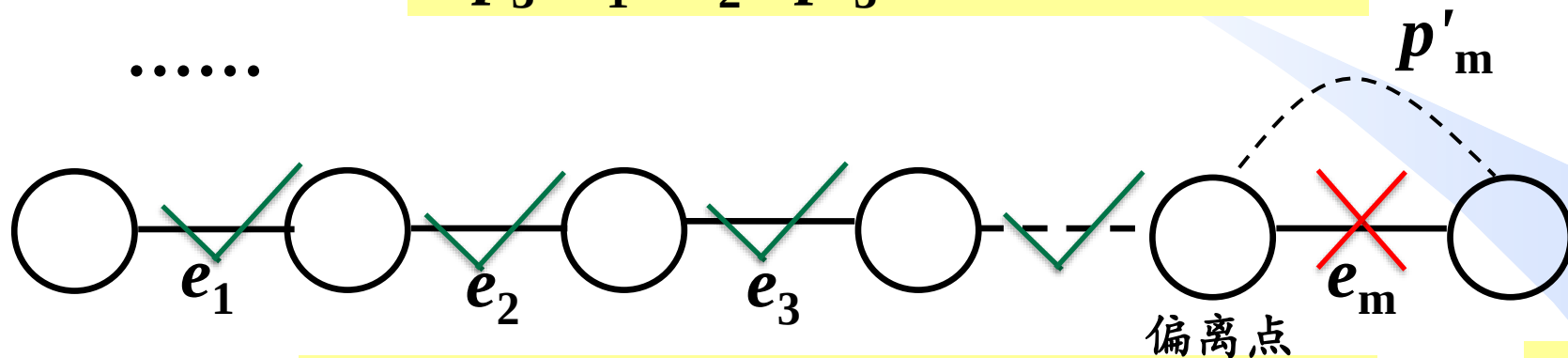
$p_2 = e_1 + p'_2$ 送入候选集合



需 $O(n^2)$ 时间

$p_3 = e_1 + e_2 + p'_3$ 送入候选集合

.....



需 $O(n^2)$ 时间

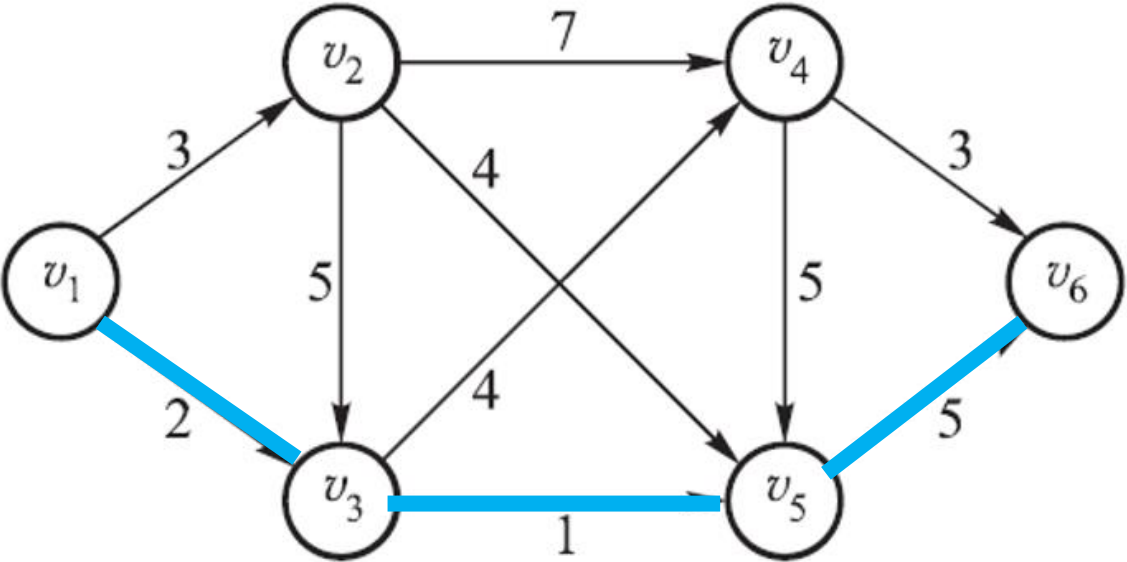
$p_m = e_1 + \dots + e_{m-1} + p'_m$ 送入候选集合

总时间复杂度
 $O(kn^3)$

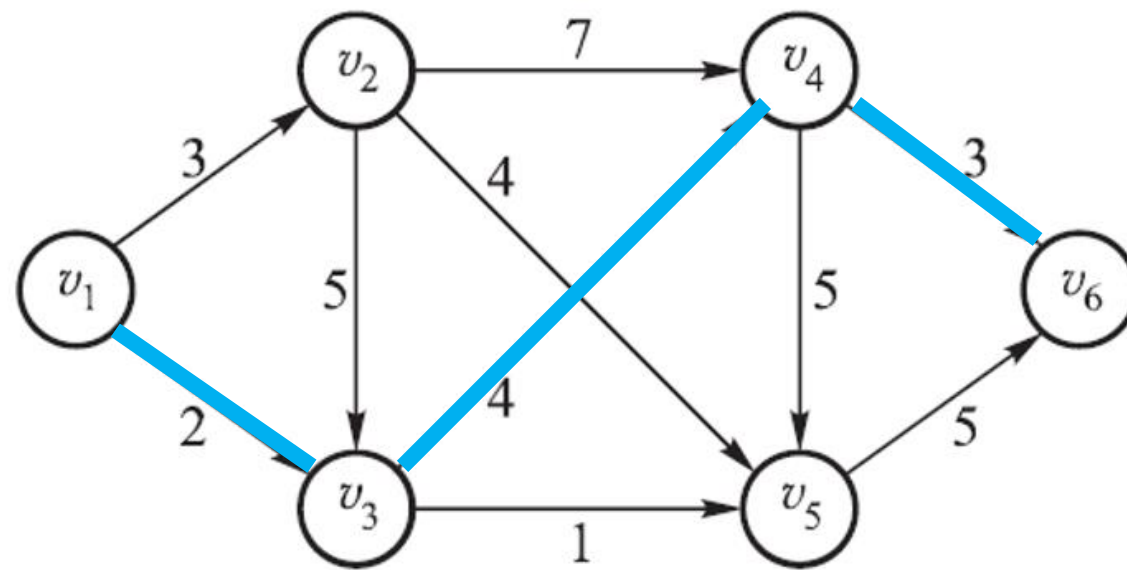
(3) 候选集合中选出最短路径即为第2短路径。

(4) 对第2短路径重复上述操作(2)-(3)，直至求出第k短路径。

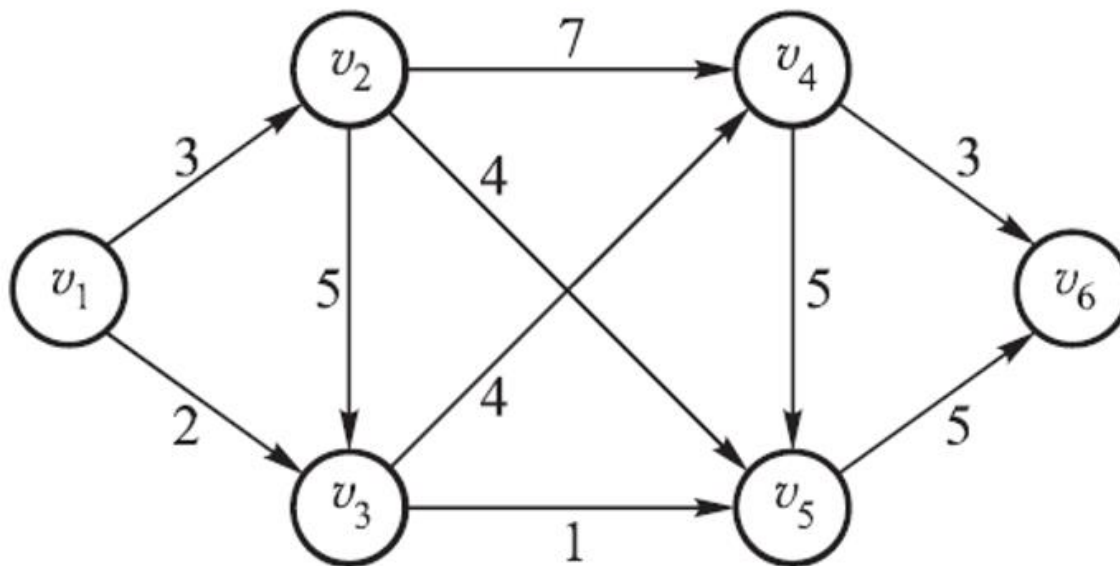
- 防止路径中有环：从偏离点 v 到 t 的最短路径不能包含 s 到 v 的顶点
- 避免与已求路径重复：从偏离点 v 出发的边不能与已求出的路径中 v 出发的边相同



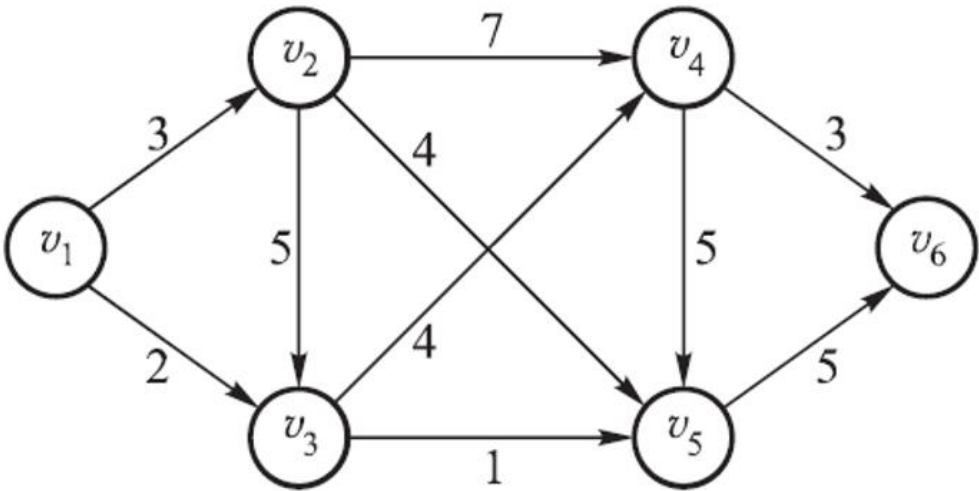
最短路径	路径长度	包含的边	不包含的边	候选路径
$v_1v_3v_5v_6$	8			
			v_1v_3	$v_1v_2v_5v_6=12$
		v_1v_3	v_3v_5	$v_1v_3v_4v_6=9$
		v_1v_3, v_3v_5	v_5v_6	∞



最短路径	路径长度	包含的边	不包含的边	候选路径
$v_1v_3v_5v_6$	8			
			v_1v_3	$v_1v_2v_5v_6=12$
		v_1v_3	v_3v_5	$v_1v_3v_4v_6=9$
		v_1v_3, v_3v_5	v_5v_6	∞
$v_1v_3v_4v_6$	9		v_1v_3	$v_1v_2v_5v_6=12$
		v_1v_3	v_3v_4, v_3v_5	∞
		v_1v_3, v_3v_4	v_4v_6	$v_1v_3v_4v_5v_6=16$
$v_1v_2v_5v_6$	12		v_1v_2, v_1v_3	∞
		v_1v_2	v_2v_5	$v_1v_2v_4v_6=13$
		v_1v_2, v_2v_5	v_5v_6	∞



最短路径	路径长度	包含的边	不包含的边	候选路径
$v_1v_2v_4v_6$	13		v_1v_2, v_1v_3	∞
		v_1v_2	v_2v_4, v_2v_5	$v_1v_2v_3v_5v_6=14$
		v_1v_2, v_2v_4	v_4v_6	$v_1v_2v_4v_5v_6=20$
$v_1v_2v_3v_5v_6$	14		v_1v_2, v_1v_3	∞
		v_1v_2	v_2v_3, v_2v_4, v_2v_5	∞
		v_1v_2, v_2v_3	v_3v_5	$v_1v_2v_3v_4v_6=15$
		v_1v_2, v_2v_3, v_3v_5	v_5v_6	∞
$v_1v_2v_3v_4v_6$	15		v_1v_2, v_1v_3	∞
		v_1v_2	v_2v_3, v_2v_4, v_2v_5	∞
		v_1v_2, v_2v_3	v_3v_4, v_3v_5	∞
		v_1v_2, v_2v_3, v_3v_4	v_4v_6	$v_1v_2v_3v_4v_5v_6=22$



最短路径	路径长度	包含的边	不包含的边	候选路径
$v_1v_3v_4v_5v_6$	16		v_1v_2, v_1v_3	∞
		v_1v_3	v_3v_4, v_3v_5	∞
		v_1v_3, v_3v_4	v_4v_5, v_4v_6	∞
		v_1v_3, v_3v_4, v_4v_5	v_5v_6	∞
$v_1v_2v_4v_5v_6$	20		v_1v_2, v_1v_3	∞
		v_1v_2	v_2v_3, v_2v_4, v_2v_5	∞
		v_1v_2, v_2v_4	v_4v_5, v_4v_6	∞
		v_1v_2, v_2v_4, v_4v_5	v_5v_6	∞
$v_1v_2v_3v_4v_5v_6$	22		v_1v_2, v_1v_3	∞
		v_1v_2	v_2v_3, v_2v_4, v_2v_5	∞
		v_1v_2, v_2v_3	v_3v_4, v_3v_5	∞
		v_1v_2, v_2v_3, v_3v_4	v_4v_5, v_4v_6	∞
		$v_1v_2, v_2v_3, v_3v_4, v_4v_5$	v_5v_6	∞

方案2：修改Dijkstra算法

优先级队列

初始时 (u 为源点), 将 $(u, 0)$ 放入队列 Q

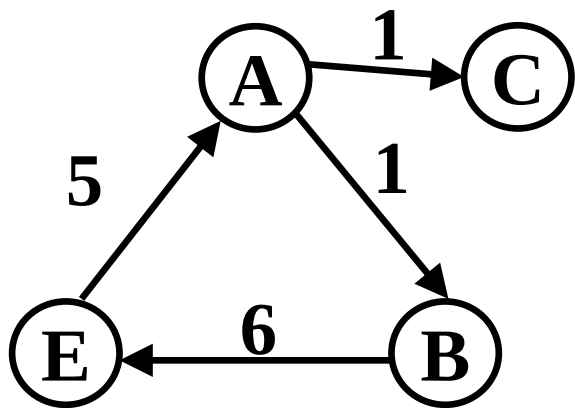
①从 Q 中出队 D_v 最小元素的 (v, D_v) 。

②依次考察 v 的邻接顶点 w , 更新 D_w 的值, 使
 $D_w \leftarrow D_v + \text{weight}(v, w)$. 将 (w, D_w) 入队 (队
 列中可能包含顶点 w 对应的多个二元组)。

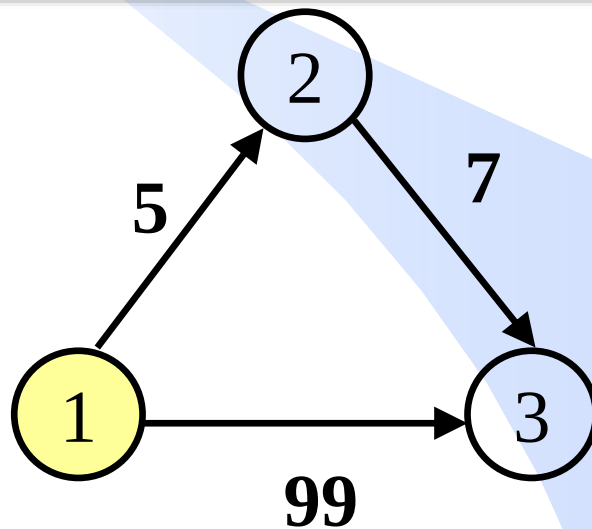
③重复① ②, 直至目标点 t 第 k 次出队。

顶点 v 第 k 次出队时, D_v 就是 u 到 v 的第 k 短距离

找到的第 k 短
路可能存在环

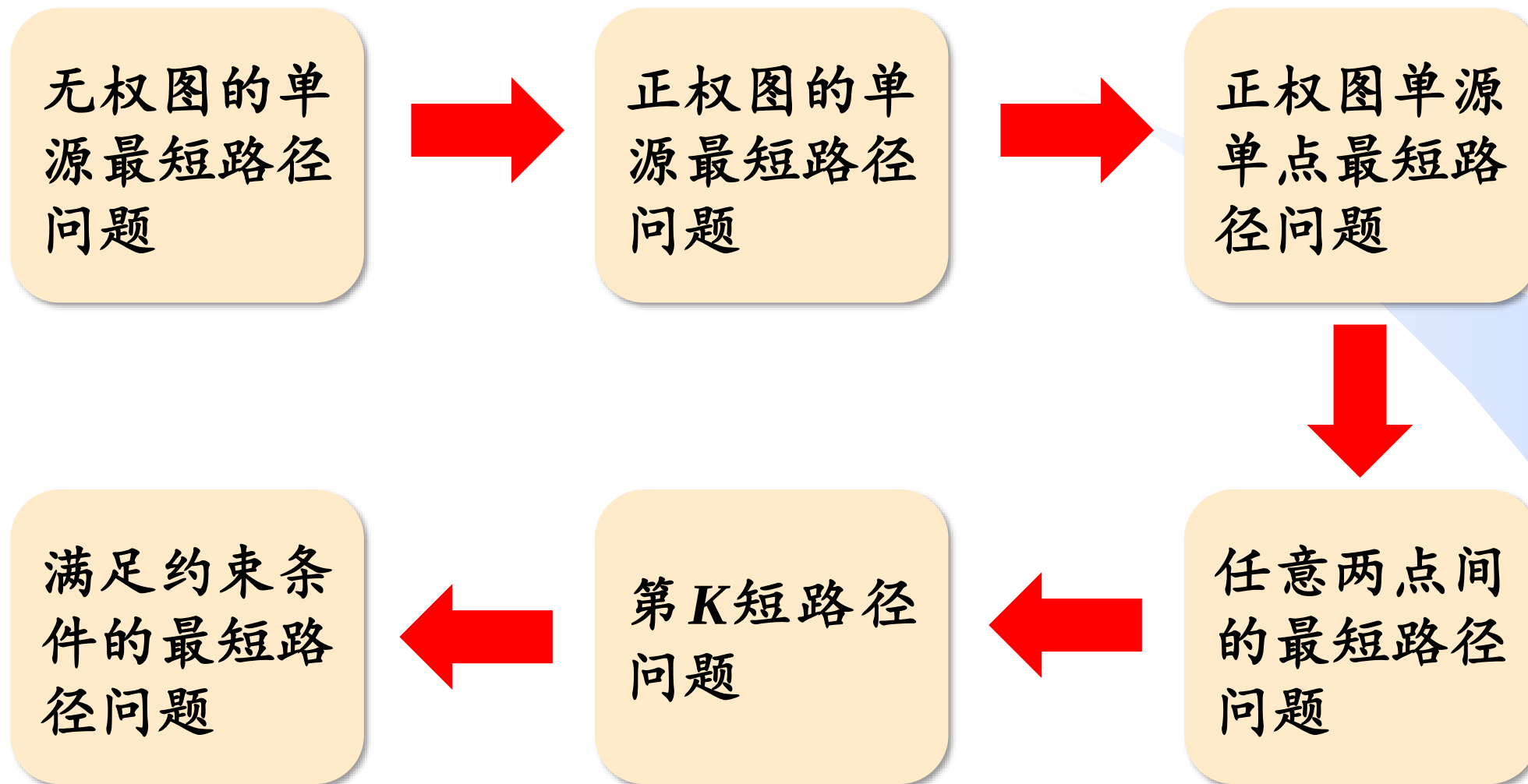


维护一个队列 Q , 队列中存储二元组 (v, D_v) 。出队时并不是队头元素队列, 而是 D 值最小的元素出队。



$(1, 0)$	$(2, 5)$	$(3, 12)$
	$(3, 99)$	$(3, 99)$

最短路径问题



满足约束的最短路径

- 某些问题需要找在某些**约束条件**下的最短路径。
- 约束条件：必须包含某些点/某些边，或必须不包含某些点/某些边等。
- 一种可行的方案：
 - ✓ 依次生成两个顶点间的第1、2、3、...、 k 短路径；
 - ✓ 然后逐一测试每条路径是否满足给定的约束条件；
 - ✓ 第1条满足约束条件的路径即为所求。

满足约束的最短路径

